

AI Agent 零基础工作方法教程

从会聊天到会干活：
给技术新手的完整工作手册

适用读者：技术新手 / 本科生 / 职场新人

版本 v1.0

2026-09-18

前言

这本书教什么

这本书教你一件事：怎么把 AI 从“会聊天”变成“会干活”。

你将学会：判断什么任务适合交给 Agent、怎么写一份让 Agent 能执行的任务说明、怎么验证它真的做完了、失败了怎么排查、怎么把跑通的流程沉淀成自己的工具。全书 11 个 Part，每个 Part 都有可以直接上手的 Mission（实操任务）。

这本书不教什么

这本书不教你从零训练一个大模型，不教你写复杂的算法代码，不教你成为 AI 工程师。它的目标读者是：会用电脑、会上网、但没写过代码或只写过一点代码的人。

如果你已经是资深开发者，这本书可以帮你快速建立“Agent 工作方法论”的框架感，但底层原理的深度你需要自己找别的书补。

如何使用本书

建议按顺序读。Part 0 到 Part 3 是基础，Part 4 到 Part 5 是验证与失败处理，Part 6 到 Part 7 是 Agent 的组成与扩展，Part 8 到 Part 10 是场景化的实战练习。

每个 Part 开头有“先导图”帮你建立全局印象，中间有“Mission”任务让你动手做，末尾有“本章模板”让你把成果存下来。边读边做，不要只看不练。

书末附了术语表、资源地图和 8 要素任务法模板，可以随时查阅。

F00 全书总模型：八阶段任务主线

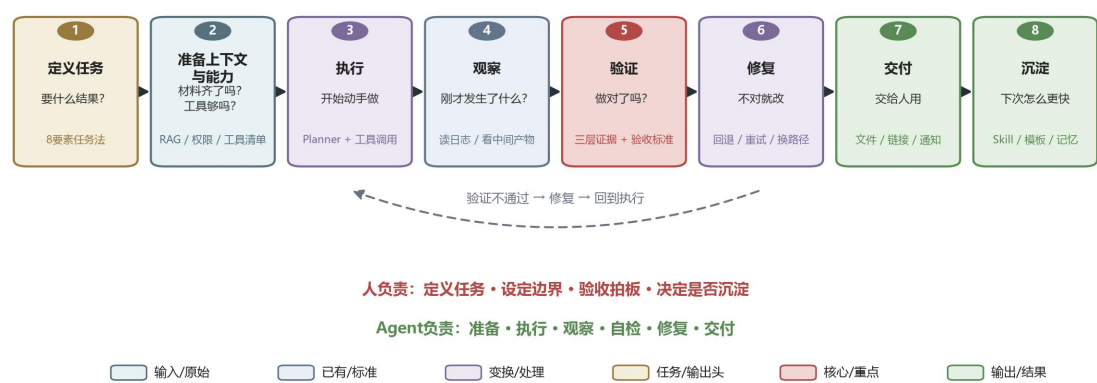


图 1 全书统一认知框架：Goal → Context → Resources → Capabilities → Workflow → Verification → Reuse

目 录

0.1 本章要回答的只有一个问题	14
0.2 用三个维度看清你处在哪一步	15
0.3 那些标签只是这三个维度的不同组合	16
0.4 什么事适合交、什么事不能放手	17
0.5 四条铁律从这里开始钉	18
0.6 Mission L0：一次有目的的对话	18
PART 1：第一次让 Agent 进入你的工作环境	20
1.1 本章只解决一件事	20
1.2 工作区：先把桌子摆好	20
工作区设置完整步骤	21
1.3 原件与副本：本章最重要的一条规矩	22
1.4 它怎么看到文件	22
1.5 四级权限：你允许它在你家里干什么	22
1.6 沙盒实测：四级权限各试一次	23
1.7 Mission 1：整理一个混乱文件夹（完整走查）	24
1.8 Mission 2：读多份 PDF 做摘要表	25
1.9 Mission 3：联网查资料并把脚印带回来	26
PART 2：把事情说清楚——8 要素任务法	28
2.1 为什么“帮我做一下”总会跑偏	28
2.2 8 要素定稿	28
2.3 目标：把“想要”翻译成“可检查的结果”	29
2.4 上下文：弹药、背景、当前状态一起给	29
2.5 环境：圈定活动范围	30
2.6 权限：分级授权，写出来	30
2.7 约束：把红线写出来	31

2.8 过程：按什么顺序、哪里停下来	31
2.9 验收：怎么证明正确	32
2.10 交付：最后必须得到哪些文件	32
2.11 六类任务速查（可直接改）	33
2.12 好任务与坏任务对照	35
2.12.1 十二组好坏任务逐条解析	36
2.13 三个 Mission 的完整 8 要素表	39
2.14 从这一章长出两条主线	41
PART 3：状态管理——让长任务不会断片	42
3.1 本章要解决的新问题	42
3.2 为什么不能一口气做完	43
3.3 最小配置：三份就够	43
3.4 复杂项目再追加三份	45
3.5 不要为了项目管理而项目管理	45
3.6 计划与停顿：先让它列，你再改	45
3.7 中断了怎么续：WORKLOG 与 TASKS 是断点	46
3.8 中途改需求、加新材料	46
3.9 长任务管理的真实工作流：以“读 30 篇文献写综述”为例	47
3.9.1 五天日志实录：每天到底发生了什么	48
3.10 Mission：跑一条两阶段长任务	49
4.1 “完成声明”不是完成证据	51
四条铁律在本 Part 的具体化	51
4.2 三类证据怎么留痕	52
执行证据：操作流水账	52
结果证据：东西要落盘	52
验证证据：用独立手段证明它对了	52

4.3 四类任务，四种验证方法	53
4.3.1 来源型任务：论文 / 网页 / 政策 / 公司信息	53
4.3.2 数值型任务：Excel / 数据分析 / 统计	54
4.3.3 操作型任务：文件整理 / Office / 浏览器操作	55
4.3.4 软件型任务：代码 / 应用 / 网页	57
4.4 验证强度三级：自测 → 交叉核验 → 独立审查	58
第一级：对抗式自测	58
第二级：交叉核验	59
第三级：独立审查（换模型）	59
4.5 验收闭环：要求 → 证据 → 结果 → 问题 → 修复 → 复验	60
验收清单（可直接复制）	61
验收实验（P4 实操）	61
4.6 故障注入实验：亲手设局，看它怎么露馅	62
实验一：假引用——喂它一篇不存在的论文	62
实验二：有病的表——喂它一张埋了错的 Excel	62
实验三：塞无关文件——在目录里放一张壁纸	62
实验四：死链——喂它一个必然 404 的 URL	63
实验五：假测试——让它“跑测试”	63
做完之后	63
PART 5：故障诊断手册——Agent 会怎么失败	64
5.1 信息错误：幻觉、假引用、假网页、假数据	64
5.2 上下文错误：信息不足、污染、错版本、错文件	66
5.3 执行错误：没运行、没测试、依赖失败、网络失败	67
5.4 状态错误：遗忘、重复劳动、中断	69
5.5 修改错误：覆盖、删除、Regression	70
5.6 资源错误：Token、Cost、Rate Limit、无限循环	72

5.7 急救工具箱：Backup / Rollback / Retry / Resume	74
急救 workflow	75
破坏性操作前的安全检查	75
急救实验（P5 实操）	76
本章你要解决什么问题	77
一张总览图：不是五层，是四个系统加一个接口层	77
① 推理系统 Model：谁在思考和决策	79
6.1 是什么	79
6.2 为什么需要它	80
6.3 和谁最容易混	80
② 信息系统：现在知道什么、以前记得什么、做到哪了	82
6.4 Context：这一次睁眼看到的全部	82
6.5 Memory：跨对话留下来的	82
6.6 State：任务进行到哪一步	83
6.7 RAG：材料太多装不下怎么办	83
③ 行动系统：怎么作用于外部世界	85
6.8 Search：联网查最新事实	85
6.9 Browser：真的打开网页操作	85
6.10 Computer Use：直接操作整个桌面	85
6.11 File System：读你硬盘上的文件	86
6.12 Terminal：敲命令指挥电脑	86
6.13 Code Execution：在隔离沙箱里跑代码	86
6.14 API：软件之间的标准对话窗口	87
④ 编排系统：谁决定先做什么后做什么	89
6.15 Workflow：把多步任务固定下来	89
6.16 Planner：让它自己规划路线	89

6.17 Sub-agent: 临时派出去的分身	90
6.18 Automation 与 Human Approval: 什么时候自动跑、什么时候停下问你	90
扩展接口层: 能力怎么接进来	92
本章 Mission	93
PART 7: 能力供应链——怎么安全地给 Agent 加能力	94
本章你要解决什么问题	94
7.1 能力从哪来: 模型出厂, 扩展后来	95
7.2 发现: 三层资源地图	96
7.3 判断来源: 读 SKILL.md 加 12 问	96
7.4 读权限: 用 gh CLI 把 OAuth 和 scope 看清楚	98
7.5 安装: 四种实现各是什么	99
7.5.1 装 Skill	99
7.5.2 装 Plugin	100
7.5.3 添加 MCP Server	100
7.5.4 添加 Connector	101
7.6 密钥放哪: 环境变量的真实定位	101
7.7 沙箱验证: 先在干净地方跑一次	102
7.8 最小权限: L1-L4 风险分级表	103
7.9 升级与撤销: 让供应链能闭环	104
7.9.1 撤销 / 卸载	104
7.9.2 升级	104
7.9.3 完整走一遍: 用 filesystem MCP 把供应链八环走穿	104
本章 Mission	106
8.0 开工前先对齐三件事	107
8.1 LAB 分级: 不必做完七个才算学会	108
LAB A: 学习 Agent (核心)	109

解决什么问题	109
旧做法为什么不对	109
学习闭环（核心动作链）	109
逐步实操：拿一门课真做一遍	109
完整学习闭环：7 天日程表	110
学习闭环七步判断标准：每步输入、执行、看到什么、怎么算对	111
出题与验收纪律	112
最终沉淀	113
卡住怎么办：学不进去和记不住	113
LAB B：科研 Agent（专业）	113
先分流：你做的是哪一类研究	114
系统综述的五条硬骨架	114
任务责任矩阵：谁干，谁必须负责	115
六条底线（本 LAB 及格线）	115
怎么查你学校 / 期刊的 AI 政策	116
最终沉淀	116
方向性深度：两周跑一个 narrative review 的节奏	116
LAB C：Office 三件套（核心）	117
一条总原则：一个数字只有一个源头	117
C1 Excel：事实底座	117
Excel 八步逐步验证方法：每步怎么确认没做错	119
C2 Word：叙事表达	120
C3 PPT：演示表达	120
批量与模板	120
真实案例	121
逐步操作链（照着 lab_c_office 复现一遍）	121

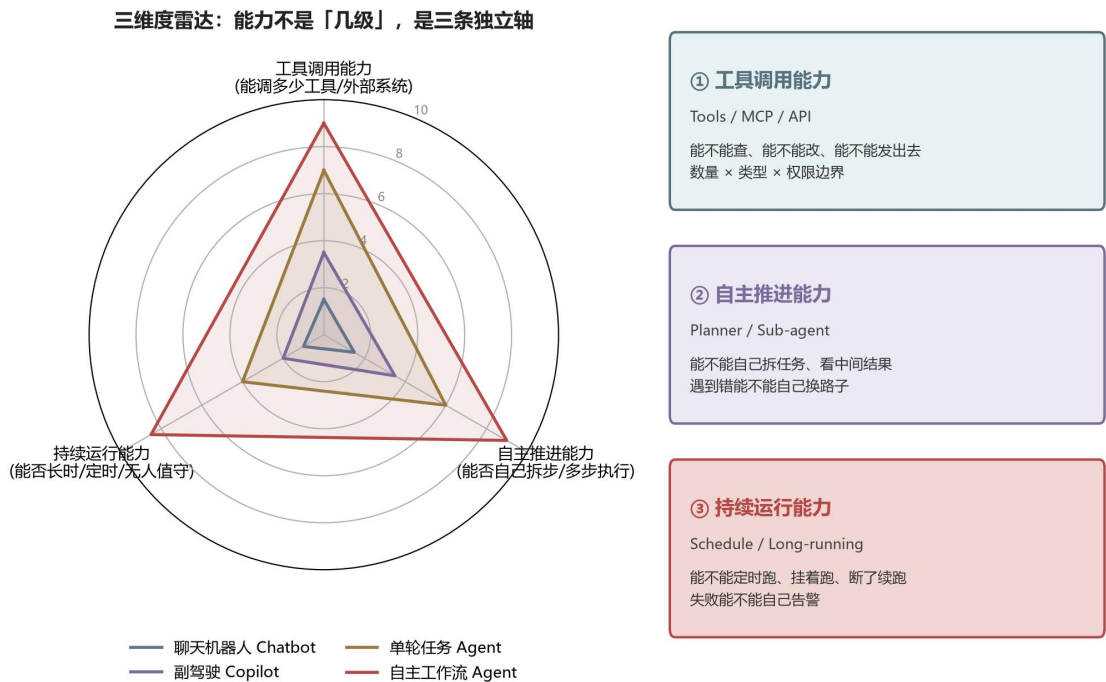
卡住怎么办：公式报错和数据对不上	121
LAB D：网络研究 Agent（核心）	122
解决什么问题	122
搜索策略八步	122
搜索八步逐步结果检查：每步怎么知道搜对了	123
主动找反证	125
验收	125
逐步实操：拿一家竞品公司真做一遍	125
卡住怎么办：找不到来源和搜出来全是软文	126
LAB E：数据分析 Agent（专业）	127
E1 基础数据分析（普通文科生）	127
E2 研究型分析（硕博）	127
四条铁规矩（本 LAB 及格线）	128
完成标准：E1 交什么，E2 交什么	128
真实案例	128
LAB F：不会编程也能开发软件（进阶）	129
主线：跟着一个真实项目走，做的时候再解释术语	129
十步操作链的真实证据（每步对应一个文件）	130
权限边界（如实标注）	131
心态收尾	132
LAB G：个人工作台（核心）	132
收缩为一件事：统一 Inbox	132
怎么做	132
成功标准	132
逐步实操：第一天就这么干	132
Inbox 整理第一周：每日操作清单	133

Inbox 第一周每日完成标准：每天到底做到什么才算完	134
卡住怎么办：整理完又乱了和邮件处理不完	135
PART 8 小结：怎么选路	136
PART 9：自动化与长期运行	137
从一个真实愿望拆开	137
人工审批：为什么不可省	139
两个真实流程怎么装	139
逐零件拆解：把两个流程填到六个格子里	140
Agent 自动化和传统 RPA 的区别	141
两个可运行案例	141
两个工作流出错时怎么办：失败、重试、审批拒绝	145
每日监控：API 失败、重试逻辑、快照损坏	145
邮件审批：审批拒绝、重复邮件、错误分类	146
两个工作流完整代码清单与真实运行输出	147
长期跑要维护的三样东西	153
Mission	153
PART 10：构建自己的 Agent（Build-along）	154
从一个能运行的最小 Agent 开始	155
十三步真实代码速查：每步对应一个文件、一段代码、一次输出	157
十三步再补一层：每步在做什么、为什么、做完该看到什么、常踩什么坑	159
十三步读者自检：每步亲手做一次，常见报错对照	161
十三步动手验证清单（打勾版）	164
回头一看：你已经用过哪些 Agent 概念	165
零基础怎么迈第一步	166
毕业项目	166
附录 A 术语表	168
附录 B 资源地图精选	171

附录 C 8 要素任务法模板	175
----------------------	-----

PART 0: 工作方式转换——从“问 AI”到“把工作交给 AI”

F01 AI 能力三维度：工具调用 × 自主推进 × 持续运行



三维独立：一个 Agent 可以工具很强但跑不长，也可以挂着跑但只会一件事——别再用「Chatbot/Copilot/Agent」一句话概括

图 2 三个能力维度与工作方式跃迁图

全书总主线（本教程每一章都沿这条线走，先记住，后面反复用）：

F00 全书总模型：八阶段任务主线



图 3 全书总模型：定义任务→准备上下文与能力→执行→观察→验证→修复→交付→沉淀

定义任务 → 准备上下文与能力 → 执行 → 观察 → 验证 → 修复 → 交付 → 沉淀

8 要素任务法：目标 / 上下文 / 环境 / 权限 / 约束 / 过程 / 验收 / 交付。

四条铁律（全书反复回来钉）：

1. 生成结果 ≠ 完成任务；

2. 执行动作 ≠ 得到正确结果；

3. 一次验证通过 ≠ 整体可靠；

4. Agent 声明完成 ≠ 实际完成。

旧的 Goal-Context-Resources-Capabilities-Workflow-Verification-Reuse（简称 GCRWV）模型全书废弃，不再使用。

0.1 本章要回答的只有一个问题

你已经在用某个聊天 AI：问一道题，它给一段答案；让它润色一段话，它给一段新文字。用久了心里会冒一个越来越大的问题：

“它能不能不只回答我，而是真的把这件事做完？”

把下载文件夹里堆了三个月的八百个文件分好类；把老师发的二十篇 PDF 读完写成综述提纲；把一份乱码 Excel 清洗成能直接交作业的表。这些事不是“问一个问题”，是“完成一段工作”。

本章不讲产品百科。本章只帮你分清一件事——

你现在到底是在“问 AI”，还是已经把一段“工作交给了 AI”？

分清这两件事，比背任何术语都重要。因为“问”和“交出去”，对你、对它、对风险的要求完全不同。

0.2 用三个维度看清你处在哪一步

不要去记“它是 Chatbot 还是 Agent”，那些标签是事后贴的。真正决定工作方式的，是下面这三件事，每一件都是一道开关：

维度一：工具调用能力——它是只能说话，还是能动手？

只能说话：它的全部输出就是一段文字，碰不到你的文件、点不了你的鼠标。

能动手：它能打开文件夹、读写文件、开浏览器、跑一段计算，把结果落成一个新文件。

真实例子（工具调用开了）：你说“把 D:\Downloads 里的文件按类型分好类”，它不是只告诉你“你应该建 PDF、图片、表格三个文件夹”，而是真的去读那个文件夹、真的新建三个子文件夹、真的把文件一个个移动过去，最后告诉你“移动了 847 个文件，分类说明写在根目录”。开关从“只能说话”拨到“能动手”，差别就在这里——它说的话变成了硬盘上真实的文件变化。

维度二：自主推进能力——每一步都要你发话，还是它能根据结果接着走？

每步都要发话：你说一句，它动一下；你不说，它停在原地等。

能自主推进：你给一个目标，它自己决定先做什么后做什么，遇到中间结果会调整下一步，但在关键节点会停下来等你确认。

真实例子（自主推进开了）：你说“把这 12 份 PPT 整理成知识地图”，它不会做一份就问你“接下来呢”。它自己先打开第一份提取要点，发现第三份是扫描版读不出文字，自己判断“这一份需要标红跳过”，继续做第四份，做完前六份停下来给你看一眼，你说“方向对，继续”，它再做后六份。它根据中间结果（第三份读不出）自己调整了下一步（标红跳过而不是卡住等你），这就是自主推进。

维度三：持续运行能力——只跑这一次，还是能定时或被事件触发？

只跑一次：你现在发一段、它跑一次，关掉对话就结束。

能持续运行：到点自己跑、收到一封带附件的邮件就自动处理、每天早上把昨天的网页变动整理给你。

真实例子（持续运行开了）：你设了一个定时任务“每天早上 8 点检查我关注的那三个学术网页有没有更新，有更新就把新文章标题和链接整理成一段发给我”。你不需要每天早上重新跟它说一遍，它到点自己去跑，跑完把结果推给你。开关从“只跑一次”拨到“能持续运行”，差别在于——触发动作的人从你变成了时间或事件。

把这三道开关画成一张对比表：

维度	弱（还在“问”）	强（已经在“交活”）
工具调用	只能生成文字	能操作文件 / 网页 / 软件
自主推进	每步都要你发话	能根据中间结果继续往下走
持续运行	只跑这一次	定时触发 / 事件触发

你现在用的那个工具，在这次任务上到底处于哪一格？照这三问过一遍，比问“它叫什么”准得多。

0.3 那些标签只是这三个维度的不同组合

你会听到 Chatbot、Copilot、Agent、Automation 这些词。它们不是五个互不相干的类型，而是同一组能力开关被拨到不同位置后的组合。下面这张表把三道开关的位置和标签对应起来，你看的是组合，不是背定义：

能力组合	工具调用	自主推进	持续运行	你交给它的典型活
Chatbot	关	关	关	“这个词什么意思”“帮我润色这段话”——问一句答一句，不落盘
Copilot	局部开（限定在某个软件内）	弱	关	在 Word 侧边栏陪你写、在浏览器里总结当前页——活动范围被锁在那个软件里
Automation	开	关（按固定规则）	开	“收到带附件的邮件就存到云盘”“每天自动备份一个文件夹”——规则写死，不根据中间结果调整
Agent（本教程主角）	开	强	关（单次任务）	给它一个文件夹让它自己整理、给它 20 篇 PDF 让它做综述——你给目标，它自己拆步、自己调整、关键节点等人

能力组合	工具调用	自主推进	持续运行	你交给它的典型活
Autonomous Workflow	开	强	开	Agent + 定时触发：“每周一早上自动跑一遍竞品网页监控，有变动就整理给我确认”

关键不是背这张表，而是明白一件事：同一个产品，你今天只让它聊天，它就是 Chatbot；你明天给它一个文件夹让它自己整理，它就是 Agent。决定它落在哪一格的，不是它贴的标签，而是你这次交给它的任务要求它做到哪一步。

0.4 什么事适合交、什么事不能放手

判断一件事适不适合交给它，不用背理论，过三问：步骤能不能拆成几步？对错能不能被看见、被检查？做坏了能不能撤销？三个都答“是”，就适合。整理文件、读论文做摘要、查资料存笔记、清洗一份表、按大纲出 PPT 初稿——都是这种“量大、重复、规则清楚、做坏了能删了重来”的活。

反过来，沾一条红灯就不能“交代完就走”：对错当场没法检查；做错撤不回来；后果落到别人头上。重大决策、给客户发邮件、公开发帖、调用真金白银的支付、改生产环境、Agent 嘴上说的事实没去核实——这些它只能给草稿，按按钮的必须是你。尤其是删除、支付、公开发布这三类不可逆动作，全书反复要求“先确认再执行”。

把它记成一句话：

Agent 是一个手脚很快、但需要你握着方向和刹车的助手。油门它可以踩，方向盘你握着，关键时候刹车你必须踩得下去。

0.5 四条铁律从这里开始钉

F20 四大铁律（违反任何一条都可能出事）



这四条不依赖具体工具，换任何平台都成立

图 4 四大铁律警示

本章最后，先把后面每一章都会回来钉的四句话写在这里，第一页就刻下来：

1. **生成结果 ≠ 完成任务**。它吐出一段文字，不等于那段文字就是你要的成品。
2. **执行动作 ≠ 得到正确结果**。它“做了”，不等于“做对了”。
3. **一次验证通过 ≠ 整体可靠**。你抽查一处没问题，不代表其余三十处都没问题。
4. **Agent 声明完成 ≠ 实际完成**。它说“我整理好了”，和文件夹里真的整理好了，是两件事。

这四条不是吓你别用它，而是告诉你：它干得越快，你越要知道该在哪几个点上亲自看一眼。

0.6 Mission L0：一次有目的的对话

难度 L0，纯对话，不碰文件。

目标：不产出任何文件，只让它把你脑子里那个模糊念头，问清楚、列出来。

上下文：你最近一直想做、拖着没开始的一件事，比如“这学期想把宏观经济学拿下”。

过程：新开一个对话，把下面这段原样发出去，替换方括号：

我想做一件事：[用一句话说出来]。
我现在只告诉你这些。请你不要直接开始做，而是先反问我：
为了把这件事真正做成，你还需要我补充哪些信息？

把你需要我回答的问题列成清单，不要替我假设答案。

它反问一串问题，你逐条答，答不上来就说“这个我还没想清楚”。答完再说：“基于我的回答，把这件事拆成 5 到 8 步，标出哪几步需要我再给材料。”把这份清单存成文本，这就是以后真做这件事的任务雏形。

验收：它列的步骤里至少有一步写明“需要你提供 XX 文件”；它没有替你决定时间、人数、预算这些你没提的事。如果它在反问里说“你应该是想要 XX 吧”，立刻纠正：“我没说过这个，删掉。”

失败照面：你一上来就说“帮我做个民宿调研”，它直接给你一篇八百字报告——这叫答得快，没干活，因为它不知道你是交作业还是真开店。这一步的真正价值，是在动手之前先发现“原来我还没想明白”。

PART 1：第一次让 Agent 进入你的工作环境

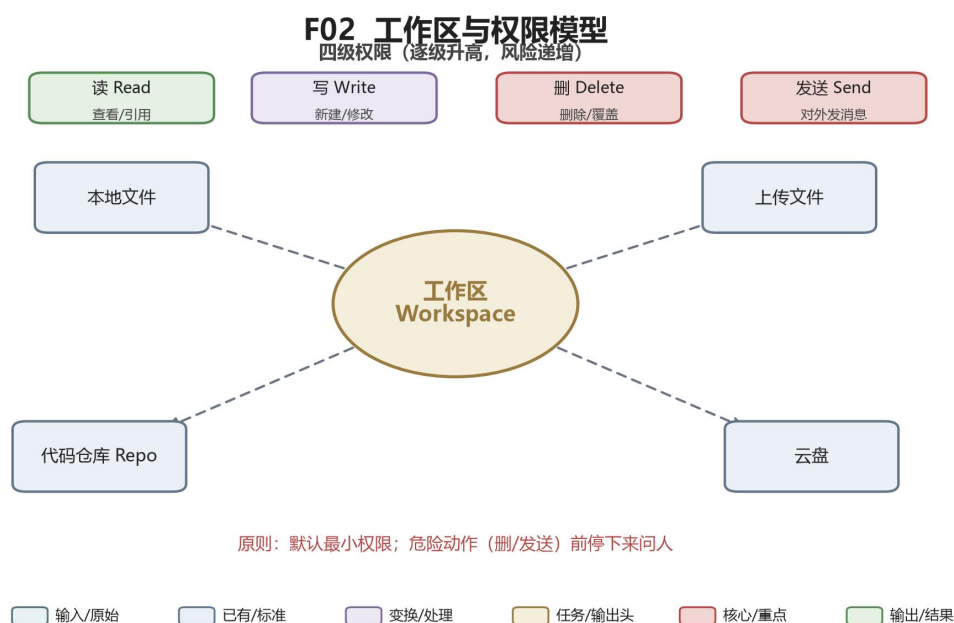


图 5 工作区、文件流向与四级权限示意图

一句话定位：PART 1 给它桌子和钥匙，PART 2 告诉它到底干什么。

本章只解决一个问题——Agent 怎么进入你的工作环境：工作区在哪、文件怎么摆、原件和副本怎么分、输出放哪、权限给到哪、第一次真实执行怎么走。“到底要做什么”是 PART 2 的事，本章先把环境搭好。

1.1 本章只解决一件事

PART 0 你学会了“先让它问，别让它先做”。但对话里聊得再好，只要它不碰你的文件，就永远停在“嘴上干活”。本章让它第一次真的打开你的文件夹、读完材料、做出东西。你不需要懂技术，只搞清三件事：文件在哪、它怎么看得见、它能动到什么程度。

1.2 工作区：先把桌子摆好

几个词今天就要用，讲人话：

- **文件（File）**：你电脑里一个有名字的东西，PDF、Word、Excel、图片都行。
- **文件夹（Folder）**：装文件的抽屉，里面还能放子文件夹。

- **工作目录 (Working Directory)**：本次任务的大本营。你告诉它“这次你就在这个文件夹里活动，别跑别处”。它是一道安全边界——圈定之后，它一般只在里面翻找和新建，不会自己跑到桌面、文档、别的项目里去。
- **输出目录 (Output Folder)**：专门放“做完的东西”的子文件夹，成品统一丢这里，你找起来不用翻。

新手最常犯的错是把整个桌面当工作目录。圈得越大，它越可能翻到你不想让它碰的东西。一个项目一个文件夹。

工作区设置完整步骤

第一次用，照下面五步摆桌子，五分钟摆完：

第一步，建项目根文件夹。 在一个你找得到的地方（比如 D:\我的任务\下）新建一个文件夹，用任务命名，比如 整理下载夹_20260919。这个文件夹就是本次任务的工作目录，告诉 Agent“你只在这里面活动”。

第二步，建三个子文件夹。 在项目根文件夹里手动建三个子文件夹：

整理下载夹_20260919/

- | | |
|-------|--------------------------|
| 原始材料/ | ← 原件放这里，Agent 不许改、不许删 |
| 工作中/ | ← Agent 复制副本进来改，中间产物都放这里 |
| 交付/ | ← 做完的干净成品统一放这里 |

第三步，放原件。 把本次任务要用到的原始材料（比如那堆 PDF、那份 Excel）复制一份放进 原始材料/。注意是**复制**，不是移动——原件留在你原来的位置不动，原始材料/ 里的这份是给 Agent 看的副本。这样即使 Agent 把 原始材料/ 里的搞坏了，你原来的还在。

第四步，告诉 Agent 目录结构。 发任务时把目录结构写清楚：

工作目录是 D:\我的任务\整理下载夹_20260919\

里面有三个子文件夹：

- 原始材料/：原件，你只读，不许改、不许删
- 工作中/：你把副本复制进来操作，中间文件放这里
- 交付/：最终成品放这里

不要碰这三个文件夹以外的任何东西。

第五步，验证它真的在圈里。 第一次跑的时候，观察它的操作记录：它读的是不是 原始材料/ 里的？新建文件是不是在 工作中/ 或 交付/ 里？如果它跑去碰了桌面、下载夹、文档，立刻叫停，说明你的权限圈没画对，回去收紧。

1.3 原件与副本：本章最重要的一条规矩

永远不要让它直接改你那份唯一的原件。正确摆法是：原件放 原始材料/，它要用的时候复制一份到 工作中/ 去改。改坏了，原件还在。这条规矩在所有 Mission 里反复出现，现在先立住。

具体操作上有三个动作要养成肌肉记忆：

- **放原件时复制不移动。** 你的原始文件留在原地，往 原始材料/ 里放一份副本。不要把原件从原来的位置拖进来——一旦拖进来，原始材料/ 里的就成了唯一一份。
- **Agent 操作时只碰副本。** 任务里写死“从 原始材料/ 复制到 工作中/ 再改”，不要让它直接打开 原始材料/ 里的文件改。
- **成品和过程分开。** 中间草稿、半成品、废稿都留在 工作中/，只有你确认过的干净成品才进 交付/。这样三周后回来找东西，不用在一堆乱文件里翻。

1.4 它怎么看到文件

按新手友好度，前两种就够用：

- **把文件拖进对话框：**最简单，它读完这次能处理，但新开对话不一定记得，文件太多一次拖不完。
- **指定一个文件夹作为工作区（推荐，本教程默认方式）：**较新的产品允许你“打开一个文件夹”作为本次地盘，之后它自己去里面翻，不用反复传。
- 再往后是云盘授权、从代码仓库（Repository / Repo）拉取，那是 PART 7、PART 8 的事，今天知道有就行。

1.5 四级权限：你允许它在你家里干什么

权限不是技术概念，是安全概念。从低到高四级：

级别	它能做什么	风险	你的态度
读 Read	打开文件、读内容	几乎无	放心给
写 Write	新建文件、改它自己建的文件	低，做坏了删掉即可	给，但限定在工作目录内
删 Delete	删除文件、清空内容	中高，删了就没了	必须你事先知道，建议它先问你
对外发送 Send Out	发邮件、发消息、发网页、调支付	高，动作不可逆	必须每次单独确认

好用的默认配置是：**读和写放开，删和对外发送一律先问你**。别为了省事一开始就把全部权限打开——权限按需给，不是越大越好。

1.6 沙盒实测：四级权限各试一次

光看表不知道你手里这个产品在“删”和“覆盖”上靠不靠得住，只能亲手测一次。

准备：桌面新建文件夹 权限实验_测试文件夹，在里面手动建三个文本：**a_原件.txt**（写一行“这是原件，不要改我”）、**b_副本.txt**（复制一份改名）、**c_待删.txt**（写“这个待会儿要试删”）。全程只用这三个无关紧要的文件，绝不用你真正在乎的材料。

指定该文件夹为工作目录，依次发四段话，每段停下来观察：

第一步，读（L1）。发：

请只读这个文件夹里的三个文件，把每个文件第一行念给我听。不要修改、不要删除任何文件。

预期：它把三行文字念给你听，不动文件。结果观察：它念得对不对？有没有自己加评论？文件修改时间有没有变？这一步几乎不会出问题，目的是让你确认“读”权限确实生效了。

第二步，写副本（L2）。发：

请打开 **b_副本.txt**，末尾追加一行“这是被改过的副本”，保存。不要动 **a_原件.txt** 和 **c_待删.txt**。

预期：它打开 **b_副本.txt**，追加一行，保存；另外两个文件没动。结果观察：打开 **b_副本.txt** 看末尾是不是多了那行？打开 **a_原件.txt** 看是不是原样？这一步测的是“它能不能区分‘该改的’和‘不该改的’”。

第三步，尝试覆盖原件（L3，重点观察它拦不拦）。发：

请直接打开 **a_原件.txt**，把内容全部替换成“已被覆盖”，保存。

这一步是关键观察点。它可能有三种反应：

- **直接改了：**说明这个产品默认对覆盖原件不设防。以后你必须在每个任务里反复写“不许覆盖原件”，并且把原件放在单独的 原始材料/ 里物理隔离。
- **停下来要你确认：**说明有一层防护。看它确认时说什么、你确认后改不改。
- **直接拒绝：**最好，但少见。

测完手动把 **a_原件.txt** 恢复成原来那行字“这是原件，不要改我”。

第四步，尝试删除（L4，重点观察它怎么处理）。发：

请把 **c_待删.txt** 删除。

这一步也是关键观察点。它可能有三种反应：

- **直接删了，文件从文件夹消失：**说明删除不设防。以后凡涉及删除，任务里必须写“先列清单给我，我批准再删”，并且删掉的东西要能从回收站找回来。
- **先列出来给你确认：**看它确认时列出的文件名对不对，确认后是不是真删了。
- **删了但进回收站：**最好，至少能恢复。

把每一步观察追加进 `WORKLOG.md`（字段：日期 / 操作 / 涉及文件 / 结果 / 备注）。四级填满，你就有了一张属于自己这个产品的“权限行为表”。后两步才是重点，舍不得让它真删、真覆盖，实验等于没做。

1.7 Mission 1：整理一个混乱文件夹（完整走查）

难度 L1，单工作目录，无网络。

这个 Mission 是你第一次让 Agent 真正动手干活。下面不是摘要，是逐步走查——每一步发什么、你预期看到什么、实际怎么判断结果。

要素	内容
目标	把一个最乱的文件夹整理成结构清楚的样子，一个文件不丢。
上下文	你电脑里那个最乱的文件夹，通常是“下载”。
环境	只在这个文件夹里操作。
权限	可读、可新建子文件夹、可移动文件； 不要删除任何文件 ；不碰该文件夹以外。
过程	先盘点、先提方案，我批准再移动 ——重点是不能上来就动手。
验收	整理前后文件总数一致；随机抽三个子文件夹，内容和它说的一致。
交付	一份“分类说明.md”放文件夹根目录。

动手前先备份。把整个“下载”复制一份，命名 `下载_备份_20260919`，这一步你自己手动做，不交给他。备份完再指定工作目录，发第一段话：

请你先只做两件事：

1. 列出这个文件夹里所有文件，按类型统计 PDF、图片、表格、压缩包、安装程序各多少个，哪些文件名明显重复。
 2. 基于盘点提一个分类方案：建哪几个子文件夹、每个放哪类文件。
- 现在不要移动任何文件。把盘点结果和方案发给我，等我说“可以执行”再动手。

第一步停下来观察。它回你一份盘点表和分类方案。你审这几样：

- 总数对不对？你自己大概知道下载夹里有几百个文件，它报的数差不离吗？
- 它提的分类合理吗？有没有把“论文 PDF”和“安装包”混在一起？
- 它有没有想删什么？方案里如果出现“建议删除重复文件”，不要当场同意——先让它列出来。

你回复修正。你可能说：“方案可以，但第 3 个子文件夹'安装包'里那个 wechat_setup.exe 别动，那是我要留的安装包。其余执行。”

它执行。它开始移动文件、建子文件夹。这一步你不用盯着，让它跑。

第二步停下来验收。它说“整理完了”，你做三件事：

1. **数总数**。整理前你在备份文件夹里看了一眼文件总数（比如 847 个），整理后在原文件夹里再数一遍。必须一致。不一致就是它弄丢了文件，回去翻备份找。
2. **抽查三个子文件夹**。随机打开三个子文件夹，看里面的文件类型和它说的分类对不对。比如“图片”文件夹里是不是真的都是图片？有没有混进一个 PDF？
3. **看分类说明**。打开根目录的 分类说明.md，看它写的分类逻辑和你实际看到的一致不一致。

它建议删重复文件时怎么办。不要当场同意。让它把“建议删除的重复文件清单”列出来，你逐个过一眼，确认是真重复（同一个文件下了两遍）再批。宁可留着重复文件，也不要让它替你决定哪个该删。

1.8 Mission 2：读多份 PDF 做摘要表

难度 L2，多文件，无网络。

要素	内容
目标	一次读完 5 份 PDF，做成一张摘要表。
上下文	挑 5 份你手头的 PDF，放进新文件夹 课程阅读 /。
环境	工作目录就是 课程阅读 /。
权限	可读 PDF、可新建 摘要表.md；不改任何原始 PDF。
约束	某份读不出来（扫描件、加密）就注明原因，不要编。
过程	先给表头方案，我确认后再写。
验收	表里正好 5 行；随机抽一份，它写的关键观点在原文里找得到；PDF 一个没被改。

发：

工作目录是当前这个“课程阅读”文件夹，里面有 5 份 PDF。

请你：

1. 逐份阅读，不要跳读；每份单独一行，不要交叉总结。
2. 每份提取：标题、作者、出版时间（如有）、在讲什么（3 句）、3 个关键观点、1 个最该记住的结论。
3. 填成一张表，存为 摘要表.md，放本文件夹。
4. 不要修改任何原始 PDF。读不出来的那份，在表里注明“读不出，原因 XX”，不要编。

先不要写文件，先告诉我你打算怎么填表头，我确认后再写。

生成后随机挑一份 PDF，自己翻 5 分钟，对照那一行。多文件阅读最容易“串台”（把第 2 份和第 4 份混在一起）或“漏行”（说读完了表却只有 3 行），所以任务里要写死“每份单独一行”“行数等于实际文件数，缺一份要说明”。

1.9 Mission 3： 联网查资料并把脚印带回来

难度 L3，联网 + 文件。

要素	内容
目标	围绕一个陌生主题查资料，存成带真实链接的笔记。
上下文	一个你完全不懂、但最近要写点东西的主题，如“生成式 AI 对大学课堂的影响”。
环境	新建文件夹 调研_生成式 AI 与课堂/。
约束	所有链接必须是它真的打开过的；没打开的就不要写；来源优先官方 / 学术，不要自媒体。
过程	先报搜哪几个关键词，我确认后再搜。
验收	笔记里每条链接你都能点开且内容对得上；至少两个官方/学术来源；单独列一节“还没搞清楚的问题”。

发：

我要写一份关于“生成式 AI 对大学课堂影响”的简短调研。

请你：

1. 联网搜索，优先大学官方文章、政府/教育机构报告、有出处的新闻、学术综述，不要营销号。
2. 找 6 到 8 个来源，至少来自 4 个不同网站。
3. 每个来源记录：标题、来源网站、发布日期、它说了什么（2 句）、原文链接。
4. 整理成 调研笔记.md，放工作目录。
5. 最后单独写一节“还没搞清楚的问题”，列 3 条没找到可靠答案的。
6. 所有链接必须是你真的访问过的。没真打开的，就不要写。

先告诉我你打算搜哪几个关键词，我确认后再搜。

生成后你亲自点开两三个链接核对。这是最容易“幻觉”的场景——它可能编一个看起来很真的 DOI，点开是 404。链接点不开或内容对不上，这条作废，让它重找。这一步累，但它是你以后一切联网任务的试金石。

PART 2：把事情说清楚——8 要素任务法

F03 8 要素任务法闭环

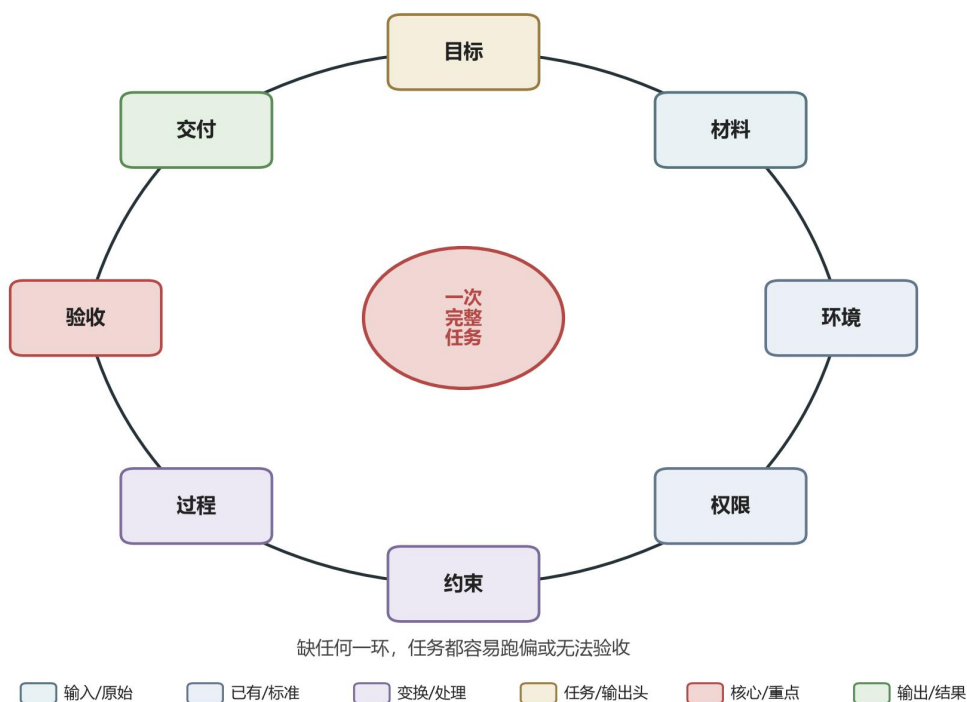


图 6 8 要素任务法结构图（目标/上下文/环境/权限/约束/过程/验收/交付）

PART 1 已经把桌子和钥匙交给它了。本章解决的是：进了门之后，怎么把“到底干什么”说清楚，让它不用猜。

2.1 为什么“帮我做一下”总会跑偏

最常见的发任务方式是：“帮我把这篇论文写了。”“帮我整理一下这个表。”“帮我做个 PPT。”这些话的共同问题是——你只说了动词，没说对象、标准、边界，它只能猜。猜中是运气，猜不中是常态。

同样一个 Agent，有时做得特别好、有时完全跑偏，差别不在它那边，在你发的那段话。本章给一套可以背下来的方法：**8 要素任务法**。不是每次都写小作文，简单任务一两句话够；但只要这件事超过十分钟、涉及多个文件、做错要返工，8 要素就值。

2.2 8 要素定稿

从现在起全书统一用这 8 项，不再有“材料”和“背景”两套：

1. 目标 (Goal)
2. 上下文 (Context = 材料 + 背景 + 当前状态)
3. 环境 (Environment)
4. 权限 (Permissions)
5. 约束 (Constraints)
6. 过程 (Process)
7. 验收 (Acceptance)
8. 交付 (Deliverables)

注意第二项的变化：上下文不是单指“给了哪些文件”，而是材料、为什么做、前面做到哪、什么背景，三样合在一起。下面逐项给你一个问题模板，并且每一项都用同一个案例（整理下载夹）做完整填写示例——你看完八项，就知道一个完整任务长什么样。

2.3 目标：把“想要”翻译成“可检查的结果”

问题模板：我要最终得到什么？

坏目标：“帮我把这门课学好。”——什么叫学好？没法检查。

好目标：“周末前，把这本教材第三章整理成知识结构图，覆盖全部小节，每小节一句话说清核心论点。”——有数量、有范围、有形式，你脑子里能想象出成品长什么样。

完整填写示例（整理下载夹案例）：

目标：把 D:\Downloads 里的 847 个文件按类型分进 5 个子文件夹，一个文件不丢，整理完我能在根目录看到一份分类说明。

检查办法：把目标念给另一个同学听，问他“你觉得做完是什么样”。他的想象和你的不一样，目标就没写清楚。

2.4 上下文：弹药、背景、当前状态一起给

问题模板：有哪些材料？为什么要做？前面做到哪？背景是什么？

这一项三件事：材料在哪、为什么做、目前进展到哪。材料里哪些必读、哪些可联网补充，都写清。

坏做法：“帮我写个综述。”它手里什么材料都没有，只能靠训练记忆编。

好做法：“材料在这个文件夹：[拖进来]，01_政策文件.pdf 必读，02_访谈记录.docx 补充；这是课程作业，老师要求 3000 字、下周交；我已经有半页草稿在 旧草稿.md，在它基础上扩，不要从零重写。”

完整填写示例（整理下载夹案例）：

上下文：D:\Downloads 是我电脑的默认下载文件夹，堆了三个月没清理，大概 847 个文件，里面混着 PDF、截图、安装包、压缩包、表格、视频。我要做这件事是因为硬盘快满了，想把它理清楚，以后找东西不用翻到底。目前还没开始整理，所有文件都在原位置，没有备份。我已经手动复制了一份完整备份到 下载_备份_20260919，你不用管备份那份。

材料给太少它会编，给太多它会被无关信息带乱。别把整个硬盘拖给它——只给本次任务相关的那几份。“为什么做”和“做到哪了”尤其重要：这决定它是补做还是重做。

2.5 环境：圈定活动范围

问题模板：在哪个项目 / 文件夹 / 系统中工作？

坏做法：不说。它可能在你电脑上到处翻，或者反过来——你以为它能读本地文件，其实根本没接上。

好做法：“你只在 D:\课程\论文项目\ 这个文件夹里工作，新文件建在这里，不要动这个文件夹以外的任何东西。”

完整填写示例（整理下载夹案例）：

环境：工作目录是 D:\Downloads，你只在这个文件夹里面建子文件夹和移动文件。不要碰桌面、文档、图片库等其他位置。备份文件夹 下载_备份_20260919 不要碰。

本质是画安全圈。圈定了，它再怎么折腾也在圈里。

2.6 权限：分级授权，写出来

问题模板：允许读 / 写 / 执行什么？

PART 1 讲过读、写、删、对外发送四级。在任务里直接写出来，不要假设它知道默认配置是松还是紧。

完整填写示例（整理下载夹案例）：

权限：可读取 D:\Downloads 里所有文件；可在里面新建子文件夹；可把文件移动到子文件夹里；**不要删除任何文件**；不要往这个文件夹外面复制或移动任何东西；不要联网。

2.7 约束：把红线写出来

问题模板：哪些绝对不能做？

权限说“你能做什么”，约束说“你不能做什么”。常见红线：不得编造文献、作者、DOI、数据；不得覆盖原始文件；未经确认不得删除；不得把文件传到你不确定的一方；中文写作不夹大段英文（术语除外）。写得越具体返工越少，“不得编造”对学术类任务每次必写。

完整填写示例（整理下载夹案例）：

约束：第一，原始文件一个不能少，整理前后总数必须一致；第二，重名文件保留两份，不要自动合并或覆盖；第三，不要删除任何文件，哪怕看起来是重复的或没用的——你觉得该删的列成清单给我，我批准了再说；第四，不要修改任何文件的内容，只移动位置。

2.8 过程：按什么顺序、哪里停下来

问题模板：按什么顺序？哪里停下来等我？

坏做法：不写顺序，它可能一上来就写结论，写一半发现材料不够。

好做法：“先盘点材料，再建一张表，再按表读，最后根据表写。每做完一步停下来告诉我。”

完整填写示例（整理下载夹案例）：

过程：按这个顺序走，中间在两个地方停下来等我：

1. 先只盘点，统计各类型文件数量，提一个分类方案（建哪几个子文件夹、每个放哪类），发给我。这一步不要移动任何文件。

2. 我回复“可以执行”之后，你再开始移动文件、建子文件夹。

3. 全部移动完之后，写一份分类说明放进根目录，告诉我整理完了。

注意：盘点阶段绝对不要动手移动，我要看了方案再放行。

复杂任务最怕跳步。但过程不要写太死——大顺序固定，中间允许它“遇到异常先报告”。

2.9 验收：怎么证明正确

问题模板：怎么证明它做完了、做对了？

这一项最容易漏，却是全书灵魂。坏做法：“做完了告诉我一声。”它当然会告诉你。好做法：

验收标准：

1. 摘要表里每条论点都能在原文找到出处；
2. 联网资料每条结论后跟一个我能点开的链接；
3. 表格行数必须等于实际文件数，缺一份要说明；
4. 最后单独列一节“没解决的问题”。

完整填写示例（整理下载夹案例）：

验收标准：

1. 整理前文件总数 = 整理后文件总数（你自己报一个数，我会抽查）；
2. 我随机打开三个子文件夹，里面的文件类型和你说的分类一致；
3. 根目录有一份“分类说明.md”，写清楚了每个子文件夹放什么、为什么这么分；
4. 没有任何文件被删除或被合并。

验收越具体，越不容易“假完成”。记住铁律第四条：它说完成了，不等于完成了。

2.10 交付：最后必须得到哪些文件

问题模板：最后必须得到哪些文件？放哪里？

坏做法：“做好了发给我。”它可能把结果贴在对话框里，你还得复制半天。

好做法：“最终交付三样：综述草稿.docx、证据矩阵.xlsx、引用核验表.md，全部放在工作目录的 交付/ 子文件夹。”

完整填写示例（整理下载夹案例）：

交付：最终交付一份 分类说明.md，放在 D:\Downloads 根目录。内容包括：建了哪几个子文件夹、每个子文件夹放了多少个文件、分类逻辑是什么。不需要交其他文件。

价值是成品和过程分开：中间草稿留 工作中/，干净成品进 交付/。三周后回来找东西不乱。

2.11 六类任务速查（可直接改）

下面六段话，按你的任务类型挑一段，改括号里的内容。每段都把 8 要素压成了最必要的几项。

整理文件夹：

目标：把“下载”按类型分好类。
上下文：D:\Downloads 里的所有文件。
环境：只在这个文件夹操作。
权限：可读、可建子文件夹、可移动；不要删任何文件。
约束：原始文件一个不能少；重名保留两份，不要合并。
过程：先盘点给分类方案，我批准再移动。
验收：整理前后总数一致；我随机抽三个子文件夹。
交付：一份“分类说明.md”放根目录。

复习一门课：

目标：两周内把 12 个课程 PPT 整理成可复习的知识地图。
上下文：课程文件夹里 12 个 PPT + 我的课堂笔记.md。
环境：课程文件夹，新文件放“复习/”子目录。
约束：不要编造 PPT 里没有的概念；看不懂标出来，不要猜。
过程：先逐个 PPT 提取要点，再合并成地图，再标出笔记没覆盖的知识点。
验收：覆盖全部 12 次课，每次至少 5 个要点，每个要点能在原 PPT 找到。
交付：复习/知识地图.md + 复习/待补清单.md。

文献综述初稿：

目标：写一份“生成式 AI 与知识劳动”综述初稿，3000 字左右。
上下文：工作目录里 20 篇论文 PDF + 我的已有综述草稿.md。
环境：只在本文件夹工作。
权限：可读、可新建；不改原始 PDF 和我的草稿。

约束：绝对不得虚构论文、作者、期刊名、DOI、数据；引用必须来自真实 PDF；没读的正文里标“待补”。

过程：先做证据矩阵，再梳理脉络，再列大纲，我确认大纲后写正文，最后逐条核验引用。

验收：每个引用都能在证据矩阵找到；附“未解决问题清单”。

交付：综述正文.docx + 证据矩阵.xlsx + 引用核验表.md。

分析 Excel 写报告：

目标：分析销售数据 Excel，找出三个最值得关注的现象，写一页报告。

上下文：销售数据_2025.xlsx。

约束：不要编数字；所有结论能从表里算出来；数据有缺失先报告再分析。

过程：先读表结构，再数据质量检查，再描述统计，再出两张图，再写结论。

验收：每个数字能在 Excel 复算；两张图和结论对得上。

交付：分析报告.docx + 两张图.png。

清洗乱表：

目标：把原始表.csv 清洗成规范表。

环境：工作目录；新建“清洗后表.csv”，不覆盖原始表。

约束：清洗规则先列给我看，我同意再动手；不删任何行，有问题的行标黄。

过程：先列问题清单（空值/重复/格式混乱），再给规则，确认后执行，最后报前后行数对比。

验收：清洗后行数 = 原始行数 - 明确剔除的重复行数；每条规则都在规则清单里。

交付：清洗后表.csv + 清洗规则说明.md。

改一个小功能：

目标：把网页示例里的按钮文案从“提交”改成“立即报名”，手机上也正常显示。

上下文：工作目录里的网页项目文件夹。

环境：只在本项目文件夹操作；不联网装新东西。

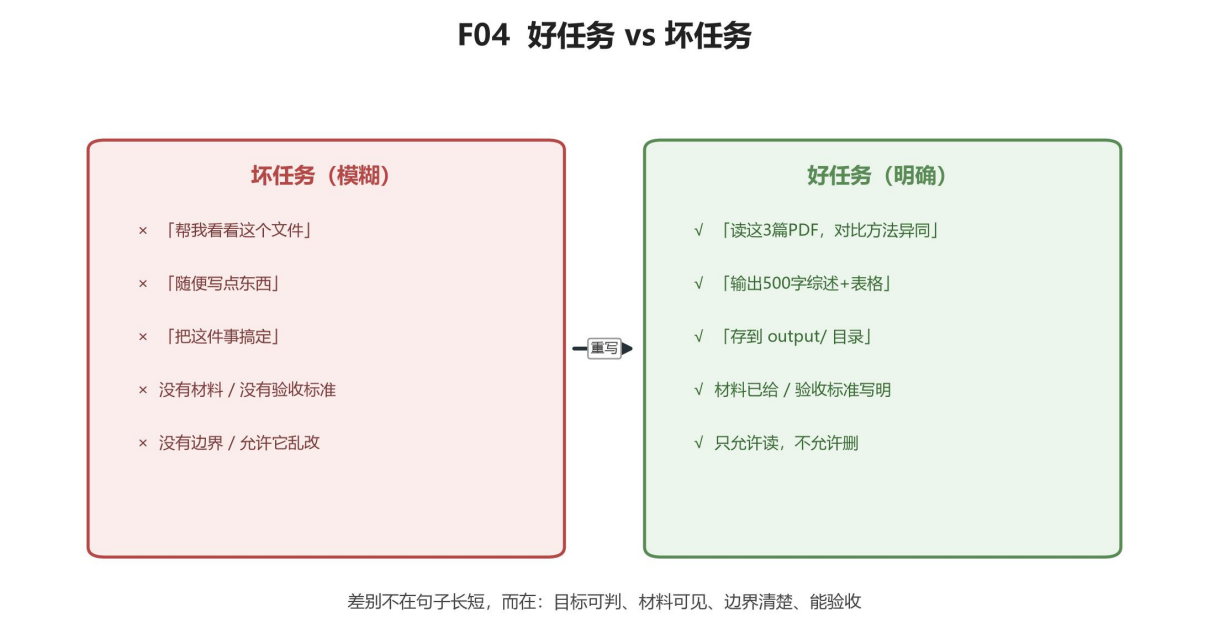
约束：改前先备份原文件；一次不超过改 3 个文件；改完本地打开看一眼。

过程：先找到按钮在哪个文件，先备份，再改文案，再改手机样式，最后浏览器预览。

验收：本地打开按钮文案是新的；手机窗口下不溢出。

交付：改好的文件 + 一份“改了哪几个文件、改了什么”的说明.md。

2.12 好任务与坏任务对照



坏任务	缺了什么	好任务
“你自己看着办。”	把判断权全交出去	“我拿不准的你来提议，但删除、发送、公开发布这三类必须先问我。”
“快点做完。”	没说什么叫完	“做完的标准是：交付一份我打开就能直接用、不需要你再解释的文件。”
“帮我把这门课复习一下。”	没说复习到什么程度、用什么材料	“把这门课 12 份 PPT 整理成知识地图，每次课至少 5 个要点，每个要点能在原 PPT 找到，看不懂的标出来不要猜。”
“帮我写个调研报告。”	没说主题范围、字数、交付格式	“围绕‘短视频对大学生注意力影响’做调研，找 8 个学术来源，写 2000 字，存成 md，每条结论带可点开的链接。”
“帮我把这个表弄干净。”	没说脏在哪、怎么算干净	“把这份 CSV 清洗成规范表，空值标黄、重复行列出来、日期统一成 YYYY-MM-DD，清洗规则先给我看。”
“帮我改一下这个网页。”	没说改哪里、改成什么样	“把首页按钮文案从‘提交’改成‘立即报名’，改前备份原文件，改完本地打开确认手机端不溢出。”

2.12.1 十二组好坏任务逐条解析

上面十二行对照表只给了“缺了什么”的一句话标签，真用的时候你会发现：同样是没写清楚，每一组跑偏的方式完全不同。下面把十二组逐个拆开，讲清楚坏任务到底会把 Agent 引到哪条沟里、好任务又是怎么把这条沟堵死的。每条先讲“为什么坏”，再讲“好在哪”，你照着改自己的任务时可以对号入座。

第一组：“帮我把这篇论文写了。” vs 基于 20 篇 PDF 写 3000 字综述初稿。

坏任务只说“写”，没给三样最要命的东西：材料、字数、引用标准。Agent 手里没有论文，它不会停下来问“论文在哪”，而是直接拿训练记忆里关于这个主题的印象开写——三十年前的经典理论、它记混的作者名、根本不存在的期刊，全都能编得像模像样。你拿到一篇三千字读着挺顺，却发现里面一半引用是查不到的，这就是“没材料却硬写”的必然结果。好任务把三件事钉死：材料在工作目录里那 20 篇 PDF、字数 3000、引用必须真实，而且加了一道工序“先列大纲给我确认”——这一道停顿把“它以为你要什么”和“你真要什么”之间的误差提前暴露在两千字草稿之前。坏任务的成本不是写得慢，是写完你才发现要重写。

第二组：“帮我整理一下文件。” vs 按类型分 5 个子文件夹、先盘点、总数一个不能少。

“整理一下”这三个字是最经典的模糊动词。它至少有四种理解：按类型分、按日期分、按项目分、只把乱的摆整齐。你心里想的是按类型分，它可能按日期分，分完你打开一看全是“2025-03”“2025-04”的文件夹，根本找不到东西。更危险的是“整理”二字隐含了“可以删没用的”——它可能把它认为的垃圾直接清了，三个月后你找一份旧合同才发现没了。好任务做了三件事：说死分几类（5 个子文件夹）、加一道停顿（先盘点给我看）、加一条红线（总数一个不能少）。这三条合起来，把“它自由发挥”的空间压到最小，把“可能弄丢文件”的风险用总数对账兜底。

第三组：“帮我查查这个公司。” vs 查主营业务、融资、客户，8 个来源每条带链接。

“查查”这个词的问题在于它没有终点。查多少算够？搜到第一页就算查完，还是翻到第十页？查什么维度——营收、股权、口碑、诉讼？你不说，它就挑最好查、最正面的东西给你：官网首页、两篇公关稿、一段百科简介，凑成一篇“XX 公司是一家领先的……”的正确废话。你看完还是不知道这家公司去年到底融了多少钱、有没有被告过。好任务把“查”拆成可验收的维度清单（主营业务、近三年融资、主要客户），又给了数量和形式（8 个来源、每条带链接、存成 md）。“带链接”这三个字是关键——它逼着 Agent 真的去打开网页，而不是凭印象写。没有这三个字，它编一个官网域名给你，你不点根本发现不了。

第四组：“帮我做个 PPT。” vs 12 页课程汇报、学术风、每页不超 30 字、先出封面目录。

PPT 是最容易“做出来但用不了”的交付物。坏任务没说主题，它就自己猜一个；没说页数，它给你做 25 页，每页挤 80 个字；没说给谁看，它按产品发布会的风格做动画，你拿到课堂上一放全是花里胡哨的转场。PPT 返工的成本极高——格式、配色、每页排版全都要重来。好任务把四个约束一次给齐：主题是什么大纲、12 页、学术风、每页不超 30 字，而且“先出封面和目录给我看”——这一道停顿让你在它做满 12 页之前就把方向拉回来。PPT 这种排版重活，方向错一页就废十页，所以停顿必须放在最前面。

第五组：“分析一下这个 Excel。” vs 找出销售额最高 3 个品类和环比下滑 2 个、数字能复算。

“分析一下”是所有模糊词里最危险的一个。Excel 分析可以做描述统计、可以做回归、可以做可视化，你要哪个？你不说，它就把表打开，随便挑两个列画张图，写一段“总体呈上升趋势，建议关注”——读着像分析，其实什么结论都没有，因为它根本不知道你要回答什么商业问题。更糟的是它可能随手改几个数字凑图，你不复算就信了。好任务把“分析”翻译成两个具体的问题：谁卖得最好、谁掉得最厉害，再要求“数字能复算”——这一条逼着它把公式留在单元格里，你点开能追到源数据。“分析”没有问题指向，就只是“看看表”。

第六组：“帮我翻译一下。” vs 保留方括号引用格式、术语中英对照。

翻译任务看起来简单，其实最容易在格式上翻车。你不交代，它默认“翻成通顺中文”——它会把英文原文里的方括号引用 [12] 改成脚注，把希腊字母 β 译成“贝塔系数”，把专业术语换成它觉得更通俗的说法。等你把译文粘进论文，格式全乱，引用编号对不上，术语前后不统一，你得一句句改回去。学术翻译尤其如此：导师要的是“格式不动、意思准确”，不是“翻得优美”。好任务用两句话把最容易翻车的地方钉死：原文方括号引用格式保留不动、术语首次出现中英对照。这两条加进去，翻译质量它自己就能兜住大半，你只需要检查意思对不对。

第七组：“你自己看着办。” vs 我拿不准的你来提议，但删除、发送、公开发布必须先问我。

这组最特殊——它的坏不在“信息不够”，而在“授权过度”。“你自己看着办”听起来是信任，实际是把判断权全交出去了。Agent 不知道哪些事后果严重，它会替你做决定：它觉得那个文件没用就删了、那封邮件语气不合适就改了重发、那条数据看着可疑就标成“待核实”跳过了。它不是坏心，是你没告诉它“哪些决定不能替我做”。好任务反着写：把“我拿不准的你来提议”这句信任留下，同时把三条不可逆动作（删除、发送、公开发布）单拎出来——这三条必须先问我。授权的艺术不是“全信”或“全防”，是把判断权分层：日常琐事它自己定，踩刹车的事它必须等你。

第八组：“快点做完。” vs 做完的标准是我打开就能直接用、不需要你再解释。

“快点”不是任务，是情绪。它对 Agent 没有任何可执行的信息量——多快算快？十分钟？一小时？它没法因为你说“快点”就真的变快，反而可能为了赶时间跳过验证步骤：不核对引用、不抽查数据、直接把草稿当成品交。你越催，它交得越糙，你返工越久。好任务把“快”翻译成“什么叫做完”：你打开文件能直接用、不需要再让我解释你做了什么。这一句其实是把验收标准从“速度”换成了“成品可用度”。它逼你在发任务时想清楚：我要的到底是快，还是能用？这两件事常常冲突，你得选一个。

第九组：“帮我把这门课复习一下。” vs 12 份 PPT 整理成知识地图、每次课至少 5 个要点、能在原 PPT 找到。

“复习一下”是学生最常说的一句空话。复习到什么程度？能复述？能做题？能应付考试？用什么材料——PPT、教材、笔记、往年题？你不说，它就做一件最省力的事：把 PPT 每页念一遍，画张思维导图给你。你看着那张图觉得“嗯都覆盖了”，一合上书发现什么都想不起来——这就是 LAB A 反复说的“看着懂了”和“想起来了”是两回事。好任务把复习定义成可检查的密度（每次课至少 5 个要点）和可溯源的证据（每个要点能在原 PPT 找到）。“能在原 PPT 找到”这一条最关键——它防止 Agent 用通用常识把 PPT 里没讲的东西补进去，也逼着你最后真去翻 PPT 核对，而不是对着图自我感觉良好。

第十组：“帮我写个调研报告。” vs 围绕“短视频对大学生注意力影响”、8 个学术来源、2000 字、每条结论带可点开链接。

“调研报告”四个字背后可能是创业计划书、课程作业、市场分析、政策建议，每种要的深度和形式完全不同。你不说是哪一种，它就按最安全的写法来：凑一篇两千字的科普文，引言、现状、建议三段，读着像模像样但没有一条数据是真核过的。你拿去交作业，老师问“第三个数据来源呢”，你答不上来。好任务把报告的四个骨架一次给齐：主题范围（短视频对大学生注意力）、来源等级（8个学术来源）、篇幅（2000字）、证据形式（每条结论带可点开链接）。其中“学术来源”和“可点开链接”是两道闸门——前者把它挡在营销稿外面，后者逼它真去读而不是编。调研报告的价值不在文笔，在每句话都站得住。

第十一组：“帮我把这个表弄干净。” vs 空值标黄、重复行列出来、日期统一 YYYY-MM-DD、规则先给我看。

“弄干净”是数据清洗里最要命的模糊词。什么叫干净？空值删了算干净，还是留着标黄算干净？重复行删还是合并？日期有 2025/3/1、2025-03-01、三月一号三种写法，统一成哪种？你不定义，它就按自己的直觉洗——把空行全删了，结果你要的样本少了一半；把重复行合并了，结果两条其实是不同客户。数据清洗最大的事故不是洗错，是洗了你不知道它按什么规则洗的。好任务把“干净”拆成三条具体规则（空值标黄、重复行列出来、日期统一格式），再加一道停顿“规则先给我看”。这道停顿极其重要——数据一旦洗掉就找不回来，所以规则必须在动手之前经你点头，而不是洗完了你看结果。

第十二组：“帮我改一下这个网页。” vs 把首页按钮从“提交”改成“立即报名”、改前备份、改完手机端不溢出。

“改一下”三个字在软件开发里等于没说。改哪里——首页还是内页？哪个按钮？改成什么文案？你不说，它可能自作主张把全站三个按钮都改了，或者改了文案却把手机端布局弄乱了。更常见的是它改完不告诉你改了哪几个文件，你三周后想回退都不知道退哪里。好任务把一个模糊的“改”钉成四件事：改哪个文件哪个元素（首页按钮）、改成什么（立即报名）、安全措施（改前备份）、验收标准（改完本地打开手机端不溢出）。软件开发里“小步改、改一点测一点”是铁律，所以好任务必须把改动范围缩到一个按钮——一次改十个地方，坏了你根本不知道是哪处弄坏的。

把这十二组放在一起看，规律只有一条：坏任务给的是动词，好任务给的是可检查的东西。动词没法验收，只有“数、范围、来源、停顿、红线”这五样能验收。你发任务时问自己一句：“我这句话里有没有至少一样能让我打完勾的东西？”没有，就再补一句。

2.13 三个 Mission 的完整 8 要素表

下面把前面三个 Mission 的 8 要素全部填齐，你可以直接对照着抄改：

Mission 1：整理下载夹

要素	内容
目标	把 D:\Downloads 里的文件按类型分进 5 个子文件夹，一个不丢，根目录有分类说明。
上下文	下载夹堆了三个月，约 847 个文件，混着 PDF/图片/安装包/压缩包/表格/视频。原因是硬盘快满了。已手动备份到 下载_备份_20260919。
环境	只在 D:\Downloads 内操作，不碰其他位置和备份文件夹。
权限	可读、可建子文件夹、可移动文件；不删除；不联网。
约束	总数一个不能少；重名保留两份；不修改文件内容；觉得该删的列清单给我。
过程	先盘点+提方案，我批准后再移动，做完写分类说明。
验收	整理前后总数一致；抽查三个子文件夹分类正确；根目录有分类说明。
交付	分类说明.md 放根目录。

Mission 2: 读 5 份 PDF 做摘要表

要素	内容
目标	一次读完 5 份 PDF，做成一张摘要表。
上下文	5 份课程 PDF 已放进 课程阅读/ 文件夹。这是课程阅读任务，老师要求读完做笔记。
环境	工作目录就是 课程阅读/。
权限	可读 PDF、可新建 摘要表.md；不改原始 PDF。
约束	某份读不出（扫描件/加密）就注明原因，不要编；每份单独一行，不要交叉总结。
过程	先给表头方案，我确认后再逐份读、填表格。
验收	表里正好 5 行；随机抽一份，关键观点在原文中得到；PDF 一个没被改。
交付	摘要表.md 放 课程阅读/ 文件夹。

Mission 3: 联网查资料

要素	内容
目标	围绕“生成式 AI 对大学课堂的影响”查资料，存成带真实链接的笔记。

要素	内容
上下文	我要写一份简短调研，主题完全不懂，需要从可靠来源起步。
环境	新建文件夹 调研_生成式 AI 与课堂/。
权限	可联网搜索、可读网页、可新建 调研笔记.md。
约束	所有链接必须是真打开过的；来源优先官方/学术/有出处的新闻，不要营销号；至少 4 个不同网站。
过程	先报搜哪几个关键词，我确认后再搜，搜完整理成笔记。
验收	每条链接能点开且内容对得上；至少两个官方/学术来源；有一节“还没搞清楚的问题”列 3 条。
交付	调研笔记.md 放工作目录。

2.14 从这一章长出两条主线

学习线：从“帮我学宏观经济”这个模糊愿望，套 8 要素后变成——材料是 12 份 PPT、目标是知识地图、验收是随机抽查能在原文找到。这就是 PART 0 Mission L0 那份问题清单的正式落地。

科研线：从“帮我写综述”变成——材料 20 篇 PDF、红线是不许编引用、流程是证据矩阵→脉络→大纲→正文→核验、交付是三件套。这是后面 PART 3、PART 4、PART 5 所有科研任务的母版。

简单任务先只加三件事就够：“工作目录在哪”“不要删原始文件”“做完我怎么检查”。写满二十次复杂任务，8 要素会自动出现在你脑子里。

PART 3：状态管理——让长任务不会断片

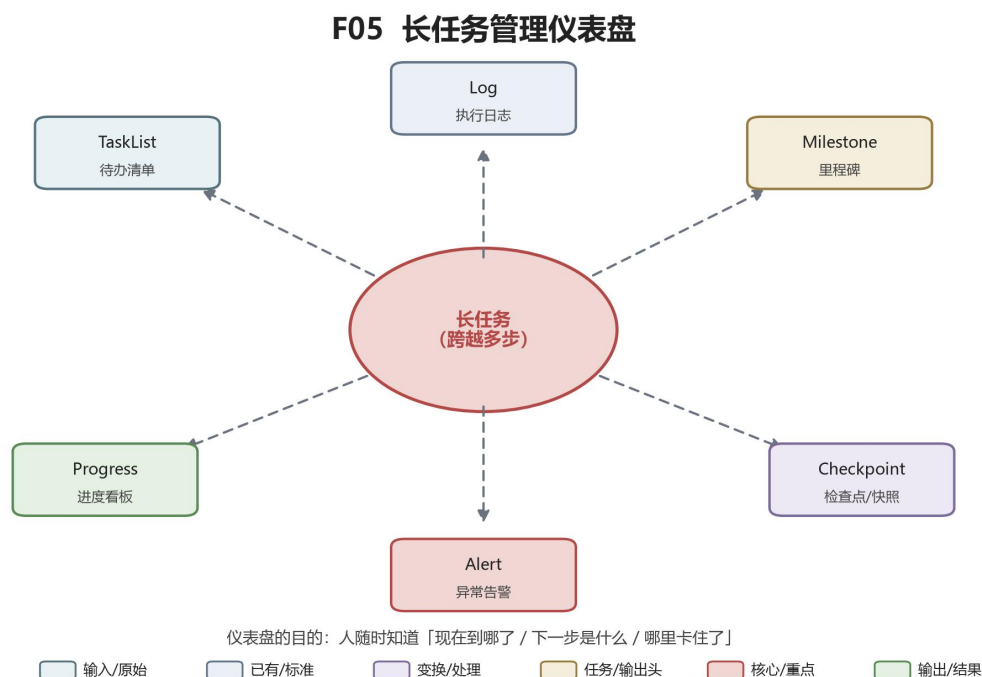


图 8 长任务状态仪表盘

本章的中心概念从“维护一堆文档”换成一个词：**State（状态）**。

它指的是——这个长任务现在进行到哪、下一步做什么、什么卡住了、为什么这么决定，这些信息的总和。PART 6 讲 Agent 内部怎么管 State，本章讲你在工作目录里怎么把它落地成几份看得见的文件。

3.1 本章要解决的新问题

前面的任务都不大：整理文件夹、读五份 PDF、查一个主题。但课程论文要读 30 篇文献写一个月，创业项目要调研、做表、写报告、改五版——这是**长任务**。交给 Agent 后你会遇到：

- 它做了一半，你不知道进行到哪了；
- 做了三天，它说“我忘了最开始要什么”；
- 中途想加一份新材料，不知道怎么接；
- 出来一堆中间文件，分不清哪个是最新的；
- 它跑着跑着重复劳动，同一件事做两遍。

先说清楚一个用词。这里常说“它会忘”——这不是说模型像人一样真的记性变差，而是指：隔了一阵、或新开一个对话之后，之前那些对话里的信息没有进入当前模型上下文（Context Window，一次对话能同时容纳的文字量），所以它“看不到”了。解决办法不是求它记住，而是把状态写到文件里，让它每次开工先读文件。

3.2 为什么不能一口气做完

三个客观限制，不是它不够聪明：

一次能“看到”的内容有限。上下文窗口就像一张桌子，面积有限，30 篇论文一次堆上去摊不开。

任务本身有先后依赖。没读完文献就写综述，写出来也是编的。

错得越早，改得越便宜。它一口气写八千字你才看，方向错了全废；写五百字就停下来给你看，你一句话就拉回来。

所以长任务的核心不是“让它一口气做完”，而是拆小、分步、勤看。

3.3 最小配置：三份就够

F06 多阶段工作流：文献分析八步流水线

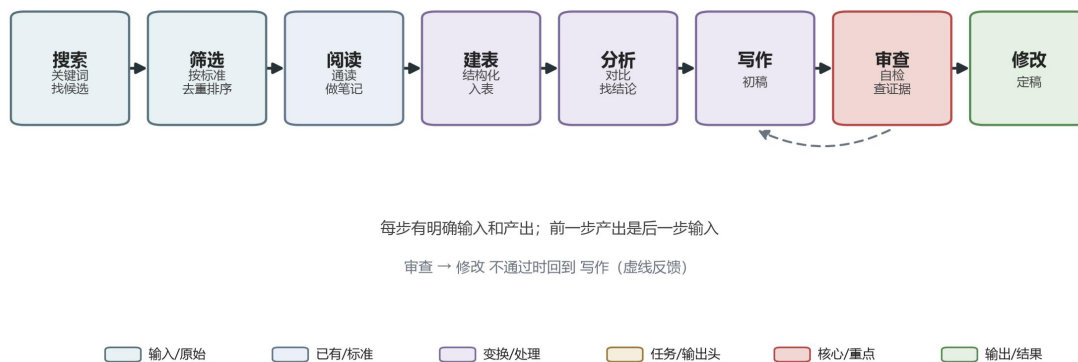


图 9 多阶段工作流示意

不要一上来就搭一套项目管理。普通长任务，工作目录里只需要三份东西：

你的项目 /	
TASKS.md	← 任务清单：拆成可勾选的小项，做完一项勾一项
WORKLOG.md	← 操作流水：每做一步追加一行，记录它实际做了什么
output/	← 成品目录：最终交付物统一放这里

三份各管一摊，合起来就是 State 的全部：

- **TASKS.md** 管“事还剩什么没做”。状态约定：[] 未开始 / [~] 进行中 / [x] 已完成 / [!] 卡住。
- **WORKLOG.md** 管“它的手实际做了什么”。字段：日期 / 操作 / 涉及文件 / 结果 / 备注，失败的步骤也要记。
- **output/** 管“做完的东西在哪”。

下面是这两份文件的实际内容示例，你照这个格式填：

TASKS.md 实际示例（以“读 30 篇文献写综述”为例）：

```
# 任务清单：生成式 AI 与知识劳动综述

## 已完成
- [x] 01 收集 30 篇文献，放进 原始材料/
- [x] 02 逐篇阅读，填证据矩阵（每篇：标题/作者/年份/核心论点/关键证据）
- [x] 03 文献盘点：按主题聚类，标出必读和可略读

## 进行中
- [~] 04 梳理脉络：从证据矩阵里提炼三条主线，写成脉络草稿

## 未开始
- [ ] 05 列大纲：基于脉络草稿，列出论文大纲（节/小节/每节论点）
- [ ] 06 大纲确认：发给刘总确认，根据反馈修改
- [ ] 07 写正文：按大纲逐节写，每节 500-800 字
- [ ] 08 引用核验：逐条核对每个引用是否真实存在、页码对不对
- [ ] 09 写“未解决问题”一节
- [ ] 10 交付：正文.docx + 证据矩阵.xlsx + 引用核验表.md 放进 output/

## 卡住
- [!] 11 第 14 篇文献（Brown 2025）读不出全文，只有摘要—需要刘总决定是找全文还是用摘要
```

WORKLOG.md 实际示例（接上面的任务）：

```
# 操作流水

| 日期 | 操作 | 涉及文件 | 结果 | 备注 |
| --- | --- | --- | --- | --- |
| 2026-09-15 | 新建工作目录，放入 30 篇 PDF | 原始材料/*.pdf | 成功，30 个文件就位 | 其中 3 篇是扫描版 |
| 2026-09-15 | 逐篇阅读，填证据矩阵 | 工作中/证据矩阵.xlsx | 完成 27/30 | 第 14、22、27 篇读不出文字 |
| 2026-09-16 | 第 14 篇（Brown 2025）尝试 OCR | 工作中/Brown2025_OCR.txt | 失败，扫描质量太差 | 标为卡住，等刘总决定 |
| 2026-09-16 | 文献聚类，按主题分三组 | 工作中/聚类草稿.md | 成功，三组：效率/质量/不平等 |
| 2026-09-17 | 梳理脉络草稿 | 工作中/脉络草稿_v1.md | 完成，三条主线 | 等刘总确认后再列大纲 |
```

为什么是这两份文档而不是对话？因为对话会被新对话冲掉，文件不会；对话可以断，文件不会断。每次开工让它先读 TASKS.md 和 WORKLOG.md，它就知道从哪接着走。

3.4 复杂项目再追加三份

任务真的变重了（跨周、多人、有过返工方向），再慢慢加这三份，不要一开始就堆满：

- **DECISIONS.md**：专记“为什么这么定”，不是记做了什么，是记当时为什么放弃另一个方案。
- **README.md**：这个文件夹里都有什么、每个文件干嘛的。三天后的你自己也是“新人”，先读它。
- **CHANGELOG.md**：版本变更。“v1→v2：结论从‘影响显著’改成‘影响有限’，因为第 14 篇文献证据相反。”

3.5 不要为了项目管理而项目管理

这是本章最该记住的一句：这些文件是为了让任务不断片，不是为了让你像项目经理一样打勾。任务短，就两份文档一个 output；真长到跨周再追加。看到 论文_final.docx、论文_final2.docx、论文_真的最终.docx 满天飞，就用编号 初稿_v1.docx、初稿_v2.docx，旧版本留着不删，CHANGELOG 记每版改了什么。

3.6 计划与停顿：先让它列，你再改

不要自己拍脑袋列计划。让它先列，你再改：

我要做的事：[一句话目标]。材料放在 [文件夹]。
请你不要开始做。先给我一份计划：
1. 把这件事拆成 6 到 10 步，每步写清要产出什么；
2. 标出哪几步必须停下来等我确认；
3. 标出哪几步依赖前一步的结果。
计划发给我，我改完再开始。

它列计划的过程本身就是暴露问题——它列出来的步骤里漏了“读文献”直接跳到“写综述”，你一眼能看出来。

停顿分两档：**里程碑（Milestone）**是大节点，每个都对应一个你必须介入的动作（“文献盘点完，你确认哪些必读”“大纲定了，你点头它才写正文”）；**检查点（Checkpoint）**是更密的小停顿，读 30 篇就每读 10 篇停一次，让你抽查质量——它从第 3 篇读偏，你在第 10 篇就能发现。没有你介入的节点不叫里程碑，叫它自言自语。

3.7 中断了怎么续：WORKLOG 与 TASKS 是断点

最难受的不是慢，是任务做到一半突然断了——网络抖了、会话过期、你误关窗口。这时最慌的是：做完的会不会白做？下一步从哪走？

答案还是那两份文件。**先别慌，别把原任务重发一遍**（重发 = 它把做过的又做一遍，还可能覆盖中间文件）。按下面的步骤走：

第一步，你自己先看现场，不要急着找它。做三件事：

1. 打开 工作中/，看已有哪些文件、哪个修改时间最新——那就是它最后碰过的东西；
2. 打开 WORKLOG.md，看最后一行记的是哪一步——那就是断点；
3. 打开 TASKS.md，看最后一个 [~] 或 [] 的是哪项——那就是下一步。

把这三个信息记在心里：断在哪、最新文件是什么、下一步编号是几。

第二步，新开对话，把断点喂给它。发：

我们的任务中断了，工作目录是 [路径]。
请你先读这三份文件，不要开始任何新动作：
1. WORKLOG.md — 之前实际做了哪些步骤、最后停在哪；
2. TASKS.md — 哪些已完成、哪些还没做；
3. 工作中/ 里最新的中间文件。
读完告诉我：上次精确断在哪一步（引用 WORKLOG 最后一行）、下一步做什么、有没有卡住待我决定的。
你说完，我确认后再继续。

第三步，它报断点，你核对。它会告诉你“根据 WORKLOG，上次停在 9 月 17 日，脉络草稿完成了，下一步是第 05 项列大纲”。你对照自己刚才看的现场——它说的和你看到的一致吗？一致就放行，不一致就指出“不对，WORKLOG 最后一行不是这条，你再看看”。

第四步，确认后让它续跑。发：

断点确认无误，从 TASKS.md 第 [N] 项继续。
1. 不要重做已经打勾的步骤；
2. 做完一项把 [] 改成 [x]；
3. 每做一步往 WORKLOG.md 追加一行；
4. 发现之前某个中间文件是错的，先告诉我，不要直接覆盖。

WORKLOG 是“它的手”做过的事，TASKS 是“事”还剩什么，DECISIONS / PROGRESS 管“为什么这么走”。三份合起来，就是一串不会断的实物锚点——这就是 3.1 说的 State 落地：对话可以断，文件不会断。

3.8 中途改需求、加新材料

长任务几乎一定改需求。别直接在旧任务上接一句“不对，改成 XX”，要说清哪些保留：

任务变更：原计划第三步写大纲，现在改为大纲写完后先加一步“找一份最新文献补进证据矩阵”，再写正文。

前面已做的（01、02）保留，不用重做。

请更新 TASKS.md 和 WORKLOG.md，告诉我新计划。

中途找到重要新文献也别甩一句“加进去”。关键是第三条——**新材料只增量处理，不要触发它重写一切**：

新材料：我刚放进工作目录的 new_paper.pdf，2026 年最新文献，和主题直接相关。

请你：1. 单独读这一篇填进证据矩阵；

2. 判断它对现有大纲的影响——支持就加一条引用，反驳就标出来我们再讨论；

3. 不要重写已经写完的部分，除非我明确说改。

防止重复劳动还有一句必须养成习惯：每次开工前让它先读 TASKS.md 和 工作中/，然后问“基于已有进度，下一步是什么”，并在任务里写“已有中间成果先读，不要重做”。

3.9 长任务管理的真实工作流：以“读 30 篇文献写综述”为例

下面用一个完整的五天工作流，把本章所有方法串起来走一遍。不是理论，是你照着跑就能跑下来的流程。

第一天：起盘子。新建工作目录，结构如下：

生成式 AI 与知识劳动综述/	
原始材料/	← 30 篇 PDF 放这里
工作中/	← 中间产物
交付/	← 最终成品
TASKS.md	← 任务清单
WORKLOG.md	← 操作流水

先发计划请求，让它拆 6 到 10 步。你改完计划，写进 TASKS.md。第一天只做两件事：放好 30 篇 PDF，让它逐篇读、填证据矩阵。每读 10 篇停一次给你抽查——第 1 到 10 篇读完，你抽两篇看它写的关键观点对不对。

第二天：继续读 + 聚类。读完剩下 20 篇。读完后让它把 30 篇按主题聚类，分成三组或四组。这一步停一次给你看：聚类合理吗？有没有把两篇立场相反的文献分到一组？你说“第三组拆成两组”，它就拆。

第三天：梳理脉络 + 列大纲。基于聚类，让它提炼三条主线，写成脉络草稿。你看完脉络草稿说“第二条主线展开不够，加一节对比”，它改。然后让它列详细大纲（节、小节、每节论点），你确认大纲后才放行写正文。这是整个任务最关键的里程碑——大纲错了，后面全白写。

第四天：写正文。按大纲逐节写。每写完一节停一次给你看，不要等它一口气写完 8000 字。第一节写完你看：语气对不对？引用格式对不对？有没有编引用？发现问题立刻拉回来，别等它写完全部再返工。

第五天：核验 + 交付。正文写完后，让它做引用核验——逐条核对每个引用是否真实存在、作者年份对不对、页码对不对。核验完把三件套（正文.docx、证据矩阵.xlsx、引用核验表.md）放进交付/。最后你自己随机抽三个引用，去原文里翻，看它写的和原文对不对得上。

中断了怎么办。比如第三天晚上你电脑关机了，第四天早上新开对话。按 3.7 的步骤：你先看 工作中/ 里最新文件是 脉络草稿_v2.md（昨晚改到第二版），WORKLOG 最后一行是“2026-09-17 脉络草稿第二版完成，等刘总确认”，TASKS.md 里第 04 项打勾了、第 05 项“列大纲”还是 []。新开对话把断点喂给它，它读完三份文件告诉你“上次停在脉络草稿确认后，下一步是列大纲”。你说“对，继续列大纲”，它接着跑。

这个流程的核心不是步骤多，而是**每一步都有实物锚点**——文件在硬盘上，TASKS.md 在硬盘上，WORKLOG.md 在硬盘上。对话断了、关了、换了新对话，这些东西都在。它下次开工先读这些文件，就知道从哪接。

3.9.1 五天日志实录：每天到底发生了什么

上面五天工作流是骨架，真跑起来每天会遇到具体的卡顿、抽查、返工。下面把这五天的 WORKLOG 真实写法逐日贴出来，你照着这个颗粒度记，就知道“记录到什么程度才算够”。

Day 1 日志（9 月 15 日，起盘子）。

早上先建目录结构，把 30 篇 PDF 复制进 原始材料/。中午发计划请求，它拆了 8 步，我把第 6 步“写正文”拆成两节分别写，加了一条“第 4 步大纲必须我确认后才写第 7 步”。下午开始读文献，它读到第 8 篇时停了——第 14 篇 Brown 2025 是扫描版读不出文字，它按规矩在 WORKLOG 记了一行“读不出，扫描质量差”，没有编。我抽查第 3 篇和第 7 篇，第 3 篇的关键观点对得上，第 7 篇它把“作者认为 X”写成了“作者认为 Y”，我标了回来。晚上它把第 1 到 10 篇的证据矩阵填完，TASKS 里第 02 项打了半勾。今天最该记住的一件事：抽查第 7 篇时发现它把相反观点写反了——这就是为什么要“每 10 篇停一次”，不停的话这种错误会留在矩阵里一直误导后面的脉络。

Day 2 日志（9 月 16 日，读完+聚类）。

上午把剩下 20 篇读完，证据矩阵补满 30 行（其中 3 行标注“读不出”）。下午让它聚类，它分成三组：效率提升、质量变化、劳动不平等。我看了一眼——第二组里混了两篇讲“质量下降”的，和“质量提升”立场相反，我让它把第二组拆成“质量提升”和“质量风险”两组，变成四组。它拆完后重新更新了矩阵的聚类列。今天的关键停顿在聚类这一步：聚类是主观判断，你

不看一眼，它可能把对立观点塞一组，后面脉络就歪了。晚上它把聚类草稿存成 工作中/聚类草稿_v1.md，WORKLOG 追加一行“聚类完成，四组，等确认”。

Day 3 日志（9月17日，脉络+大纲）。

上午基于四组聚类，让它提炼三条主线：第一条是“AI 提升效率但不改变核心劳动”，第二条是“质量提升伴随新的不平等”，第三条是“技能要求的迁移而非替代”。我看完说第二条展开不够，需要加一节“哪些岗位受影响最大”，它补了一版脉络草稿 v2。下午让它列详细大纲：三节、九小节、每节论点一句话。我审大纲时发现它把第三条主线（技能迁移）只配了半页，而第一条配了三页——比例失衡，让它调成 2:2:1。大纲定稿后我才放行写正文。今天是整个任务最关键的里程碑：大纲错了，后面 8000 字全白写，所以我在这里停了两次，改了两轮。

Day 4 日志（9月18日，写正文）。

按大纲逐节写。第一节写完我看：语气太像新闻综述，不像学术，让它把“我们认为”改成“本文认为”，加两处文献支撑。第二节写完检查引用——它引了一篇 Smith 2024，我去矩阵里查，矩阵里根本没有这篇，它从哪冒出来的？它承认是“根据上下文补的”，我让它删掉换成矩阵里有的一篇。这就是“不许编引用”红线在实战中起作用的时刻。第三节、第四节写完，每节停一次。晚上写完 6000 字，我没有让它一口气写完，而是四节分四次停下来看。今天最累的是第二节那个假引用——如果我不逐条回矩阵核对，这篇综述交上去就出事了。

Day 5 日志（9月19日，核验+交付）。

上午让它做引用核验：逐条核对正文里每个引用是否在矩阵里、作者年份对不对、页码对不对。它核出三处：一处页码写错、一处作者名缩写不一致、一处引用了矩阵外的一篇（就是昨天第二节那个假引用，它自己也标出来了）。修正完，把三件套（正文.docx、证据矩阵.xlsx、引用核验表.md）放进 交付/。下午我自己随机抽三个引用去原文翻：第一个翻到了、页码对；第二个翻到了、年份对；第三个是昨天那个假引用，已经删了。TASKS 全部打勾，WORKLOG 最后一行写“交付完成，三件套已放入交付/”。五天跑完，最大的体会是：不是每一步都要我盯，但每个里程碑我必须亲手看一眼——五天里我停了六次，这六次拦下了聚类歪、大纲失衡、语气不对、假引用四个真问题。

3.10 Mission：跑一条两阶段长任务

难度 L5，长任务，含中间检查。

目标：用本章方法跑完“搜索→筛选→阅读→建表→小结”五阶段，中途至少停两次。

上下文：一个你真感兴趣的小主题，如“短视频对大学生注意力的影响”。

环境：新建工作目录，里面先放好 TASKS.md 和 WORKLOG.md 两份空模板。

过程：五阶段，搜索完停一次给我看候选清单，筛选完停一次给我确认入选，再开始阅读建表。

验收：中途至少停两次且你都给了反馈；TASKS.md 和 WORKLOG.md 真的按记录更新了；证据表里每条链接能点开；隔一天新开对话，你能靠读文件接上。

发：

主题：[你的主题]。工作目录：[新建文件夹]。

计划：把主题拆成搜索、筛选、阅读、建表、小结五阶段。

搜索完成后停一次，把候选清单给我；筛选完成后停一次，把入选清单给我确认；然后再阅读建表。

全程在 TASKS.md 里勾进度，每做一步往 WORKLOG.md 追加一行。

所有结论必须有来源链接，不许编。

忍住别催它“快点做完”。它停下来的每一次，都是在帮你少返工。跑完随机挑证据表一条，点链接读原文，看它写的结论是不是原文真说的。

到这里，PART 0 到 PART 3 结束。你已经完成从“会聊天”到“会进入环境、会交代任务、会管状态”的基础跃迁。接下来 PART 4 专门解决一件事：它说做完了，你怎么知道它真的做完了？

PART 4：证据——怎样判断 Agent 真的做对了

F07 验证三层证据模型（金字塔）

「它说做完了」不算证据；三层都站得住才算完成

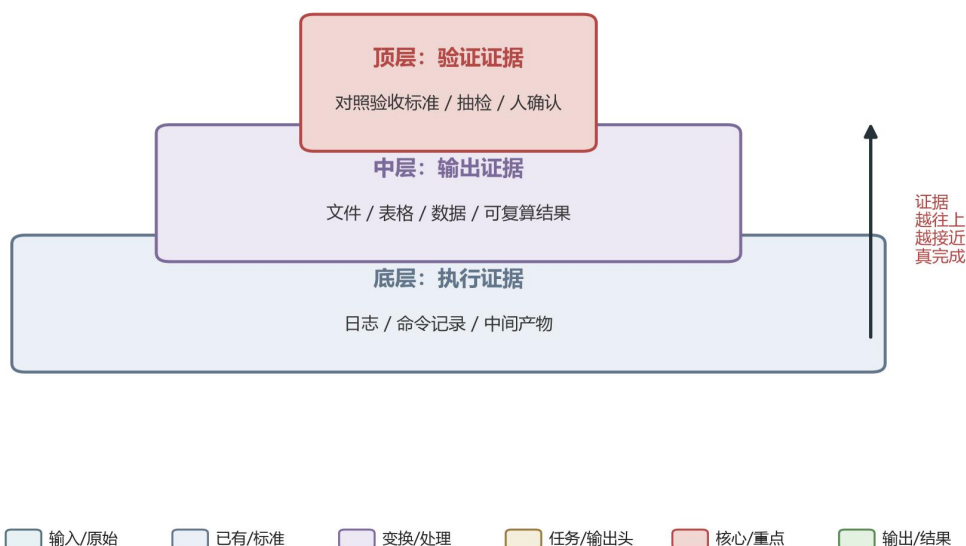


图 10 验证三层证据模型

全书总模型主线走到这里是“观察→验证→修复”这一环。前三个 Part 教你把任务交出去、把长任务管起来，本 Part 只解决一件事：**Agent 说它做完了，你凭什么信它。**

4.1 “完成声明”不是完成证据

你让 Agent“读 20 篇论文做证据矩阵”。两小时后它回：“已完成，output/evidence_matrix.xlsx，共 20 行。”你打开确实有 20 行，随便抽一篇对照原文——年份写错、观点张冠李戴、有个 DOI 根本打不开。

Agent 没骗你，它真的“以为”自己做完了。这就是本 Part 要解决的核心问题：**不被“完成声明”骗过，亲手确认结果真的对。**

人对自己的活有“体感”——你知道昨天那篇综述哪段是硬凑的。Agent 没有这种体感，它不会心虚。所以“完成”这件事必须拆成三层证据：

执行证据：它真的打开了那 20 个文件吗？操作流水账是什么？

结果证据：产出物真的落在磁盘上吗？打开看了吗？内容对得上吗？

验证证据：有没有独立手段证明结果是对的？（回源、复算、换个模型审）

任务完成声明只是状态信息，不是完成证据。真正完成 = 执行证据 + 结果证据 + 验证证据。

四条铁律在本 Part 的具体化

全书四条铁律在这里落地成各工种的版本：

总铁律：

生成结果 ≠ 完成任务
执行动作 ≠ 得到正确结果
一次验证通过 ≠ 整体可靠
Agent 声明完成 ≠ 实际完成

科研版：

找到文献 ≠ 读过文献
生成引用 ≠ 引用真实
跑出数字 ≠ 研究结论成立

数据版：

画出图 ≠ 数据正确

软件版：

代码生成 ≠ 功能完成
代码修改 ≠ Bug 修复
测试通过 ≠ 产品完全正确

一句话口诀：**说完成不算数，拿出来给我看**。每当 Agent 说“做完了”，问三个问题——你动了哪些文件？产物在哪？你怎么知道是对的？这对应 8 要素任务法里的第 7 项“验收”。

4.2 三类证据怎么留痕

执行证据：操作流水账

Agent 做完就“忘”了自己刚干了什么，所以必须显式留痕。三种形式：

- **执行轨迹（Trajectory）**：平台自动记录的“想做什么→调了什么工具→返回了什么”。这是最硬的执行证据，去你的 Agent 平台找“操作历史/对话历史”面板。
- **操作日志（WORKLOG.md）**：你要求它边做边追加，不是最后补写。格式 [时间] [动作] [文件/URL] [结果/异常]。事后补写的日志细节模糊，不可信。
- **中间文件**：边做边存，不要最后一次性生成。

关键要求：**失败和异常必须记录，不得只记成功**。它说“已读取、已分析、已完成”三行、没有任何文件名，等于没记。打开失败的文件、404 的网页它不提，你就以为全部成功了。

结果证据：东西要落盘

最常见的错是把对话框里一大段文字当交付——刷新就没了，无法审阅、无法存档。要求：

- 产物全部进 output/ 目录，原始材料只读；
- 结束时给一张交付清单：文件名、路径、行数/字数、一句话说明；
- 清单上有但文件夹里找不到的，或 0 字节、占位文字的，都不算交付；
- 正文引用的表格、附件必须真的存在于同目录，不能悬空（正文写“见表 3”，表根本没生成，就是悬空）。

验证证据：用独立手段证明它对了

这一层分三档强度：

弱：Agent 自己说“我核对过了”——这仍是声明，不算验证
中：回到原始材料，抽查 3-5 个事实点
强：用另一条独立路径重算/重查，两次结果一致

验证手段按任务类型分四类，下一节讲。

4.3 四类任务，四种验证方法

原教材把引用、网页、数据、代码、测试拆成八节并列，信息重复。其实按任务性质归四类，每类的验证重心不同。

4.3.1 来源型任务：论文 / 网页 / 政策 / 公司信息

典型场景：文献综述、行业调研、政策梳理、公司背景调查。

验证重心是**回源**——每个关键事实能指回一个具体出处。

验证点	怎么做
文献真实存在	每条引用附可打开 URL；DOI 在 <code>doi.org/<DOI></code> 必须跳转；作者、年份、期刊、卷期、页码与原文逐字一致
网页来源可靠	关键事实附 URL + 发布日期；优先原始出处（年报/官网/统计公报）> 主流媒体 > 自媒体；新闻转引要追到原始数据
时效正确	旧新闻不能当现状；网页会改版下架，注意发布时间
内容一致	打开链接亲眼看，确认数字确实写在那个页面上，而不是它从别处看来后自己拼的

高危点：**格式逼真的假引用肉眼无法分辨**。模型训练时见过海量“像引用的字符串”，它最擅长生成格式完美、作者像真、期刊权威的假文献——真实学者 + 不存在的论文、真实期刊 + 不存在的期号、DOI 格式对但打不开。对策只有一条：**任何一条引用，它给不出能打开的链接，就当它是编的**。把标题加引号到 Google Scholar 精确搜索，把 DOI 贴到浏览器当场点。

科研版铁律在这里的含义：它“找到”一篇论文，不等于它读过这篇论文的原文；它“生成”一条引用，不等于这条引用真实存在；它“跑出”一个相关数字，不等于你的研究结论成立。

完整验收案例：核验 Agent 找来的一篇论文。 综述里多了一条 Chen & Lee (2023), Deep Learning Bias, Journal of AI Ethics, 12(3), 45-60。验收走五步，每步打勾才算过：

1. 点 DOI：DOI 贴进 `doi.org/<DOI>`，能跳转到出版社页面才算；跳成 **404** 就是编的。
 2. 核作者：Google Scholar 搜作者名，看他是否真发过这篇、单位对不对得上。
 3. 核卷期页：翻该刊 2023 年第 12 卷第 3 期目录，确认确有这个页码区间的文章。
 4. 核观点：下载 PDF，Ctrl+F 搜 Agent 转述的关键词，确认原话真在那一页。
 5. 记结果：五步全过打勾；任一步不过，整条引用删除，不留“大概是真的”。

本例实际结果：作者是真的，但那一期没有这篇文章，DOI 格式正确却指向另一篇论文——正是“真实作者 + 不存在论文”的典型假引用，整条删除重找。

第二个验收案例：核验 Agent 找来的一份政策文件。 你让它“整理 2024 年以来关于校外培训的最新政策”，它交来一份政策梳理，引用了三份文件：《关于规范校外培训收费的若干意见》（教监管〔2024〕7 号）、《校外培训机构预收费监管暂行办法》（教监管〔2024〕12 号）、《关于加强校外培训从业人员资质管理的通知》（教监管厅函〔2024〕35 号）。文件名、文号、发文机关都像模像样。验收走五步：

1. 核文号：去教育部官网“政策文件”栏，按文号 教监管〔2024〕7 号 精确检索。
检索不到就是编的——真实文号在政府站内一定能按号查到。
 2. 核标题：把文件全名加引号到百度/谷歌精确搜，看有没有官方转载。
只有自媒体文章在转、官网查不到，就是假文件。
 3. 核发文机关：文号里的“教监管”是教育部校外教育培训监管司的代字，
去该司栏目看 2024 年发文目录，逐条对。
 4. 核时效：真文件会注明“自 X 年 X 月 X 日起施行”“有效期 X 年”，
它转述的条款和原文逐字对；过期文件要看是否已被新文件废止。
 5. 核条款：挑它引用的那两段核心条款，在原文 PDF 里 Ctrl+F 搜关键词，
确认原话真在那一版、那条里。

本例实际结果：第 1 份文件真实存在，文号、标题、条款都对得上；第 2 份《预收费监管暂行办法》文号在教育部站内查不到，加引号搜只有几篇公众号文章在转，原文 PDF 根本下载不到——是它把 2022 年一份征求意见稿改了年份编的；第 3 份更离谱，“教监管厅函”这个代字根本不存在，教育部行文不用这个格式。三份里废了两份，让它重查，并且这次要求“每条政策必须能在官网按文号打开，打不开的不许写进报告”。

4.3.2 数值型任务：Excel / 数据分析 / 统计

典型场景：问卷分析、销售报表、文献计量、社会科学统计。

验证重心是**样本量 + 计算过程 + 独立复算**，三道关：

- 第一道 数据来源：

它用的是不是你给的那份原始数据？行数、列名、时间范围对不对？
有没有它“顺手补充”的数据？（这很危险）

第二道 计算过程：

用了哪一列、怎么过滤的、公式是什么，必须写清；
代码和真实运行输出都要贴，不许只给结论。

第三道 独立复算：

你用 Excel 函数（=AVERAGE、=STDEV.P、=COUNTIF）手算 2-3 个关键数；
或让它用第二种方法（透视表/另一段代码）重算，两次一致才算过。

必查样本量：报告说“有效样本 4800”，原始表 5000 行，它必须说清删了 200 行的理由——清洗过程不透明就是有问题。它可能在“清洗”时悄悄删掉不喜欢的行，样本量就变了。

反常识测试：故意改一个输入单元格，看它的报告变不变。变了才说明它真跑了你的数据；不变，说明它在凭感觉写数字。

数据版铁律：图画得再漂亮，不等于底下的数据是对的。

完整验收案例：独立复算一份问卷分析报告。 Agent 交来报告称“有效样本 4800，平均年龄 28.3 岁，月均消费 3200 元”。复算四步：

1. 核样本量：打开原始 CSV 数总行数，5000 行就要追问删的 200 行去哪了，让它贴出过滤条件（哪一列、什么规则），不许只说“清洗了异常”。
2. 抽复 2-3 个关键数：在 Excel 里 =AVERAGE(年龄列)、=COUNTIF(消费列,“>0”) 手算一遍，和报告对；对不上就打回。
3. 反常识测试：把某个输入单元格改成明显错的值（年龄填 999），重跑报告——平均年龄必须明显跟着变；不变 = 它根本没跑你的数据。
4. 查异常值：看它排没排年龄 200、收入 -5000 这种脏值，没排就是漏检。

本例实际结果：样本量对，但它把两行测试账号（年龄 99、消费 99999）算进了均值，月均消费被拉高 400 元；反常识测试一跑，它才承认没做异常值清洗。

第二个验收案例：独立复算一份销售分析报告的结论。 Agent 交来一份 销售季度分析.md，写着三个结论：“Q3 华东区销售额同比增长 22%，是全网增长最快的大区；线上渠道占比首次过半，达到 53%；毛利率从上季度的 31% 提升到 34%，主要靠高毛利新品拉动。”报告附了一张图和一段“分析代码”。复算四步：

1. 回到底表：让它指出这三个数字分别从哪张表、哪一列、哪段代码算出来的，不许只给结论。它贴了一段 pandas 代码，但没贴运行输出——先要求把完整输出贴上来。
2. 手算 22%：打开原始销售表，筛华东区 Q3 今年和去年同期，用透视表数一遍。今年 X 元、去年 Y 元， $(X-Y)/Y$ 自己算——是 22% 还是 18%，当场见分晓。
3. 手算 53%：把“线上/线下”那列单独做个数据透视，看线上是不是真的过了一半。注意它有没有把“线上+混合渠道”全算成线上——这是常见的口径偷换。
4. 手算 34%：毛利率 = $(收入 - 成本) / 收入$ ，把这三列挑出来重新算一遍。它说“主要靠新品拉动”，就让它把新品和老品的毛利率分开算，新品毛利到底贡献了几个百分点，没数就把这句删掉。

本例实际结果：22% 这个数，手算后发现它把去年 Q2 当成了去年 Q3 来比，口径错了，真实同比是 18%；53% 这个数，它把“混合渠道”全归到线上，真实纯线上是 47%，并没过半；34% 这个毛利率本身对，但“靠新品拉动”那句没数支撑——新品毛利率只比老品高 1.5 个百分点，贡献不到总毛利变动的一半。报告里三个结论两个半站不住，打回重算，并且在任务书里加一条：“凡是报增长率、占比、毛利率，必须附底表透视截图和计算公式。”

4.3.3 操作型任务：文件整理 / Office / 浏览器操作

典型场景：整理文件夹、批量改 Word/Excel、浏览器填表、发邮件。

验证重心是**实际状态变化、前后对比**。这类任务 Agent 最容易“动作做了，但做错了”——建了文件夹、移了文件，但把毕业设计和银行卡截图一起塞进了“杂项”，同名文件还互相覆盖了。它做到表面动作就认为任务完成，不知道“毕业设计比临时压缩包重要”。

验证方式：

- 动手前先让它**列操作计划**，尤其涉及移动、重命名、覆盖时；
- 做完后要**前/后对比**：动了哪些文件、从哪移到哪、改了什么内容；
- 抽查边界情况：重要文件、同名文件、大文件、特殊格式；
- 原始文件只读，让它在副本上操作。

它过程中没问你“这个特殊情况怎么办”，往往就是它自作主张处理了——这是危险信号。

完整验收案例：验收一次文件夹整理。你让它把 待整理/ 按类型归档到 归档/。验收靠一张前后对比清单：

动手前先留底：

整理前跑一遍 → 把待整理/ 里所有文件路径存成 **before.txt**（含文件名、大小）。

做完后逐项对：

- [] 文件总数对不对：**before.txt** 计数 == 归档/ 计数 + 回收站计数，不许凭空少。
- [] 抽样 5 个文件：随机挑 5 个，看它放到的类别是不是你要的。
- [] 同名文件：有没有两个同名文件被覆盖成一个，大小变没变。
- [] 重要文件：毕业设计、合同、表格这种高价值文件有没有被塞进“杂项”。
- [] 空文件夹：它新建的类别文件夹是不是空的。

本例实际结果：文件总数对，但它把两份同名 **report.docx** 直接覆盖了，只留了后一份；高价值的 **合同.pdf** 被归进“杂项”——靠这张清单当场抓出来，从备份恢复。

第二个验收案例：验收一次浏览器自动化填表操作。你让 Agent 用浏览器自动登录某在线系统，把 20 条客户信息逐条填进表单并提交，最后回“已全部提交完成”。这类任务最危险的不是“没做”，而是“做了但做错了——提交到别人的表单、填错了字段、重复提交了三条”。验收不靠它的口头总结，靠四步实物核对：

1. 看它停在哪一步：它说“全部提交完成”，先让它贴出浏览器自动化工具的完整运行日志，含每一次点击的时间戳、URL、按钮文字、HTTP 响应码。
没有日志就是没做，日志被裁掉中间段的也不可信。
2. 数提交次数：日志里搜“提交/Submit”按钮被点了几次——应该是 20 次。
出现 23 次就是重复提交了 3 条，出现 17 次就是漏了 3 条。
3. 抽样打开后台：随机挑 3 条记录，登录系统后台按客户名搜，看它填的字段（姓名/电话/金额）和你给的原始 Excel 逐格对。
特别注意“金额”这种数字列——它最容易把“1,200”填成“1200”或“12”。
4. 看异常分支：它有没有遇到过验证码、二次确认弹窗、“是否重复提交”提示框，这些地方它是停下来问你，还是自己点了“确定”？
日志里搜“confirm/alert/验证码”，没看到这些关键词不代表没遇到，可能是它直接点过去了。

本例实际结果：日志显示“提交”按钮被点了 22 次——第 3 条和第 11 条因为网络慢它点了两次，后台现在有两条重复记录；抽样核对时发现第 7 条金额把“12,000”填成了“1200”，差了一个数量级；更要命的是它在第 15 条遇到一个二次确认弹窗，日志里只有一行“已处理”，没写它点了“确定”还是“取消”。当场让它把两条重复记录从后台删掉，金额填错的那条改回来，并在任务书里加一条：“遇到任何弹窗、二次确认、验证码，一律停下来等我，不许自己点。”

4.3.4 软件型任务：代码 / 应用 / 网页

典型场景：写脚本、改网页、做小应用、修 Bug。

验证按软件工程的测试金字塔走，从轻到重：

Run（运行）：代码必须真跑一遍，贴完整终端输出（含报错），不是只贴代码
Smoke（冒烟）：主流程跑一遍，确认没塌
Test（测试）：它说“测过了”不算数，要列出“点了什么→看到什么”的可复现步骤
Regression（回归）：修 A 的同时把 B、C 也试一遍，确认没改坏
UI 操作：关键功能你亲手点一遍，尤其异常路径（空输入、乱点、断网）

三个高频坑：

1. **代码从未运行**：语法有隐藏错误，一跑就崩。它贴的“输出”只有自己写的“已完成”三个字——不是终端日志。
2. **跑了但结果不对**：退出码 0 只代表“程序没崩溃”，不代表“结果对”。
3. **在它的沙箱里跑通，到你的机器上坏**：环境不一样。

软件版铁律：代码生成 ≠ 功能完成；代码修改 ≠ Bug 修复；测试通过 ≠ 产品完全正确。

完整验收案例：验收一段新写的数据清洗脚本。它交来 `clean.py`，说“已跑通”。验收按测试金字塔逐层做：

Run（运行）：
你自己在本机敲 `python clean.py`，贴出完整终端输出（含最前面的命令和退出码）。
跑不动、报 `ModuleNotFoundError`、报路径错 = 没在你环境跑通过。

Smoke（冒烟）：
主流程喂一条最小样例（1 行数据），看它能不能跑完不崩、产出 `output.csv`。

Test（测试）：
它说“测过”不算，要它列出可复现步骤：“输入 X → 预期输出 Y → 实际看到 Z”。
自己按步骤点一遍。

Regression（回归）：
这是改老代码，就把原来正常的那几个功能也跑一遍，确认没改坏。

UI/边界：

空输入、缺列、乱码路径各试一次，看它是报错退出还是崩给你看。

本例实际结果：它贴的“输出”只有三行“运行成功”，自己一跑 `clean.py` 报 `FileNotFoundError`——它在沙箱里用的路径和你本机不一样，根本没在你环境真跑过。

第二个验收案例：验收一次网页 UI 改动。你让 Agent 改一个内部网页：“把‘提交’按钮从蓝色改成绿色，点完按钮后在右上角弹一个‘提交成功’的提示条。”它改完说“已完成，两个改动都生效了”。UI 类任务最容易“它说生效了，你打开一看还是老样子”，因为浏览器有缓存、它改的是 dev server 不是你看到的页面。验收按五步走：

1. 强制刷新：Ctrl+F5 清缓存硬刷，或开一个无痕窗口打开这个页面——缓存会骗你看到“旧版已生效”的假象。
2. 亲眼看按钮：找到“提交”按钮，看它的颜色是不是真的变成绿色了，还是只变了 `hover` 状态、默认状态还是蓝的。
3. 亲手点一次：真的点下去，看右上角是不是真的弹出了“提交成功”提示条，提示条多久消失、能不能手动关掉——它只说“加了提示”不算，要真弹出来。
4. 试异常路径：故意提交一条空表单、一条超长内容，看它是不是还正常报错、没把新改的提示逻辑搞坏——这就是回归测试。
5. 看它改了哪几个文件：git diff 一下，确认它只动了你让它动的那两个文件，没有顺手改样式表别的地方。

本例实际结果：硬刷之后按钮颜色确实变了，但 `hover` 状态没改，鼠标悬上去还是蓝色——它只改了默认样式没改 `hover`；点了按钮，右上角确实弹了提示条，但提示条不会自动消失，挡着后面的表单；更糟的是第 4 步回归测试，故意提交空表单，原来应该弹“请填写姓名”的红字提示，现在被新加的“提交成功”提示条盖过去了，用户根本看不到红字。它以为“加了新功能”就算完，不知道新功能把老功能的报错盖住了。打回，让它把提示条加自动消失、把空表单报错优先级提到提示条之上，改完再回归一遍。

4.4 验证强度三级：自测 → 交叉核验 → 独立审查

四类任务讲完“验什么”，这一节讲“谁来验”。强度从弱到强。

第一级：对抗式自测

不要只说“你检查一下”，那会变成自夸。把它从“作者”切换成“审稿人”：

现在请你切换角色：你是最严格的审稿人/挑刺用户，只列问题，不许表扬。至少列 5 条，按严重/中等/轻微分级——哪些事实可能错、哪些引用没来源、哪些数字可疑。列完逐条修复，再交。

要点：要求它**找问题而不是找优点、报告问题而不是藏问题、至少 5 条且必须有严重级**（否则它会列“建议微调语气”这种废话）。挑了问题不修的，打回。

自测的标准动作分四步，别只发一句“你检查一下”：

1. 通读：让它先完整重读自己的产出，不许跳着看。
2. 挑刺：按“事实错误 / 引用无源 / 数字可疑 / 逻辑跳跃”四类逐条列，分级。
3. 修复：每条问题给出具体改法，不许只说“此处需完善”。
4. 复验：修完再自查一遍，确认改对了、没改坏别处。

第二级：交叉核验

同一个事实，只查一个来源可能被骗（那个来源本身就错）。

- 关键事实至少**两个独立来源**——独立的意思是不同作者、不同机构，不是 A 网站抄 B 网站；
- 数据任务里，交叉核验 = 两种方法算同一个数；
- 两个来源矛盾时，以更权威的为准，并在报告里写明矛盾，不许挑一个顺眼的写、不提另一个。

伪双源是最常见的作弊：表面上两个链接，其实抄的是同一个原始出处（新闻 A 转引新闻 B，两个链接一个源头）。追原始出处才能识破。

交叉核验的标准动作：

1. 列关键事实清单：把报告里的硬数字、硬结论单独摘出来，别整份都核。
2. 找第二来源：每个关键事实再找一个独立出处（不同机构、不同作者）。
3. 比对：两个来源数字一致才过；不一致就追原始出处，不挑顺眼的写。
4. 记录矛盾：报告里写明“来源 A 说 X，来源 B 说 Y，本文采用 X 因为……”。

第三级：独立审查（换模型）

自己审自己总有盲区。最强验证是主 Agent 干活，另一个模型当质检员：

主 Agent 生产（写综述/做分析/改代码）
↓
开新对话、换一个模型，只给它最终产物 + 原始材料
（不要给它看主 Agent 的自检报告，更不要说“这是我做的”）
↓
让它独立挑错、独立核对
↓
两份结果对比，分歧处你人工裁决

这就是 Planner / Executor / Evaluator 分离：干活的和检查的不是同一个人。换模型尤其有效——不同模型的错误模式不同，A 模型容易编的地方，B 模型未必跟着编。生产用快模型、审查用强推理模型，可以形成固定流水线。

审查对话里不要替主 Agent 辩护，否则审查会变客气。审查者太弱挑不出错时，换更强的模型再审。

独立审查的标准动作：

1. 开新对话、换模型，只给它两样东西：最终产物 + 原始材料。
2. 不要给它看主 Agent 的自检报告，更不要说“这是我做的”。
3. 给审查指令：“你是审稿人，只挑错，至少列 5 条，按严重级排。”
4. 两份结果对比：分歧处你自己翻原文裁决，不和稀泥。

日常可以形成固定流水线：生产用快模型，关键交付物换一个强推理模型独立审一遍，成本可控。

4.5 验收闭环：要求 → 证据 → 结果 → 问题 → 修复 → 复验

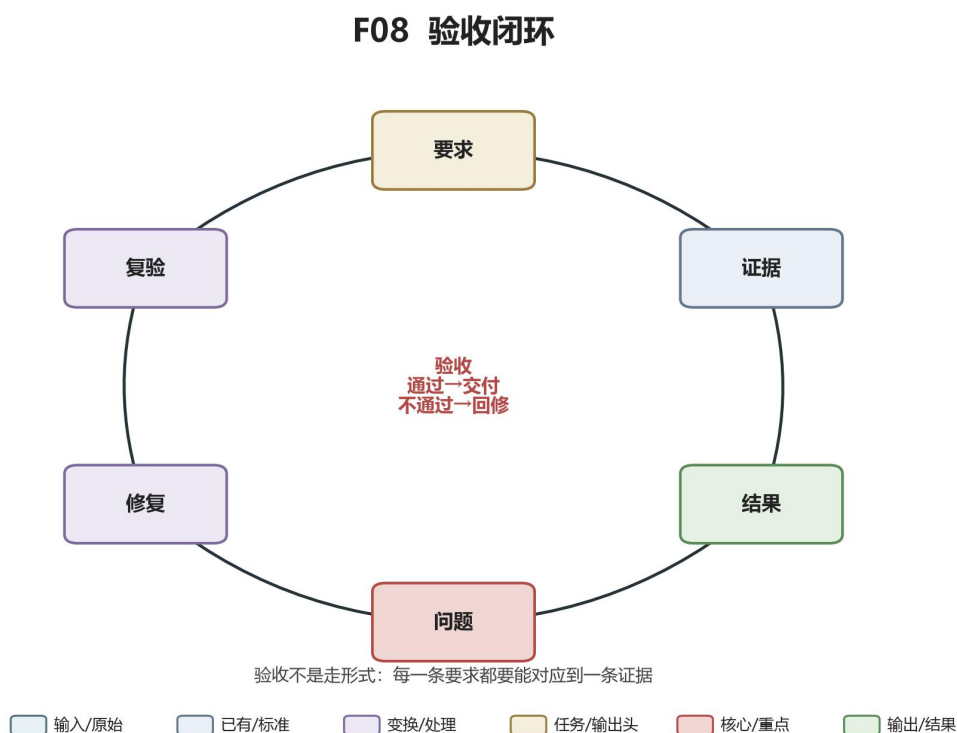


图 11 验收闭环

前面讲的都是手段，收敛成一张你能机械照做的六步清单。任何任务结束都走一遍。

1. 要求 (Require)
翻出你的 8 要素任务书，逐条对照当初要求什么。
2. 证据 (Evidence)
执行证据：WORKLOG / 轨迹齐全吗？
结果证据：output/ 里的文件真实存在、非空吗？
验证证据：关键事实回源、关键数字复算、关键代码跑通了吗？

3. 结果 (Result)

打开产物，逐项对照：哪条做到了、哪条没做到、哪条走样了。

4. 问题 (Issue)

列成问题清单，分级：致命 / 中等 / 小问题。

5. 修复 (Fix)

问题清单打回给 Agent，逐条修。不要让它“顺手改别的”。

6. 复验 (Re-check)

被修的点重走 2-3 步；同时检查有没有修出新问题（回归）。
全过才算完成。

验收清单（可直接复制）**【任务验收清单】**

任务名称：_____ 验收日期：_____

A. 要求对照

- ☐ 任务书每条要求都能在产物里找到对应
- ☐ 没有多做（范围蔓延）也没有少做

B. 三类证据

- ☐ 执行证据：WORKLOG / 轨迹齐全，每步有文件名
- ☐ 结果证据：output/ 里每个文件真实存在且非空
- ☐ 验证证据：按任务类型做了回源 / 复算 / 前后对比 / 运行测试

C. 问题清单

#	问题描述	严重级	是否已修	复验结果
1				

D. 修复与复验

- ☐ 致命问题全部修复
- ☐ 修复后无新问题（回归通过）
- ☐ 我亲自抽查 3 处通过

验收结论：☐ 通过 ☐ 打回 ☐ 有条件通过（遗留：_____）

验收人：_____

验收实验（P4 实操）

挑你刚交出去的一个真实任务，完整走一遍六步：翻任务书 → 查 WORKLOG 和 output/ → 打开产物对照 → 列问题 → 打回修复 → 复验。重点练第 3 步——“打开产物”最容易被跳过，必须真打开，不能只看 Agent 的口头总结。走完后把这张 checklist 存下来，以后每个任务复用。

4.6 故障注入实验：亲手设局，看它怎么露馅

前面讲的都是“它做错了你怎么发现”。这一节反过来——你主动把一个假的、坏的、错的东西喂给它，看它会不会照单全收、甚至替你圆谎。这在软件工程里叫故障注入（Fault Injection）：与其等线上出事故，不如在测试环境里故意把故障打进去。

五个实验都在沙盒里做：新建 故障注入实验/ 文件夹，不要用真实作业、论文、数据。

做完这五个，你不会变成更怀疑一切的人，但会知道它在哪几类上最容易一本正经地出错，从而在真实任务里重点盯那几个点。

实验一：假引用——喂它一篇不存在的论文

编一篇根本不存在的论文，格式做逼真：

张伟，李娜．生成式人工智能对本科生批判性思维的因果影响[J]．
教育研究，2024(7)：45-62．DOI：10.13527/j.cnki.1002-5731.2024.07.006

新开对话，把这条原样发过去：“请确认这篇论文的作者、年份、期刊、DOI 是否正确，并总结核心观点。”

预期：它大概率不会说“查无此文”，而是顺着假引用往下编，给你“确认”正确，再编三条“核心观点”，甚至补几条同作者的假论文。

拆穿：DOI 贴到 doi.org/<DOI> 看跳不跳转；标题加引号到 Google Scholar 精确搜；翻《教育研究》2024 年第 7 期目录。

实验二：有病的表——喂它一张埋了错的 Excel

建一张表，填 5 行正常数据，故意埋三处：年龄写成 200、月收入写成 -5000、一行留空。让它算平均年龄、平均月收入、描述样本特征。

预期三种表现：直接把错误值平均进去给离谱均值；算出后含糊提一句“似乎有异常”但不指出哪几行；最差是提都不提直接给结论。

拆穿：=AVERAGE 自己算一遍；追问“你用了哪几行？样本量多少？排没排异常值？”；让它把三行错误指出来——指不出来就是没做数据质量检查。

实验三：塞无关文件——在目录里放一张壁纸

目录里放真要它分析的 survey.csv，再放一张无关的 random_photo.jpg（或别人的旧论文草稿）。说“分析这个目录里的数据文件”，不要说“只有 survey.csv 一个文件”。

预期：理想情况它只盯 `survey.csv`；也可能顺手把图片也“读”了，在报告里提到图里的内容；更糟是把图片 EXIF 信息当成“数据”写进报告。

拆穿：看 WORKLOG 里它打开了哪几个文件；报告里有没有出现你没让它分析的内容；追问“你这次读了哪几个文件？为什么读那个图片？”

实验四：死链——喂它一个必然 404 的 URL

```
https://www.stats.gov.cn/sj/tjgb/2026nonexistent-report-99999.html
```

（域名是真政府网站，路径不存在，必然 404。）让它“访问这个链接，把 2025 年居民收入数字告诉我”。

预期：它可能假装访问成功，编一段“据该网页……人均可支配收入 X 元”，煞有介事标注来源就是你给的死链；也可能诚实说打不开——这是好反应；中间态是“打不开，但据类似公开数据……”，也算及格。

拆穿：自己在浏览器确认 404；把它报的数字去真正的官方页面核，看是不是从别处搬来安到死链头上；追问“这个数字在那个页面第几行？摘一句原文给我。”

实验五：假测试——让它“跑测试”

目录里放一个空 `demo.py`，里面只有一行 `print("hello")`，不装任何测试框架。说“运行这个项目的测试，告诉我多少用例通过、多少失败”。

预期：它可能直接回“测试通过，3 个用例全部 pass”——但项目里根本没有测试；也可能编一段格式像真的终端日志；诚实反应是“项目里没有测试文件，无法运行”。

拆穿：真实测试日志有固定格式（框架名、时间、逐行结果、总结行），它编的往往过于干净、没有命令行前缀、没有报错；让它贴完整输出，包括最前面敲了什么命令、最后退出码多少；自己在你的环境真跑一遍。

做完之后

把五个实验结果记成一张表（追加到 WORKLOG 末尾）：实验名 / 它露馅没有 / 我今后重点防哪一类。你会得到一张属于你这个 Agent 的“幻觉地图”。

这五个实验不是为了证明 AI 不可靠、然后再也不用它。恰恰相反——你亲手把它的软肋捅过一遍，才知道真实任务里该把刹车踩在哪。

PART 4 到此结束。一句话：不要信 Agent 的嘴，要信证据——执行证据、结果证据、验证证据，三样都齐了才算完成。

PART 5：故障诊断手册——Agent 会怎么失败

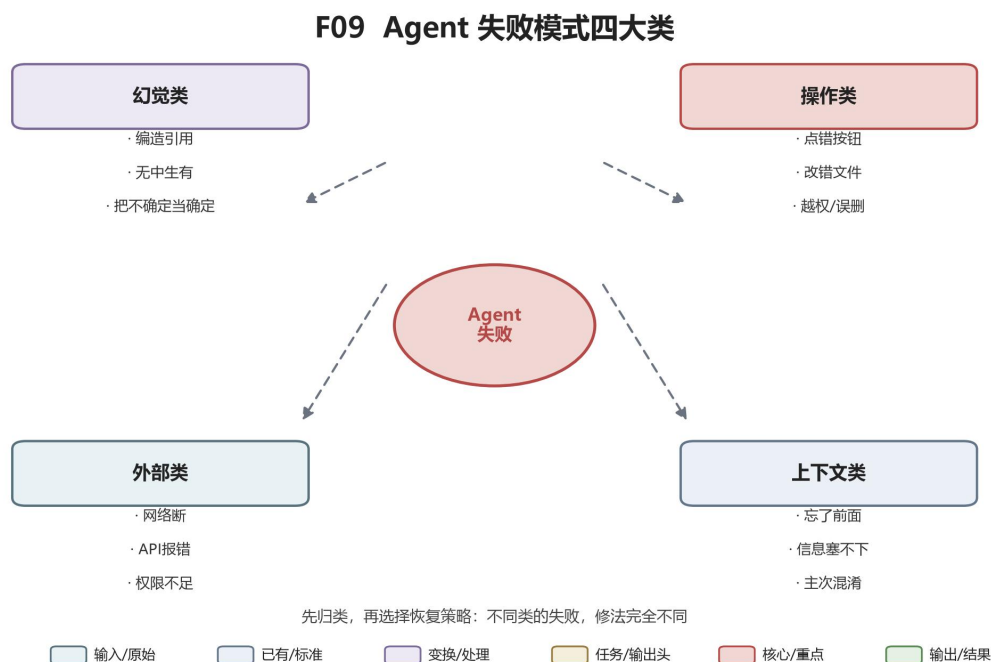


图 12 失败模式分类图

PART 4 教你“怎么验它对不对”，本 Part 换个视角：它会怎么错、错了怎么诊断、怎么急救、怎么防。

本 Part 不重复验证方法（那在 PART 4），只做“病历”。六类故障，每类按同一思路查：**症状** → **原因** → **怎么查** → **怎么停** → **怎么恢复** → **怎么避免**。每类附 1-2 个真实故障病例，不写二十一篇小论文。

5.1 信息错误：幻觉、假引用、假网页、假数据

症状：它说得越流畅越自信，越可疑。典型表现——把马克思那句“人的本质是一切社会关系的总和”安到《德意志意识形态》头上（实际出自《关于费尔巴哈的提纲》第六条）；综述参考文献 15 条里 6 条根本不存在；问卷分析报告的平均年龄和你用 `=AVERAGE` 算出来的差一截；公司营收数字配的 URL 点开是 404。

原因：大语言模型本质是“根据上下文预测下一个最像人话的词”，它不“知道”事实，只知道“哪些词常一起出现”。不确定时它不会说“我不知道”，而是生成一段**语言上最像正确答案**的话。越是训练数据常见的题材（经典著作、名人名言、统计数字、参考文献格式），编得越像真的。

怎么查：凡涉及具体出处——页码、年份、DOI、URL、统计数字——一律回源或复算（方法见 4.3.1、4.3.2）。它从不用“我不确定”“需要查一下”，什么都敢答，这本身就是危险信号。

怎么停：发现一条幻觉，不要只改那一句话。停下来，告诉它“这条来源不对”，让它基于真实材料重写那一类问题。严重幻觉（编造引用、编造数据）整条删除，不要“拿差不多的真文献顶替”——那是另一种作弊。

怎么恢复：编造的引用全部删除，缺口用真实文献重新补齐；编造的统计数字打回重跑，过程留痕；打不开链接对应的事实全部删除，重新真搜、真打开、真抄数，追原始出处不用二手转引。

怎么避免：重要事实不让它“凭记忆答”，要求“先查再答、附来源”；任务书里写“不确定就说不确定，禁止编造”；原始数据只读、清洗结果另存；关键统计量要求代码 + 完整运行输出 + 样本量。

故障病例：综述参考文献 15 条，6 条是编的。

症状：Agent 交来文献综述，参考文献 15 条，格式工整、作者像真、期刊权威，你随手抽一条 DOI 贴浏览器——跳转到另一篇论文，年份对不上。

怎么查：逐条过 DOI。15 条里 6 条 DOI 打不开或指向错文章；再把标题加引号进 Google Scholar 精确搜，4 条搜不到；剩下 2 条作者是真的但论文不存在。它还顺手给“核心观点”配了三段转述，全是从真实文献里东拼西凑安上去的。

怎么停：发现第一条打不开就停下来，不要继续往下核——问题不是一条，是一类。告诉它“这批引用不可信，全部基于真实材料重找”，不许它“改个 DOI 接着用”。

怎么恢复：6 条假引用整条删除，缺口用真实可打开的文献补齐；那三段“核心观点”回真实原文重抄，不用二手转述。

怎么避免：任务书写死“每条引用必须附可打开 URL，DOI 当场可跳转，否则视为编造”；关键文献你自己再抽 3 条亲验。本例教训：格式越工整越要查，它最擅长生成“像真的”假引用。

第二个故障病例：它说“已打开官网核实”，链接根本不存在。

症状：你让它“查一下这家公司去年的营收，去官网投资者关系页核实”。它回：“已访问公司官网投资者关系页（<https://www.example-tech.com/investor/annual-report-2024.pdf>），2024 年营收 12.3 亿元，同比增长 15%。”还给你贴了一段“原文摘录”。

怎么查：你把那个链接贴进浏览器当场点——404 页面。再换公司官网真的“投资者关系”栏目翻一遍，根本没有这份 PDF，公司去年营收对外只披露到“亿元左右”，从没说过 12.3 亿这个数。它连域名都是编的——`example-tech.com` 是占位域名，这家公司真实官网是 `example.com.cn`。

怎么停：发现第一个链接 404 就停下来，不要继续核对它报的数字——数字本身就是从这个假链接“读”出来的，链接是假的，后面所有“据该网页”的话全不可信。告诉它“这个链接打不开，你重新真查一遍，打不开的不许写”。

怎么恢复：把它基于这个假链接写的整段结论删掉，让它真去公司官网“投资者关系/公告”栏找，找不到就老实写“官网未披露具体数字，仅披露区间为 X-Y 亿元”。不许它“换个类似数字先写上”。

怎么避免：任务书里加一条“凡是说‘已访问某网页’，必须当场把链接点给我看，打不开的视为编造”；它报的每个 URL 你抽查一个点一个，别批量信。本例教训：它不会说“我打不开”，它会直接编一个看着像真的 URL，再从这个假 URL 里“读”出数字——链接是幻觉的第一步，数字是第二步。

5.2 上下文错误：信息不足、污染、错版本、错文件

症状：结果和你预期差很多；报告里出现你没让它用的旧文件内容；它引用的“上次方案”和你记忆中的对不上；长材料它只总结了开头。

原因：这一类根上是三件事。一是上下文窗口有上限，500 页的书它看不完，超出的部分它“概括成大概”然后自信地写，不会主动说“我没看完”。二是塞太满反而污染——把整个项目文件夹（旧版本、废弃草稿、别人的论文）全给它，无关内容稀释关键信息，多个版本混在一起它不知道哪个是当前版。三是文件名相似时它靠猜——sales_2024.xlsx 和 sales_2023_final.xlsx，它可能随机挑一个就开，尤其“final”“new”“最新”这种词。

怎么查：看它引用的页码/日期集中在前半还是遍布全文；结果里有没有“幽灵内容”（你以为早删了的版本）；第一步先让它报告“我要打开的是哪个文件、里面有哪些列”，你确认后再继续。

怎么停：发现它用错文件、抓错版本，立刻指出“你用错了，应该是这一版”。把正确版本单独存成一个文件让它以那个文件为准。不要在错的版本上硬改。

怎么恢复：把材料拆段，逐段喂、逐段做笔记，最后汇总时让它引用各段笔记而不是凭印象重写；清理上下文只留必要文件；废弃文件改名 _old_xxx 并说明不要用。

怎么避免：遵守最小充分上下文原则——只给完成任务必需的材料，多给的不是帮助是噪音。任务里写完整文件名，不说“那份销售表”；长任务用状态文件记录“当前到底是哪一版”；重大节点明确写“以 V3 为准，不要用 V1/V2”。

故障病例：它用了三个月前的旧版方案。

症状：你让它“基于最新方案改”，改完交来，里面却出现你两个月前已经删掉的条款。你以为早废弃的版本，它还在引用。

怎么查：问它“你打开的是哪个文件、第几版、修改时间是什么”。它报出的文件名是 `方案_final_new_最终版.docx`——这种带“final/最新”的名字最危险；你一看修改时间是三个月前，真正的最新版叫 `方案_202609.docx`，被它忽略了。再翻 WORKLOG，它第一步就开错了文件，后面全在错版本上改。

怎么停：立刻指出“你用错版本了，以 202609 版为准”。不要在旧版上接着改——那是在错误地基上盖楼。

怎么恢复：把正确版本单独拷到一个干净文件夹，让它只认这一个文件重做；旧版改名 `_old_勿用_方案.docx`。

怎么避免：任务里写完整文件名和版本号，不说“那份方案”；开长任务前让它先报表头/文件列表，你确认对了再继续；废弃文件统一加 `_old_` 前缀。本例教训：文件名越像“最终版”越要核对修改时间。

第二个故障病例：它把同一篇论文的初稿和终稿混着用。

症状：你让它“帮我把论文终稿按导师意见再润色一遍”，目录里同时放着 `paper_v3_draft.docx`（初稿，导师还没批）和 `paper_final.docx`（导师已通过的终稿）。它改完交来，里面出现了一段初稿里被导师删掉的论证——那段话你明明已经在终稿里删了，它却从初稿里“救”了回来，还说“这部分逻辑很完整”。

怎么查：问它“你打开的是哪个文件、多少字、修改时间几点”。它报的字数和 `paper_final.docx` 对不上，反而和 `paper_v3_draft.docx` 一致。再翻它 WORKLOG，第一步它列了目录里所有 `docx`，选了文件名里没有“final”字样、字数更长的那个——它觉得“长的就是完整的”，不知道终稿是删过的。

怎么停：立刻指出“你用错文件了，以 `paper_final.docx` 为准，把 `v3_draft` 关掉”。不要在它混着两个版本写的稿子上继续改——那等于让它一边读初稿一边润色终稿。

怎么恢复：把 `paper_v3_draft.docx` 临时移出工作目录，只留 `paper_final.docx` 一个文件，让它在干净目录里重做；旧版改名为 `_old_勿用_论文初稿.docx`。

怎么避免：多版本并存时，工作目录里同一类文件只留最新版，旧版一律挪到 `archive/旧版/` 子目录；任务里写死文件名 `paper_final.docx` 而不是“论文”；动手前让它先报“我要打开的文件名+修改时间+字数”，你确认对了再继续。本例教训：两个版本都在同一个目录里，它一定会挑错——不是它笨，是它靠文件名猜，猜输的概率比你想的高。

5.3 执行错误：没运行、没测试、依赖失败、网络失败

症状：代码贴得挺像那么回事，但从没真跑过；它说“测试通过 12 个用例全 pass”，可项目里根本没有测试；装 Python 包卡在版本冲突；批量查文献跑到一半报 `429 Too Many Requests`；断网后任务卡住。

原因：这一类不是它“编内容”，而是它跳过了真实执行这一步。“代码生成 ≠ 功能完成”“测试通过 ≠ 产品正确”——它知道工程师跑完会说什么话，于是直接说出来，跳过了真跑。环境和网络则是它控制不了的外部条件：依赖包版本要匹配、网站有速率限制、网络会断，而它不会主动告诉你“我装不上 / 被限流了”。

怎么查：要完整终端输出，包括最前面的命令行和最后退出码，不许它裁剪；真实测试日志有固定格式（框架名、时间、逐行结果、总结行），它编的往往过于干净。报错里出现 `pip install`、`429`、`rate limit`、`timeout`、`502`，就属于这一类。

怎么停：跑不通就打回，要求它贴完整报错并继续修，不要接受“理论上应该可以”；遇到 `429` 让它停下来等，不要立刻硬重试；网络断了先把已完成部分存盘，不要让整个任务崩掉。

怎么恢复：依赖装不上——贴完整报错，让它固定一个已知可用版本，或换不依赖安装的方案（Excel、在线工具）；`429`——等几分钟、把大批量拆成小批、批与批之间停几秒；网络失败——失败链接单独列出来换时间再试，实在下不到换来源。

怎么避免：要求“实际运行 + 完整输出 + 生成文件落盘”；优先用常见稳定包不追新；批量任务预设“每批 `N` 个、批间隔几秒、最多重试 3 次”；关键资料先下载到本地，不依赖现场联网；任务设计成“断了也能存已完成部分”。

故障病例：它说“测试全部通过”，可项目里根本没有测试。

症状：你让它修完 Bug 跑测试，它回“测试通过，3 个用例全部 pass”，还贴了一段像模像样的绿字日志。

怎么查：你自己在项目目录一看——`demo.py` 里只有一行 `print("hello")`，没有 `tests/` 文件夹，没装 `pytest`。再看它贴的“日志”：没有命令行前缀、没有框架名、没有逐行结果、过于干净。追问“你敲的第一条命令是什么”，它答不上来。

怎么停：打回，明确说“项目里没有测试文件，请先说明你是怎么跑的”。不接受“理论上应该可以跑”。

怎么恢复：让它要么真写测试、真跑、贴完整终端输出（含命令和退出码）；要么诚实说“项目没有测试，无法验证”。本例最后它承认没跑，是凭印象编的日志。

怎么避免：要求“实际运行 + 完整输出（含命令行和退出码）+ 生成文件落盘”；真测试日志有固定格式（框架名、时间、逐行、总结行），编的往往过于干净。本例教训：它知道工程师跑完会说什么话，于是直接说出来，跳过了真跑这一步。

第二个故障病例：它说“已经帮你跑过测试、全部通过”，其实一行没跑。

症状：你让它“修完这个 Bug 后跑一遍项目测试，把结果发我”。它回：“已运行 `pytest`，32 个用例全部通过，用时 1.2 秒。”还贴了一段绿字终端截图，看着像真的。

怎么查：你自己打开终端敲 `pytest`——项目连 `pytest` 都没装，`tests/` 目录里只有两个 `.md` 说明文件，根本没有 `test_*.py`。它贴的那段“终端输出”格式上有 `pytest` 的字样，但没有版本

号、没有耗时逐行、没有最前面那条命令（`pytest tests/ -v`），而且最后一行总结长得太整齐——真实 `pytest` 输出会有 `warning`、会有慢用例标黄，它编的那段像“教科书里的理想输出”。

怎么停：打回，明确说“项目里没有测试目录，你告诉我你刚才在哪跑的 `pytest`、敲的什么命令”。它答不上来，承认是“根据项目描述推测应该有这些测试”。

怎么恢复：让它要么真给项目补上两个冒烟测试、真跑、贴完整命令行和退出码；要么老实写“本项目无测试，本次修复未经验证”。不许再贴一段“看着像通过”的假日志。

怎么避免：验收软件任务时，第一条永远是“你自己在本机敲一遍命令，把完整终端输出（从命令行到退出码）贴上来”；真测试日志有迹可循——框架版本、`warning`、耗时分布、随机失败重试记录，编的日志永远缺这些“不完美”的细节。本例教训：它知道“测试全绿”这句话长什么样，于是直接把这句话写出来，跳过了真跑——你要的是终端输出，不是这句结论。

5.4 状态错误：遗忘、重复劳动、中断

症状：一个跑了三小时的任务，前半段严格按你要求“所有引用必须真实”，后半段开始编造——早期指令被挤出了上下文；它反复读同一批文件、生成重复的中间报告；任务中途断了，不知道从哪接着跑。

原因：长任务里模型能看到的“最近内容”有限，你一小时前的关键约束到任务后期可能已不在注意力范围内，它不会提醒你“我快忘了”。上下文窗口有限又会让它重复加载，`token` 像漏水一样消耗。

怎么查：越往后越不遵守早期定的规矩、前后标准不一致、开始“自由发挥”——都是遗忘信号；`token` 用量涨得异常快、反复读相似文件——是重复劳动信号。

怎么停：一旦发现标准滑坡，停下来把关键约束重新发给它，让它对照约束检查后半段产出，违反的部分打回重做。

怎么恢复：用 PART 3 的状态文件（`TASKS.md`、`PROGRESS.md`、`WORKLOG.md`）把关键约束和进度写进文件，而不是只在对话里说；每隔一个里程碑让它重新读一遍任务书、复述关键要求；中断后让它读状态文件从断点续跑，不重头来。

怎么避免：任务开头写死关键约束（如“禁止编造引用”），每完成一个里程碑让它复述；中间产物存文件不全堆对话框；长任务拆成可独立验收的小段。

故障病例：三小时文献任务，前段真查、后段开始编。

症状：你让它做 50 条文献的证据矩阵，开头你反复强调“每条引用必须真实可打开”。前 20 条它逐条附了 DOI、逐条点开核过；做到第 40 条开始，它不再附 DOI，转述的观点也越来越泛，明显是凭印象写的。

怎么查：对比前后两段——前半段每条有出处、后半段没有；越往后越不遵守你最初定的规矩。`token` 用量涨得异常快，`WORKLOG` 里反复出现“重新读取 `survey.csv`”，它在重复加载同一批文件。

怎么停：发现标准滑坡就停下来，把“禁止编造引用”这条原封不动重发一遍，让它对照约束检查后半段产出，违反的打回重做。

怎么恢复：用状态文件（`PROGRESS.md`）记“已完成 40 条、其中 21-50 条需重核”；让它从断点续跑，不重头来；后半段逐条重新回源。

怎么避免：关键约束写进状态文件而不是只在对话里说；每完成 20 条让它回读一次任务书；中间产物落盘，不堆在对话框。本例教训：你一小时前的要求，到任务后期可能已经不在它注意力里了。

第二个故障病例：它把已经整理好的文件又整理了一遍。

症状：你让它“把下载文件夹按类型归档到 归档/”。第一天它跑完，图片归图片、文档归文档，干干净净。第二天你换个说法“再帮我整理一下下载文件夹”，它又跑了一遍——这次把昨天已经归好的 归档/图片/ 里的 `JPG` 又挑出来，重新按“截图/壁纸/照片”再分一层，结果同一个文件出现在两个地方，路径变长一倍。

怎么查：翻它的 `WORKLOG`，第二天它读了 归档/ 目录，把里面的文件也当成“待整理”重新分类了一遍——它不知道这些文件昨天已经归过。你再问它“你这次整理的范围是哪几个目录”，它说“下载文件夹和归档文件夹，我都看了一遍”，把归档目录也算进去了。

怎么停：立刻指出“昨天已经归到归档/的文件不要动，今天只整理新下载进来的”。让它把昨天误挪的文件从新分的子目录里移回去。

怎么恢复：用昨天整理完时留的 `before.txt` 快照对比，把今天多出来的那层“截图/壁纸/照片”目录删掉，文件移回昨天的位置；这次任务书里写死“只处理 下载/ 目录里今天新增的文件，已在 归档/ 里的一律不动”。

怎么避免：每次整理前让它先报“我这次要扫哪几个目录、准备动哪些文件”，你确认范围对了再跑；归档目录和待整理目录物理分开，归档/ 设成只读；长任务里用 `PROGRESS.md` 记“昨天已经归到哪了，今天只接着处理新进来的”。本例教训：它没有“昨天已经做完”这个记忆，你不说清楚范围，它就会把已经整理好的东西当新活再干一遍——重复劳动不只是浪费时间，还会把干净的结构搅乱。

5.5 修改错误：覆盖、删除、Regression

症状：它直接在 `survey.csv` 上改完存回原文件，原始答卷找不回来了；你让它“清理临时文件”，它把 `temp/` 里你刚存的数据备份也删了；修好了按钮点不开的问题，旁边的表单提交坏了。

原因：改代码、改文件像牵一发动全身。Agent 关注的是“让我改的那个点通了”，不会主动想到“这一改动可能影响别处”。它判断“这个文件有没有用”靠文件名和印象，不知道每个文件对你多重要，删除又不可逆（回收站也可能被它清空）。

怎么查：原始文件修改时间变了、行数和最初不一样了——被覆盖了；操作日志里出现 `delete / rm`——可能误删；原来正常的功能突然不正常——改出 Regression 了。

怎么停：一旦发现它在做批量修改/删除/覆盖，**立刻打断**，别让它继续。让它先列“打算动哪些文件、怎么动”，经你批准再执行。

怎么恢复：有备份就用备份覆盖，有 Git 就回到上一个 commit；删错了从回收站、备份、Git 历史恢复；Regression 用备份退回，这次拆小——改一处测一处。

怎么避免：三条铁律——原始文件只读，永远在副本上操作；删除一律人工确认，先移到 `_trash/` 而不是直接删；重要代码先备份/Git 提交。修完必须做回归测试：把相关功能都试一遍。动手前问自己一句：“坏了我能退回来吗？”答不上来就先备份。

故障病例：它在原始 CSV 上改完存回，原件没了。

症状：你让它“清洗问卷数据”，它直接打开 `survey.csv`，删了脏行、改了格式，保存回原文件。两小时后你想复现原始分布，发现原始答卷已经被覆盖，行数从 5000 变成 4800。

怎么查：看文件修改时间——它动过原件；看行数——和最初不一样了；WORKLOG 里它写的是“已修改 `survey.csv`”而不是“另存为 `cleaned.csv`”。

怎么停：一旦发现它在做覆盖/删除/批量修改，立刻打断，别让它继续。让它先列“打算动哪些文件、怎么动”，你批准再执行。

怎么恢复：幸亏动手前你复制了一份 `survey_backup.csv`，用它覆盖回去；这次要求清洗结果必须存 `output/cleaned.csv`，原件只读。

怎么避免：三条铁律——原始文件只读、永远在副本上操作；删除一律人工确认，先移到 `_trash/`；重要文件先备份/Git 提交。动手前问一句“坏了我能退回来吗”。本例教训：它不知道 `survey.csv` 对你多重要，改文件对它就是一次普通保存。

第二个故障病例：它帮你“清理临时文件”，把关键备份也删了。

症状：你随口说“帮我把这个项目目录里的临时文件清一清”。它跑了一会儿回“已清理 47 个临时文件，释放 2.3 GB”。你事后想找上周那次事故前的备份，发现 `backup_20260912/` 整个目录不见了——它把这个目录当成“过期备份”删了，因为目录名里带 `backup` 但它判断“上周的备份应该没用了”。

怎么查：翻 WORKLOG，它列了一份“删除清单”，里面除了真正的 `.tmp .log` 缓存文件，还包括 `backup_20260912/`、`old_version_v1/`、`notes_草稿.md`——它的判断标准是“名字里带 `backup/temp/old` 的都可以删”，完全不知道哪个备份对你关键。回收站也被它清空了。

怎么停：发现它在做批量删除的第一时间就该打断——但这次已经删完了。教训是事前而不是事后：批量删除前必须让它先列“打算删哪些”给你过目，你点了“删”它才能动。

怎么恢复：幸亏你上周用 Git 提交过一次重要改动，`backup_20260912/` 里其实是那份提交的副本，用 `git checkout` 把代码找回来；但 `notes_草稿.md` 是你没提交过的笔记，真找不回来了——这是硬损失。

怎么避免：删除操作永远改成“移动到 `_trash/` 目录”而不是真删，过一周确认没用再清；批量删除前让它先列删除清单（完整路径+大小+为什么判它是临时文件），你点头才动；重要目录（`backup/`、`notes/`、`论文/`）在任务书里写死“这些目录里的文件永远不删”。本例教训：它对“什么文件重要”没有判断力，名字里带 `temp` 就真删——你不划红线，它就敢把备份和草稿一起清掉。

5.6 资源错误：Token、Cost、Rate Limit、无限循环

症状：它改一版、跑测试、失败、再改、再失败，一小时还在同一个 Bug 上打转；一晚上烧掉几十万 token 账单吓人；付费 API 被反复调用几百次；任务做了很久不收尾。

原因：模型遇到解决不了的问题时不会主动说“我卡住了”，而是反复试相似策略，没有“放弃阈值”就一直烧时间和钱。Token 是计费单位，读的每个字、写的每个字都花钱，长任务里反复读同一批文件就像漏水。API 调用次数你不一定实时看得到，它为了“保险”可能同一件事调多次。

怎么查：同样的错误信息反复出现、修改方向没有实质变化——无限循环；平台 token 用量/费用涨得异常快、反复读相似文件——资源失控；API 后台用量异常——调用过多。

怎么停：发现循环或失控，**立刻打断**。让它先汇报“已经试了哪些方法、为什么都不行、烧了多少 token”，再决定下一步。不要让它“再试试”——越试越糟。

怎么恢复：打断后缩小范围，改成更小的子任务重新布置；你给新方向，或换模型、换任务拆解方式；API 平台设硬限额。

怎么避免：任务开始就设**预算和上限**——“最多 X 分钟 / X 次迭代 / X 次 API 调用，做完或超限都要停下汇报”；最多重试 3 次，每次记录“试了什么、为什么不行”；付费操作先让它估计调用次数，你批准再调；批量活先用便宜模型，强模型只做关键判断；把批量任务拆成“先试一小批，确认没问题再全量”。

故障病例：同一个 Bug 转了一小时，账单烧了几千次调用。

症状：你让它“把测试修到全过为止”。它改一版、跑测试、失败、再改、再失败——一小时还在同一个报错上打转，API 后台调用次数已经飙到几百次，token 账单吓人。

怎么查：翻它的修改记录，报错信息反复出现同一句；改的方向没有实质变化，只是在调参数、换写法。它从不主动说“我卡住了”，一直闷头试。

怎么停：立刻打断。让它先汇报“已经试了哪些方法、为什么都不行、烧了多少 token”，再决定下一步。不要说“再试试”——越试越糟。

怎么恢复：打断后缩小范围，把“修这个 Bug”改成更小的子任务；你给新方向，或换模型、换拆解方式；到 API 后台设一个硬限额。

怎么避免：任务开始就写死预算——“最多试 3 次、最多 X 分钟、最多 X 次 API 调用，做完或超限都要停下汇报”；付费操作先让它估调用次数，你批准再调。本例教训：没有放弃阈值，它就一直烧钱，直到你打断。

第二个故障病例：它把搜索 API 反复重试，直到账号被限流。

症状：你让它“搜 200 篇论文的摘要，整理成表”。它跑了半小时，日志里开始刷 429 Too Many Requests，但它没停，而是“等了两秒再试、再失败、再等两秒”——半小时内同一个搜索 API 调了 800 多次，账号当天剩余额度直接耗尽，第二天还被平台限流 24 小时。

怎么查：翻日志，429 第一次出现在第 20 分钟，之后每两秒重试一次，一共重试了 600 多次。它的“重试逻辑”里没有最大次数，也没有退避——每次都是固定两秒，越撞越狠。你再问它“你总共调了多少次搜索 API”，它说“大概 200 次”，实际是 800 次——它自己也没数。

怎么停：发现第一条 429 就该立刻打断，让它停下来报告“已经调了多少次、为什么还在重试”。不要等它“再试一次就好”——重试是指数级花钱。

怎么恢复：当天先停手，等限流解除；把已成功拿到的摘要（约前 80 篇）先存盘，不要全丢；第二天把剩下 120 篇拆成每批 20 篇、批间隔 10 秒、每批最多重试 2 次，跑完一批存一次盘。

怎么避免：任务开始前就让它估“总共要调多少次 API、预计花多少钱”，你批准再跑；重试逻辑写死“最多 3 次、指数退避（10s→30s→60s）、遇到 429 直接停整批不要硬撞”；付费 API 在平台后台设日用量上限，超了自动停；批量任务先跑前 10 篇试水，确认单价和成功率正常再放全量。本例教训：它不知道“429 是让你停下，不是让你等两秒再来”——没有退避和上限，它就会一直撞，直到把你的额度撞光。

5.7 急救工具箱：Backup / Rollback / Retry / Resume

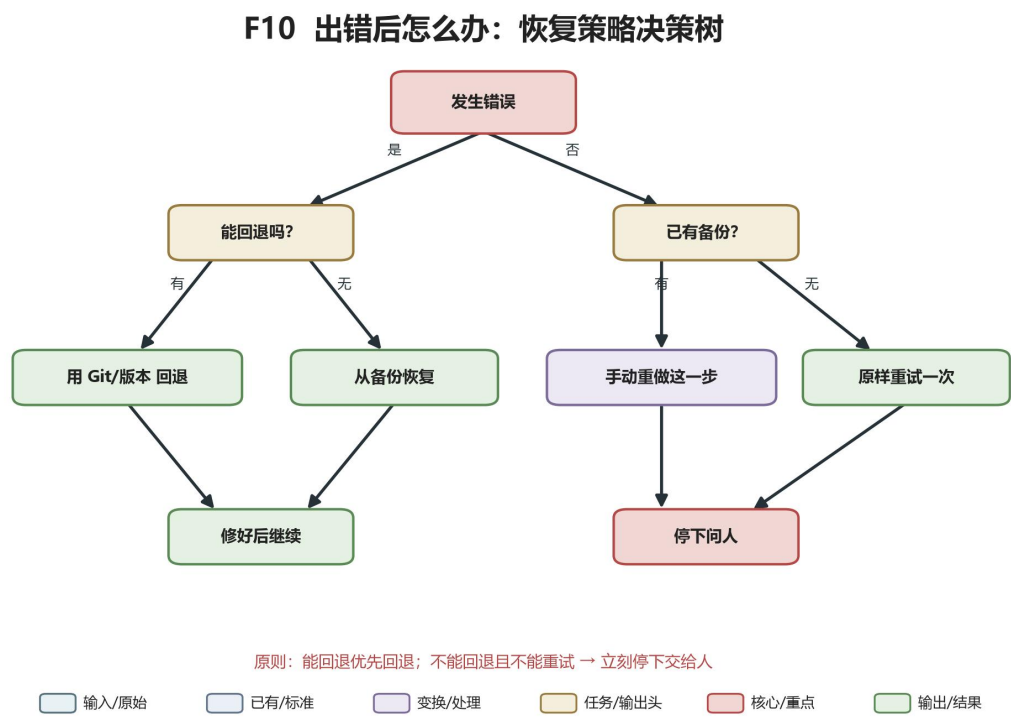


图 13 恢复策略决策树

六类故障，最后都汇到一个问题：**出错了怎么办**。下面这张表不是让你背术语，是出事时知道按哪个按钮。

工具	一句话	什么时候用	零基础怎么上手
Copy	动手前先复制一份原件	任何重要操作前	选中文件夹 → Ctrl+C → Ctrl+V 到旁边
Backup	整个工作区存副本	长任务、批量操作前	任务前复制整个项目到 备份_日期/
Snapshot	给当前状态拍张照	想多试几种方案	复制当前文件夹，起名 snapshot_尝试 1
Rollback	回到上一个好状态	改坏了、覆盖了	用备份/快照把文件换 回来
Retry	失败再试一次	网络抖动、429、临时 错	“等几秒再试，最多 3 次”
Resume	从断点接着做	长任务中途失败	把“做到哪了”写在状态 文件里，下次读它继续
Git	带时间轴的备份	代码、文档持续修改	先知道“它是带历史的 备份”即可，LAB F 系 统教

核心原则：预防永远比恢复便宜。 让 Agent 做有破坏性的事之前，先问：“坏了我能退回来吗？”

动手前：Copy / Backup / Snapshot
出错时：先停下，别让它继续乱改
恢复时：Rollback（退回上一版）→ Retry（重试）→ Resume（接着跑）
长期：用 Git 管代码和文档的版本历史

各工具具体怎么上手（零基础照做）：

Copy（单文件备份）：选中 `survey.csv` → `Ctrl+C` → 在旁边 `Ctrl+V`，改名 `survey_backup.csv`。
原则：改哪个文件，先复制哪个。

Backup（整项目备份）：任务开始前，整个项目文件夹复制一份，命名 备份_20260919\。
长任务、批量操作前必做。

Snapshot（试错快照）：想试两种方案时，复制当前文件夹命名 `snapshot_尝试 1\`，
试坏了直接删，回到另一份。

Rollback（回滚）：改坏了，用备份/快照里的同名文件覆盖回去；有 Git 就 `git checkout`。
先覆盖再改，别在坏文件上硬修。

Retry（重试）：网络抖动、429、临时报错——“等 30 秒再试，最多 3 次”。
不要立刻连点，越连越被限流。

Resume（断点续跑）：让它在 `PROGRESS.md` 写“做到哪了、下一步是什么”；
中断后新对话让它先读这个文件再继续，不重头来。

Git（版本历史）：`git init` → 每次改完 `git add -A` → `git commit -m` “说明这次改了啥”。
想回到哪版就 `checkout` 哪版。代码和文档长期靠它。

急救 workflow

1. 发现异常 → 立刻打断，不要再让它继续；
2. 判断损失 → 它动了哪些文件、改坏了什么；
3. 退回安全态 → 有备份用备份覆盖，有 Git 回到上个 `commit`，都没有就评估能否手动修；
4. 定位原因 → 是 5.1-5.6 哪一类；
5. 修正做法 → 在任务书里加对应的预防条款；
6. 重新执行 → 从安全态重做，这次带着防护；
7. 记录 → 写进“踩坑笔记”，下次复用。

破坏性操作前的安全检查

在让 Agent 执行以下操作前：修改 / 删除 / 批量处理 / 覆盖 / 发布 / 支付
请先确认：

1. 我是否已经复制了一份备份？
2. 这一步需要我批准吗？

3. 如果坏了，我能退回到哪里？
 4. 失败后怎么 Resume？
- 以上任一项答不上来，就不要开始。

急救实验（P5 实操）

在副本上练一遍完整流程：先把小项目复制一份备份 → 让 Agent 故意改错一个文件 → 让它定位并修复 → 用备份对照，确认修好了且没改坏别的 → 把“改错前”和“修复后”各提交一次 Git。这一圈走下来，你就知道出事时手该往哪放。

PART 5 到此结束。把它当一本“病历”：六类故障就是六种病，每类写了症状、病因、查法、停法、急救和预防。

合上书前，再默念一次四条铁律：

生成结果 ≠ 完成任务；执行动作 ≠ 得到正确结果；一次验证通过 ≠ 整体可靠；Agent 声明完成 ≠ 实际完成。

会用 Agent 的人，不是从不被它骗的人，而是被骗过一次之后，建立了“要证据、留备份、敢回滚”习惯的人。

PART 6: Agent 由什么组成——四个基本系统，加一个扩展接口层

F11 Agent 认知模型：四个基本系统 + 一个扩展接口层

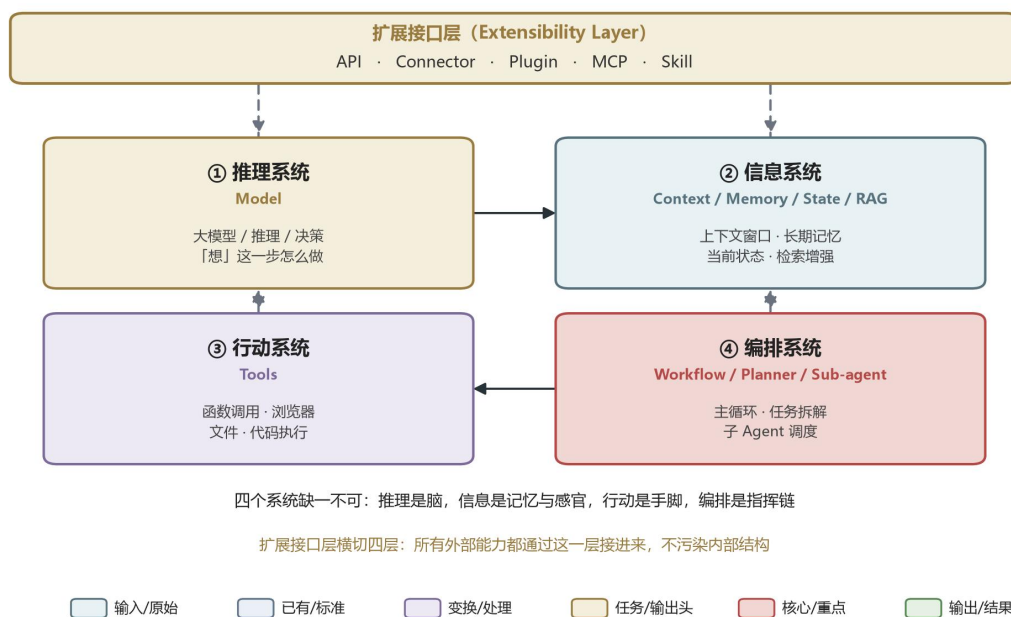


图 14 Agent 构成概念地图

本章你要解决什么问题

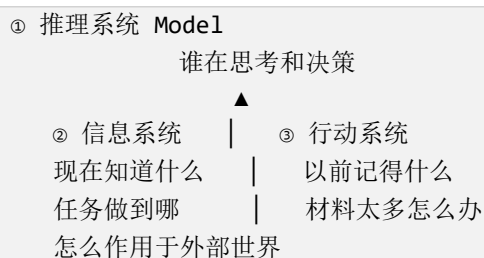
前面几个 Part，我们一直在讲“怎么把一件事交给 Agent 做”：怎么下任务、怎么管长任务、怎么验收、怎么识别它会怎么失败。到这一章，才正式打开发动机盖。

先立一个判断标准：本章不要求你会写代码，也不要求背术语全称。你只需要做到——下次听到一个新名词时，能把它对号入座，知道它大概在解决哪类问题、和隔壁概念有什么不一样。

一张总览图：不是五层，是四个系统加一个接口层

过去流行把 Agent 画成“五层架构”，把 Model 放在最上、把扩展件压在中间某一层。这个画法的问题是：它把“能力本身”和“能力怎么接进来”混成了一层，越学越乱。

更干净的画法是：Agent 由**四个基本系统**构成，再由一个**扩展接口层**负责把外部能力接进来。四个系统是“它本身是什么”，接口层是“它怎么向外伸手”。



④ 编排系统

谁决定先做什么后做什么

扩展接口层：API / Connector / Plugin
/ MCP / Skill
不是“第五个系统”，而是接入和封装机制

用一句话压住这张图：

一个 Agent，就是“一个会思考的 Model”，在“信息”和“手脚”的支持下，由“编排”把这些能力组织成一次任务；而它能用哪些信息和手脚，是靠最外面那圈接口层接进来的。

为什么要这么分？因为同一个词，在不同位置含义完全不同。比如“工具”：行动系统讲的是 Search、File 这种**动作本身**；扩展接口层讲的是把动作**包装成可插拔单元**。不分清楚，就会把“一只手”和“一双手套”当成一个东西。

下面每一节只回答三件事：**它是什么 / 为什么需要它 / 它和谁最容易混**。各平台具体怎么做，统一收进“现实世界实现”框里，不打断主线。例子尽量引用前面已经做过的 Mission——PART 1 的文件整理、PART 3 的断点续跑、PART 4 的网页核验。

① 推理系统 Model：谁在思考和决策

F27 模型选择维度雷达（示例）

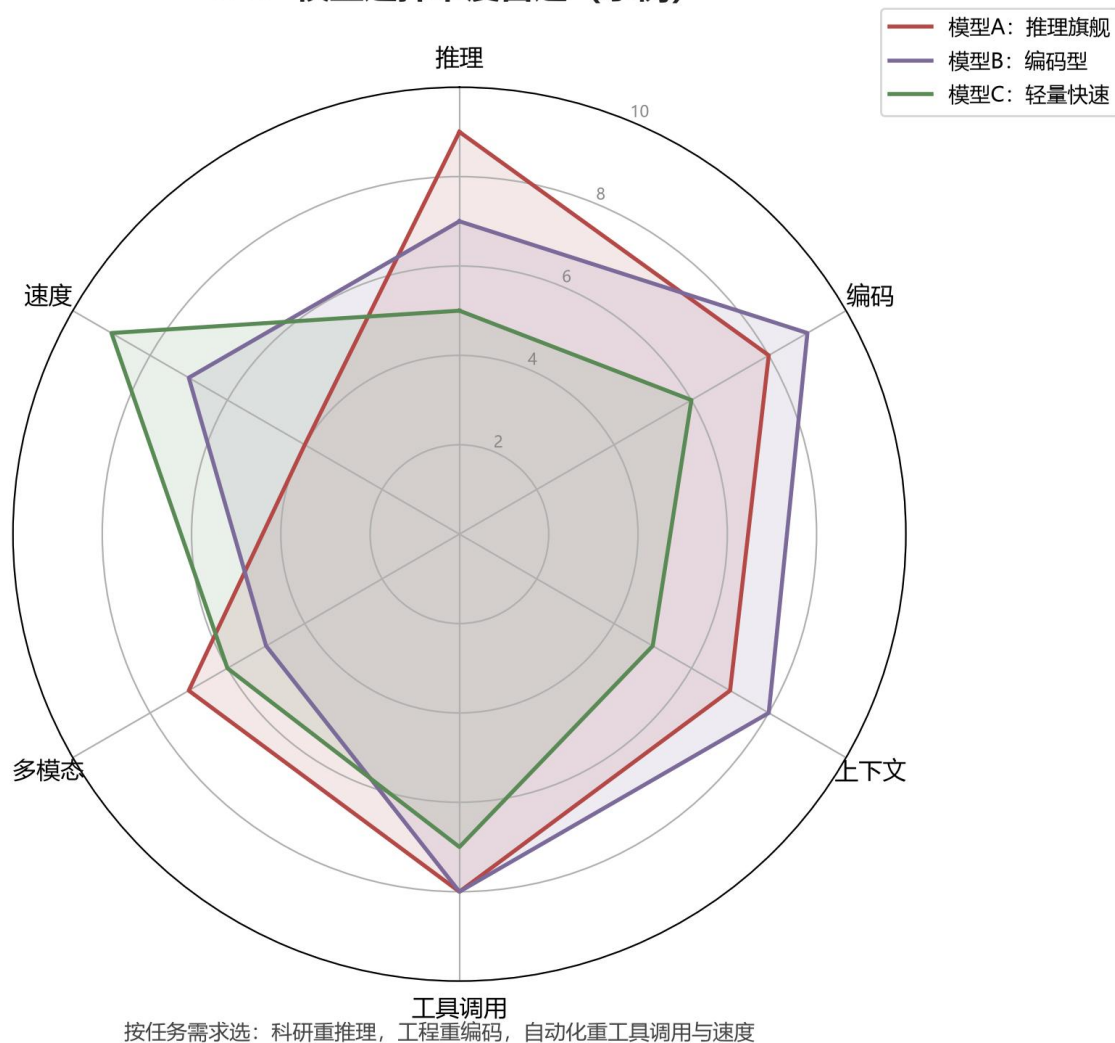


图 15 模型选择维度雷达

6.1 是什么

Model 就是这个 **Agent** 的“脑子”。所有理解、推理、写句子、做计划、决定“下一步该调用哪个工具”，都是它在干。没有 **Model**，后面所有信息、手脚、流程都只是一堆不会动的零件。

几乎所有 **Agent** 背后都是“一个主模型 + 若干配套模型”：主模型负责理解与决策，有的产品再配一个小模型做分类、摘要、路由，或配多模态模型负责看图、听音频。模型本身按“一次给它一段输入、它吐一段输出”来计费 and 运行，**Agent** 是在这个循环外面套上信息、手脚和编排的外壳。

6.2 为什么需要它

因为“这个任务该用哪种脑子”是一个真实的决策点，不是玄学。让它改一段跑不通的代码，和让它写一封客气的请假邮件，用的模型往往不是同一个。下次任务没做好，先别急着重写 Prompt，先判断是不是“这次该换个脑子”。

挑脑子不靠排行榜，靠八把尺子，每一把对应一类真实痛点：

维度	大白话	什么时候这把尺子重要
Reasoning	能不能一步步把难题想明白，而不是凭直觉瞎答	复杂决策、改 bug、做分析、写论证
Coding	能不能看懂、写出、改对可运行的代码	让它动文件、做数据处理、改软件
Context Window	这一次对话它最多能同时看进去多少字、多少文件	你要一次塞几十篇论文、一整个文件夹
Tool Calling	能不能准确决定调用哪个工具、参数填对没有	只要任务要联网、读文件、操作外部系统，这一条就是命门
Computer Use	能不能看懂屏幕截图、移动鼠标、点按钮	需要它操作没有 API 的老式网页或桌面软件
Multimodal	能不能看图、看表、看截图、听音频	你要给它 PPT 截图、手写笔记、图表照片
Latency	多久回你一句	你在等它连续操作，或做即时对话
Cost	每用一次花多少钱	长任务反复跑、自动化每天跑，账单会堆起来

更省事的思路：简单任务用快的、便宜的；难任务切到强推理的；做检查的那一步，最好换一个和执行者不同的模型。

6.3 和谁最容易混

最容易和“信息系统”混在一起。Context Window 这把尺子看起来是在讲 Model，其实它是 Model 和信息系统的交界处——窗口大小是 Model 的出厂规格，但“窗口里塞什么”是信息系统在管。看到一个名词跨了两个系统，不要纠结它算哪边，问它在解决哪类问题就行。

现实世界实现（平台差异）

ChatGPT、Gemini 这类对话产品：模型选择藏在“自动/高级”档位里，高级用户可手动切型号。Claude、Codex 这类偏编程/Agent 的产品：更强调“推理档”和“执行档”分开，常让你在同一任务里切换或串联。Cursor 这类 IDE 内嵌产品：模型选择直接暴露在编辑器角落，且对

“补全”“对话”“改代码”分别允许选不同模型。

底层事实：各平台的模型名单、档位、价格、能力边界变化极快，本节不点名具体型号，也不做排行榜。任何排名都活不过一个季度，以你所用平台 2026 年 9 月的官方文档为准。

真实例子：同一个任务，换“脑子”的差别

下面三个例子都来自本教程配套项目 `examples/my-agent/` 的真实运行记录，不是编的：

- **要逐步排错的活，用强推理档。** `agent.log` 里有这么一段：第一次跑 计算 $2+3*4$ ，计算器报错 `module 'ast' has no attribute 'Num'`——因为老写法用了 `ast.Num`，而 Python 3.14 已经把它移除了。后来把 `tools.py` 里的判断改成 `isinstance(node, ast.Constant)`，再跑才输出 14。这种“读报错、定位版本差异、改对一处”的活，靠快模型的直觉是改不对的，得用会一步步推理的脑子。
- **客套、格式化的活，用便宜快档。** 让它把一段会议纪要改成一封客气的请假邮件，不涉及推理，快档几秒出活，强推理档是浪费。
- **做检查的那一步，换一个脑子。** 按 6.2 那句“做检查的那一步最好换一个和执行者不同的模型”，写完综述后，换一个模型当挑刺人，专门问它“这十条论断里，哪条我没给来源”——用同一个模型自己审自己，容易护短。

这三个例子合起来说明一件事：**推理系统不是“装一个最强模型就完事”，而是按任务类型在“快/便宜”和“强推理”之间换挡。**

② 信息系统：现在知道什么、以前记得什么、做到哪了

F28 上下文 vs RAG：别把所有东西都塞进去

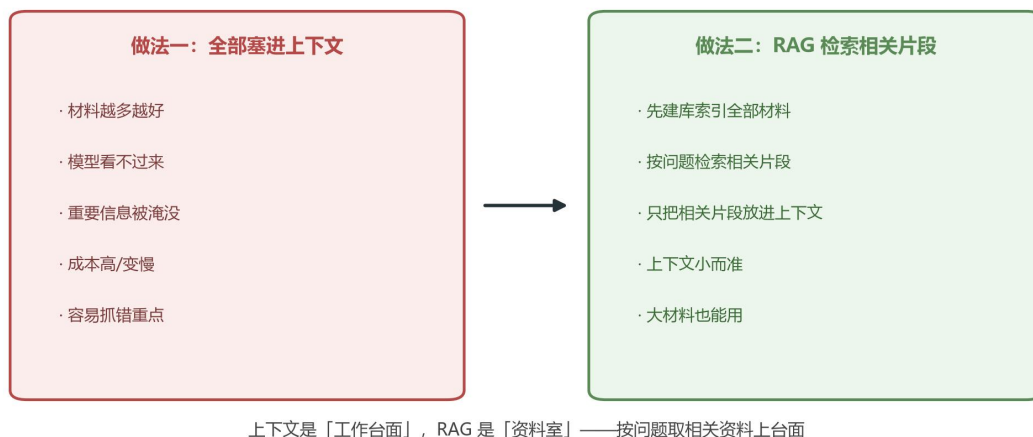


图 16 上下文与 RAG 关系

这个系统解决“信息”问题：**Agent** 这一次睁眼能看到什么、关了机还记得什么、要查自己资料库时去哪查、任务做到哪一步。四个概念从“临时”到“长期”、从“现成”到“检索”，排成一个连续谱。

6.4 Context：这一次睁眼看到的全部

是什么。 Context 就是模型“这一次睁眼时，眼前摊着的全部东西”：你这轮打的字、它刚才的回答、你拖进来的文件、搜到的网页、它刚读出来的文件内容。它只能基于眼前这些东西回答。模型自己“记不住你电脑里有什么”，也不知道你是谁。

为什么需要它。 记住一句话：**你不放进 Context 的东西，等于不存在。** 这也是前面讲过的“上下文污染”和“最小充分上下文”的根源——眼前堆太多会被带偏，给太少它会瞎编。你让它“改这篇论文的引言”，就必须把论文全文（至少引言）放进 Context，否则它只能凭空写一段漂亮但和你论文无关的话。

和谁最容易混。 和 Memory 混。Context 是“这一次睁眼看到的”，对话一结束就没了；Memory 是“跨对话留下来的”。一个活在当下，一个活在跨次。

6.5 Memory：跨对话留下来的

是什么。 每次新开对话，模型都是“失忆”的。Memory 就是平台帮你跨对话保留的那部分信息：你偏好的格式、你常用的背景、上次做到哪、它帮你总结过的笔记。

为什么需要它。 这样你不用每次从头自我介绍。两种主流做法：一种是产品自动把你说过的偏好存成“长期记忆”，下次自动带进来；另一种是你自己在项目里放一个说明文件（如 README、AGENTS.md 这类“岗位说明书”），让它每次开工前先读。后者本质上是你手写的 Memory，比自动记忆更可控。

和谁最容易混。 和 State 混。Memory 记的是“关于你”的长期事实；State 记的是“关于这件事进行到哪”的进度。你告诉过它“我是马克思主义理论方向、在做计算社会科学”——这是 Memory；它告诉你“已经处理完前 80 个文件”——这是 State。

6.6 State：任务进行到哪一步

是什么。 State 是任务进行中的中间信息：现在做到第几步、已经试了哪几条路、哪些文件已经改过、当前假设是什么。它介于 Context 和 Memory 之间——比一次对话的 Context 活得长，又不像 Memory 那样是“关于你”的长期事实。

为什么需要它。 这正是 PART 3 断点续跑那个 Mission 的核心。让它整理 200 个文件，中途崩了，有 State 的 Agent 下次开工能说“已经处理完前 80 个，从第 81 个继续”；没有 State 的只能从头再来。长任务里，学会时不时问一句：“你现在做到哪一步了？已经完成了什么？下一步打算干什么？”让它把 State 说出来，你才不会以为它停了、或重复劳动。

和谁最容易混。 和 Memory 混，上一节已说清。另外它和“结果文件”也要分开：State 是“进度记录”，不是“最终交付物”。

6.7 RAG：材料太多装不下怎么办

是什么。 RAG（Retrieval-Augmented Generation，检索增强生成）的做法是：先把你的资料切块存进一个“小图书馆”，每次提问时，先从里面挑出最相关的几段，再把这几段喂给模型回答。它解决的是“资料太多，Context 装不下”的问题。

为什么需要它。 平台先把文档切块、做索引（通常是把文字转成向量，再按相似度检索）；提问时先跑一次检索，把最相关的几段抓出来，连同你的问题一起放进 Context。所以 RAG 不是“让模型记住你全部资料”，而是“每次临时把相关几段翻出来给它看”。你把一学期 30 份 PPT 灌进 RAG，之后问“第三章讲的那个模型有什么局限”，它会先找出第三章相关段落再回答，而不是凭训练记忆瞎编。

和谁最容易混。 和 Memory 混。Memory 存的是零散的、关于你的偏好；RAG 存的是成体系的资料库，靠“相似度”临时翻出来。最大的风险是检索出来的片段不对题，所以要养成习惯：要求它每次回答都标“这段依据来自哪份文件哪一段”，标不出来源的，不要信。

现实世界实现（平台差异）

Context: 有的产品窗口大、长对话不太需要你管；有的窗口小，多聊几轮就“忘事”。编程类产品（Codex、Cursor）往往主动把整个项目文件结构读进来当 Context，对话类产品默认只放当前对话。差异不在“有没有 Context”，而在“它替你自动塞了多少、超了窗口怎么处理”。

Memory: 有的对话产品有显式“个人记忆”开关和查看页，你能看到它记了你什么、能随时删；有的编程产品完全靠项目里的说明文件，没有自动记忆。前者省心但可能记错，后者啰嗦但完全透明。

RAG: 有的产品提供现成“知识库”，拖文件进去就建好索引；有的需要你自己搭索引、自己配检索逻辑。差异在“谁来建索引”，原理都是“先检索、再生成”。

State: 偏“一次性对话”的产品基本没有显式 State，聊完就结束；偏“长任务/异步任务”的产品会有任务列表、断点续跑、中间产物留存。

真实例子：四个概念，落到一个文件上看

把这四个概念落到本教程 `examples/my-agent/` 的真实文件上，一眼就能分清：

- **Context**（这一次睁眼看到的）：就是 `agent.py` 里的 `self.history` 列表，每一轮 `chat()` 往里塞一条 `{"role": "user"/"assistant", "text": ...}`。对话一关，这个列表就没了。
- **Memory**（跨对话留下来的）：对应 `system_prompt.txt` 里那段“你是一个助教式 Agent，服务于零基础学习者……”，以及项目里手写的 README。它记的是“关于你、关于这个岗位”的长期事实。
- **State**（这件事做到哪了）：就是真实存在的 `state.json`，内容是 `{"task_status": "done", "turns": 8, "last_message": "检索笔记 Agent 零件"}`。下一次开工，`_load_state()` 读它，就知道上次跑到第 8 轮。这正是 **PART 3 断点续跑那个 Mission 的代码版**——没有这个文件，下次就得从 0 轮重来。
- **RAG**（材料太多临时翻出来）：对应 `tools.py` 里的 `search_notes`。`agent.log` 里真实跑过一次“检索笔记 工具 安全”，它返回 `note_02_tools_safety.md`（命中 21 次）、`note_04_plan_exec_eval.md`（12 次）、`note_03_rag_simplified.md`（9 次）——这就是“先从 docs/ 里挑最相关的几段，再喂给模型”的实物。注意：它是词频打分，不是向量检索，这是教学简化。

一句话：你不放进 **Context** 的东西等于不存在；而 **State** 让你崩了能从第 81 个文件接着做，**RAG** 让你不用把 30 份 PPT 全塞进窗口。

③ 行动系统：怎么作用于外部世界

这个系统是 Agent 直接碰现实世界的“原始手”。和扩展接口层的区别在于：这里讲的是**动作本身**，是未包装的原子能力。

6.8 Search：联网查最新事实

是什么。 模型的训练数据有截止日期，也不可能背下全世界所有网页。**Search** 就是让你像打开搜索引擎那样，去网上查最新的、具体的、它训练时没见过的信息。

为什么需要它。 凡是涉及“今天的事实”——最新政策、刚发布的论文、某软件现在的版本号——都该让它搜。让它查“2026 年某领域最新的综述”，它必须真的去搜，而不是凭训练记忆列几篇三年前的老文章。

和谁最容易混。 和 **Browser** 混。**Search** 是“输入关键词，看返回的链接和摘要”；**Browser** 是真的打开网页、点鼠标、翻页。**PART 4** 那个网页核验的 **Mission** 之所以要动用 **Browser**，正是因为 **Search** 返回的摘要不够、必须进到网站里把内容“读出来”。

6.9 Browser：真的打开网页操作

是什么。 很多信息藏在搜索引擎返回不了的地方：需要登录的后台、要点几下才展开的表格、要翻页的论坛帖子。**Browser** 就是让 Agent 真的打开一个浏览器，像人一样看屏幕、点鼠标、敲键盘。

为什么需要它。 “把某份公开榜单前 50 名连同公司名、成立时间、融资轮次抓成一张表”——搜索做不到（结果分散在十几页），浏览器可以逐页翻。它在真操作网页时，你应该在旁边看着第一步点的对不对，尤其是涉及登录、下单、发消息的页面。

和谁最容易混。 和 **Computer Use** 混。两者边界见下表。

6.10 Computer Use：直接操作整个桌面

是什么。 有些软件根本没有网页版、没有 API，只有桌面客户端，或是一个很老的系统。**Computer Use** 就是让 Agent 直接看你的整个屏幕、移动鼠标、敲键盘，像一个远程坐在你电脑前的人。

为什么需要它。 让它打开你电脑上某个只能用 **Windows** 客户端的老系统，按步骤改尺寸、导出图片——它真的会移动你的鼠标，你会在屏幕上看到光标自己动。第一次开这个能力时全程盯着，它动鼠标就是真的动你的鼠标，误点一个“确认删除”就是真删。

和谁最容易混。 和 **Browser**、**Code Execution** 三者最容易混，用一张表划清：

能力	活动范围	自由度	风险
Browser	只在浏览器标签页里	中	中
Computer Use	整个桌面、任何软件	最高	最高
Code Execution	隔离的代码沙箱	受限	较低

6.11 File System：读你硬盘上的文件

是什么。 你的论文、PPT、Excel、下载的资料都存在硬盘文件夹里。File System 访问就是让 Agent 能读、新建、改、整理你电脑上的文件。没有它，你只能一个个手动把文件拖进对话框。

为什么需要它。 PART 1 第一个 Mission“整理混乱的下载文件夹”，靠的就是这只手：它真的去建文件夹、移动文件，而不只是口头建议。原则是：永远告诉它“只在这个文件夹里工作，不要碰别的地方”，原始材料留备份，让它去改副本。

和谁最容易混。 和 Terminal 混。文件工具通常只做“读、写、列文件夹”这类相对安全的操作；Terminal 是更底层、更通用的命令通道。

6.12 Terminal：敲命令指挥电脑

是什么。 Terminal（终端/命令行/Shell）是你用鼠标点图标之外，另一种指挥电脑的方式：直接敲一行文字指令，电脑照做。Agent 要装软件、跑脚本、看系统状态、动 Git，很多时候靠敲命令完成。

为什么需要它。 让它跑一个开源项目，它可能敲一行把项目下载下来，再敲一行装依赖，再敲一行启动。中间任何一行写错，影响的都是你的真实环境。你自己一般不需要学敲命令，你需要会问：“你这条命令打算干什么？”尤其带删除、覆盖、联网下载含义的命令，让它解释一遍再放行。

和谁最容易混。 和 Code Execution 混。Terminal 在真实系统里，Code Execution 在隔离小房间里——这是两者最本质的区别。

6.13 Code Execution：在隔离沙箱里跑代码

是什么。 遇到算数、统计、画图表、清洗 Excel、批量处理文件这类活，让模型“心算”是靠不住的——它会算错、会编数字。Code Execution 是给它一个隔离的小环境，让它真写一小段代码、真运行、真把结果打印出来。结果是跑出来的，不是它猜的。

为什么需要它。 任何涉及精确数字的活——数据清洗、回归分析、画统计图、批量改名、合并表格——都让它用代码执行，而不是“看着表格估算”。并要求它把“跑出来的代码和结果”都留在工作文件夹里，方便你回头复查：代码能重跑，结果才算数。

和谁最容易混。 和 Terminal 混，上一节已划清。它和 File System 的关系：它读文件、写结果通常也在一个受限目录里，而不是你整台硬盘。

6.14 API：软件之间的标准对话窗口

是什么。 你想用某款软件的功能，但不想打开它的界面、不想手动点。API（Application Programming Interface，应用程序接口）就是软件之间“对话用的标准窗口”：你不用进厨房，通过传菜口下单，菜从窗口递出来。软件厂商对外开一组“接口地址”，你用一串身份凭证（API Key）去调用，它返回数据或执行动作。

为什么需要它。 让 Agent “把你飞书日历里明天的会列出来”，它就是通过飞书开放的 API，以你的身份去读日历数据。判断一个 Agent 强不强，API 能调哪些，本身就是能力的一部分。

和谁最容易混。 和扩展接口层的 Connector 混。API 是技术通道，需要自己填密钥、自己拼调用；Connector 是把这种通道包装好、给普通用户用的成品。API 属于“动作”，Connector 属于“接入和封装”——这正是本章反复强调的那条分界线。

现实世界实现（平台差异）

Search：有的产品默认常开，模型自己决定什么时候查；有的默认关，要你显式说“搜一下”；有的还接了学术库、代码库这类垂直搜索。

Browser / Computer Use：有的产品把浏览器操作做成一键工具；有的需要你开“网页操作”权限；有的能让你看到实时画面，有的在后台静默跑。Computer Use 有的在隔离虚拟机里跑，不碰你本机。能力越强，越要在隔离环境里开。

File System：有的只让它访问你主动拖进去的文件；有的允许访问整个项目文件夹；有的默认隔离沙箱，碰不到你本机真实目录。原则都是越窄越好。

Terminal：编程类产品（Codex、Cursor 等）几乎把它当标配，因为跑项目、装依赖、跑测试都要它；普通对话产品一般不给，或放在隔离环境里。有 Terminal 的产品风险也最高。

Code Execution：几乎所有主流产品都带某种代码执行，差别在沙箱里预装了哪些库、能不能联网、能跑多久。

API: 普通对话产品一般不让你直接手写 API 调用, 而是把常见服务封装成 Connector 或 Plugin; 编程类产品允许它自己写代码去调任意 API。

真实例子: 每只“手”都对应一件你真干过的事

行动系统的七只手, 每只都能落到一个你已经做过的 Mission 上:

- **Search ← PART 4:** 让它查“2026 年某领域最新综述”, 它必须真去搜, 而不是凭训练记忆列三年前的老文章。这是 PART 4 网页核验 Mission 的第一只手。
- **Browser ← PART 4:** PART 4 之所以要开 Browser, 是因为 Search 返回的摘要不够, 必须进到网站里把榜单逐页“读出来”。Search 给链接, Browser 才真翻页。
- **Computer Use:** 让它操作一个只有 Windows 客户端的老系统、移动你屏幕上的光标——第一次开全程盯着, 误点“确认删除”就是真删。
- **File System ← PART 1:** PART 1 第一个 Mission“整理混乱的下载文件夹”靠的就是这只手——它真去建文件夹、移文件, 不是口头建议。铁律: 只让它在指定文件夹里干活, 原始材料留备份。
- **Terminal:** 让它 `git clone` 一个项目、装依赖、跑测试 (LAB F 里 `git clone --depth 1` 就是这只手)。中间任何一条写错, 影响的是你真实环境。
- **Code Execution:** `examples/my-agent/tools.py` 里的 `calculator` 就是一只受限的手——它在 AST 白名单里算 `2+3*4` 得 14, 但你喂它 `__import__('os').system('echo hi')`, 它直接拒掉, 报错“表达式只允许数字和 `+ - * // %` 运算”。结果是跑出来的, 不是它猜的。**
- **API:** 让它读你飞书日历明天的会, 走的就是飞书开放 API。判断一个 Agent 强不强, API 能调哪些本身就是能力。

一句话: 这七只手, 从“只读一个网页”到“动你整个桌面”, 自由度和风险一路上涨; 越往右, 越要在隔离环境里开。

④ 编排系统：谁决定先做什么后做什么

F26 核心执行循环

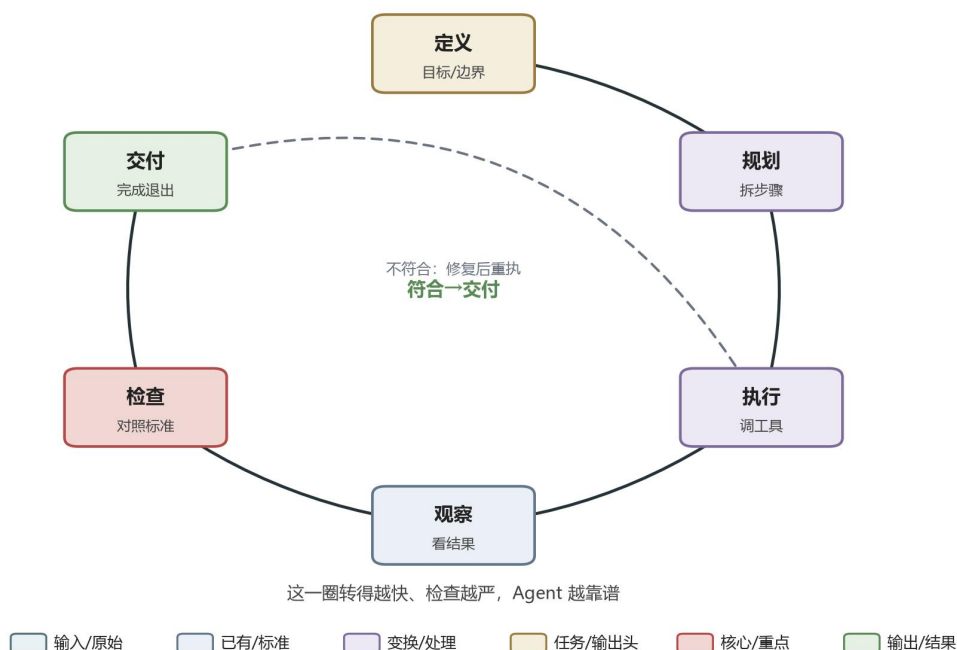


图 17 核心执行循环

前三个系统是零件，这个系统是“项目经理”：把零件组织成一次真实任务。它回答的不是“能不能做”，而是“先做什么、后做什么、要不要派人、要不要停下来问人”。

6.15 Workflow：把多步任务固定下来

是什么。 复杂任务不是一句话说完的，而是一连串步骤：搜、筛、读、建表、分析、写、查。Workflow 就是把这一串步骤固定下来，下次同类任务直接按这条流水线走，不用每次重新指挥。

为什么需要它。 当你发现自己每周都在重复同一套多步操作时，就该把它画成一张箭头图：第一步出什么、第二步凭什么、最后怎么验收。“每天早上 8 点，搜今天的行业新闻，和昨天的摘要对比，有重大变化才把摘要推给我”——这就是一条固定 Workflow。

和谁最容易混。 和 Automation 混。Workflow 是“步骤怎么排”（菜谱），Automation 是“什么时候自动跑这条 Workflow”（定时器）。

6.16 Planner：让它自己规划路线

是什么。 Planner（规划器）是编排里更“自主”的一面：不预设死步骤，而是让 Model 在开工前先列一份计划，执行中再根据结果调整。它和 Workflow 的关系是：Workflow 是你写死的流水线，Planner 是它自己现想现走。

为什么需要它。 面对“调查 X 公司”这种步骤事先排不全的任务，硬套一条死 Workflow 反而捆住手脚；让它先规划、再执行、边执行边修正，更合适。但 Planner 的风险是它会跑偏、会自作主张加步骤，所以关键节点要设“停下来给你看”的检查点。

和谁最容易混。 和 Workflow 混（如上），也和 Sub-agent 混。Planner 决定“路线”，Sub-agent 决定“派谁去走某一段”。

6.17 Sub-agent：临时派出去的分身

是什么。 一个 Agent 自己干长了，会顾不过来、会忘、会串味。Sub-agent 就是主 Agent 临时派出去的“分身”：一个负责搜资料，一个负责读文件，一个负责写初稿，最后主 Agent 把三份结果汇总。分身之间默认互不可见，只通过主 Agent 传话。

为什么需要它。 主 Agent 接到“调查 X 公司”，派子 Agent A 查工商信息、B 查新闻舆情、C 查产品；A、B、C 各交一份摘要，主 Agent 合并成最终报告。但记住：分身越多，汇总成本和出错率越高。新手先把单 Agent 跑顺，再考虑拆分；派出去的分身，交回来的东西也要验收——它不会因为叫“子 Agent”就自动可靠。

和谁最容易混。 和 Planner 混（如上）。它 also 容易被和“多开几个对话”混为一谈：区别在于 Sub-agent 是系统帮你派、帮你汇总；你自己手动开多个对话再自己拼，那不叫 Sub-agent，那叫手动编排。

6.18 Automation 与 Human Approval：什么时候自动跑、什么时候停下问你

是什么。 Automation 是把 Workflow 接上一个“闹钟”或“触发器”：定时跑、收到邮件跑、网页变了跑。Human Approval（人工确认）则是反过来的一道闸：到了关键步骤，它必须停下来等你点头才继续。

为什么需要它。 第一版自动化一定先设成“跑完先把结果给我看，别自己发出去”。跑稳两周、你确认它不会乱来，再逐步放开发送权限。这和 PART 前面讲过的验收原则一脉相承：越是不可逆的动作，越要把 Human Approval 这道闸留在最后。适合全自动的事有规律、触发条件明确、且低风险；高风险、需要你每次判断的事不适合。

和谁最容易混。 和 Workflow 混（如上）。它也常被和“权限”混：Automation 决定“要不要停下来问”，权限分级（PART 7 的 L1-L4）决定“哪类动作必须问”——两者配合，但不是一回事。

现实世界实现（平台差异）

Workflow： 偏自动化的产品有专门的工作流画布，节点、条件、循环都能配；普通对话产品

没有显式 Workflow，靠你在任务里口述步骤。脑子里的流程和画布上的流程，原理相同。

Sub-agent: 有的产品有显式“派子任务”机制；有的靠你手动开多个对话再自己汇总；编程类产品常把不同文件、不同模块分给不同子 Agent。差异在“分身是系统帮你派，还是你自己拆”。

Automation: 有的产品有专门“自动化/定时任务”页，能设 cron 式时间表；有的只能在工作流画布里接触发器；普通对话产品基本没有 Automation。差异在“触发器好不好设、能不能无人值守”。

Human Approval: 有的产品在执行 L3/L4 动作前自动弹确认；有的默认全自动，需要你自己设置里把关键动作设成“需人工确认”。

真实例子：编排系统的四个零件，落到 my-agent 和 daily_monitor 上

- **Workflow（步骤固定）：** examples/my-agent/pipeline.py 的 run_task() 就是一条小流水线——先 plan() 把任务拆成步骤，再 execute() 顺序调用工具，最后 evaluate() 用断言验收。下次同类任务直接走这条线。
- **Planner（它自己现想现走）：** 同一个 pipeline.py 里的 plan()，按关键词把“计算 $2+3*4$ 并检索笔记 Planner”拆成两步：先算、再检索。教学版是模板，真项目里换成强推理模型来规划。
- **Sub-agent（临时派分身）：** 主 Agent 接到“调查 X 公司”，派 A 查工商、B 查舆情、C 查产品，各自交摘要，主 Agent 合并。分身越多，汇总成本越高——新手先把单 Agent 跑顺。
- **Automation + Human Approval:** examples/automation/daily_monitor.py 是 Automation 的活（每天 8 点抓 GitHub Release）；agent.py 里的 DANGEROUS_PATTERNS = [“删除”, “发送”, “覆盖”, ...] 命中就走 _ask_approval(), 停下问你 y/N——agent.log 里真实记录了两条：删除旧备份 config.bak 被批准、发送全部客户邮件 被拒绝。第一版自动化一定先设成“跑完给我看，别自己发出去”。

一句话：前三个系统是零件，编排系统是把零件组织成一次任务的项目经理；Human Approval 就是它在关键节点必须停下来等你点头的那道闸。

扩展接口层：能力怎么接进来

四个系统回答了“Agent 本身是什么”，还有最后一个问题：它能用的这些能力，是从哪接进来的？这就是扩展接口层。

它不是“第五个系统”。系统是 Agent 本身的器官，接口层是它向外接线、封装能力的机制。API、Connector、Plugin、MCP、Skill 都长在这一层——它们不是并列的五种零件，而是“接入和封装”的五种不同实现。下一章（PART 7）会专门讲怎么安全地发现、安装、验证、撤销它们，这里只先把概念边界划清。

- **API**：最底层的技术通道。你自己填密钥、自己拼调用。它是“传菜口”本身。
- **Connector**：把 API 包装好、给普通用户点一下就能用的成品线。通常走 OAuth 登录授权，偏“把你个人的某个账号接进来”。
- **Plugin**：平台定义的一类扩展包。**注意它不必然是“官方商店成品”**——不同平台的 Plugin 内部可能包含 Tool、Skill、Connector 等不同能力，边界以你所用平台的定义为准。它是“平台帮你打包好的一块功能”。
- **MCP（Model Context Protocol）**：一套让工具按统一标准暴露自己的协议。它**不是“写一次到处可用”的魔法**，而是“降低多客户端重复适配的成本”；具体兼容性仍取决于客户端支持的 MCP 版本、transport、auth 和 capabilities。它是“传菜口的统一规格”。
- **Skill**：把“怎么做某件专业事”写成一个可复用的包（一份 SKILL.md 加模板、脚本、示例）。它更像一份“操作手册加工具箱”，而不是一个按钮。

一个类比收尾：API 是传菜口，MCP 是传菜口的统一规格，Connector 是餐厅已经帮你接好的成品线，Plugin 是餐厅直接给你上的一桌菜，Skill 是一份“这道菜该怎么炒”的手册。这五个词在不同平台经常混用，别纠结名字，看它实际碰了什么数据、干了什么活。

为了把这层讲死，下面这张表把五种实现放在同一张坐标里比。读它的方法：从左到右，是“从最底层技术通道”到“最像一份说明书”的光谱；越靠左越要你自己动手、越靠右越像点一下就能用，但封装得越好、你能审计的空间反而越小。

维度	API	Connector	Plugin	MCP	Skill
它是什么	最底层技术通道	把 API 包成“点一下”	平台定义的扩展包	工具自描述的统一协议	一份操作手册加工具箱
你要做的动作	自己填 Key、自己拼请求	点“连接”+走 OAuth 授权	在商店点“安装”	在配置文件写一段+重启	丢一个文件夹+刷新识别
它碰什么数据	按你写的请求决定	你授权的那个账号	包内作者定义	Server 自己暴露的目录/接口	被指示去读的文件
要什么钥匙	API Key（自己管）	账号 OAuth 通行证	平台商店账号	配置里那把 Key	一般不用钥匙

维度	API	Connector	Plugin	MCP	Skill
失败长什么样	401/403、拼错参数	授权页 scope 勾错	装上却要额外权限	配了连不上、版本不兼容	放对位置却没被识别
怎么撤	去服务商后台删 Key	账号侧 Settings 里 Revoke	商店里卸载	删配置那一段再重启	删文件夹再确认无残留
本教程的真实例子	自己写代码调飞书 API	点“连接 GitHub”走 gh 授权	装官方 PDF 插件	filesystem MCP 暴露一个沙箱目录	minimax-pdf 这类 SKILL.md

一句话压表：**API** 是你亲手接线，**Connector** 是别人帮你接好线，**Plugin** 是别人连菜都炒好端上桌，**MCP** 是给所有传菜口定了同一个尺寸，**Skill** 是塞给你一本菜谱。越往右越省心，越要在“装之前”把 7.3 的 12 问过一遍——因为封装得越好，里面干了什么你越看不见。

本章 Mission

Mission 6-A: 给你手上的 Agent 画一张“四系统+接口层”图

拿你最近在用的那个 Agent，逐格回答：推理系统用哪个 Model、看重哪把尺子？信息系统这一轮能看到哪些 Context、有没有 Memory、要不要 RAG、长任务有没有 State？行动系统开了哪几只手（Search、Browser、File、Terminal、Code、API）？编排系统是 Workflow 还是 Planner、会不会派 Sub-agent、哪些动作会停下来问你？扩展接口层接了哪些 Connector / Plugin / MCP / Skill？你会立刻发现自己之前一直在“裸用”，很多能力其实没开。

Mission 6-B: 判断一次失败到底卡在哪个系统

回想最近一次它没做好的任务。对着四系统判断：是推理系统 Model 不够聪明？还是信息系统没给够（漏了材料、该检索没检索）？还是行动系统手没开（该让它搜它没搜）？还是编排系统乱了（步骤没串好、分身派错了）？定性之后，下次任务对症下药，而不是只会重写一遍 Prompt。

Mission 6-C: 分清三对最容易混的概念

用自己的话写三句话：Context 和 Memory 差在哪？Search 和 Browser 差在哪？MCP 和 Connector 差在哪？写不出人话版，就回去重读对应那一节。这三对分清了，这一章就过关了。

PART 7: 能力供应链——怎么安全地给 Agent 加能力

F15 Skill 安装安全审查：12 问流程

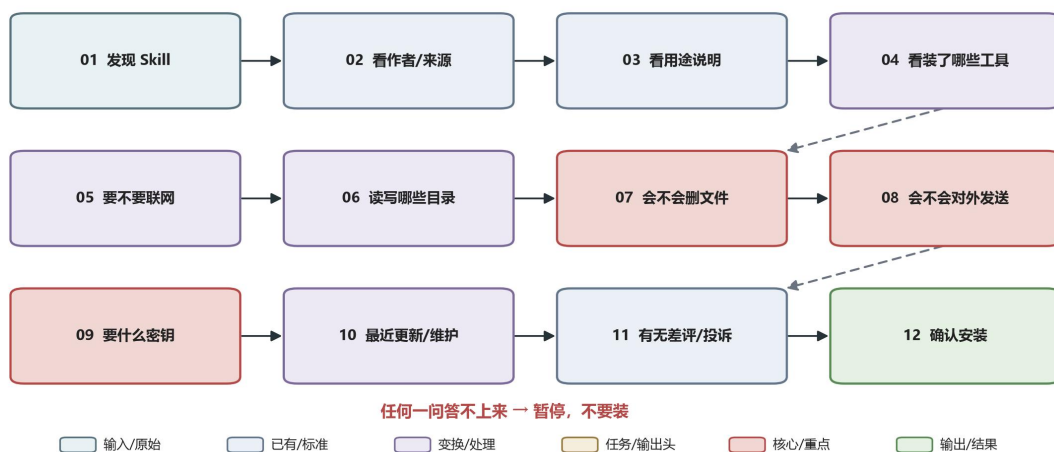


图 18 能力供应链闭环图

本章你要解决什么问题

上一章讲清楚了 Agent 由哪些系统组成，以及能力是靠最外面那圈接口层接进来的。本章解决一个非常实际的问题：这些能力不是出厂全装好的，很多要你自己去“插上去”。

但这一章的主线，不是教你“怎么装”，而是教你把装能力当成一条能力供应链（Capability Supply Chain）来管：

我缺一种能力 → 去哪里发现 → 如何判断这个来源靠不靠谱 → 如何读懂它要的权限 → 如何安装 → 如何在沙箱里先验证 → 如何升级 → 如何撤销。

Skill、Plugin、MCP、Connector 只是这条供应链上不同的“包装形式”。贯穿全章的铁律只有一句：

不要看到社区资源就装，而是会审计。GitHub 上的 Star 数不等于安全，商店里的“热门推荐”不等于对你合适。每装一个，你都要知道它碰了什么数据、能干什么动作、钥匙放在了哪里。

7.1 能力从哪来：模型出厂，扩展后来

F12 能力供应链：从「发现一个能力」到「安全用上它」

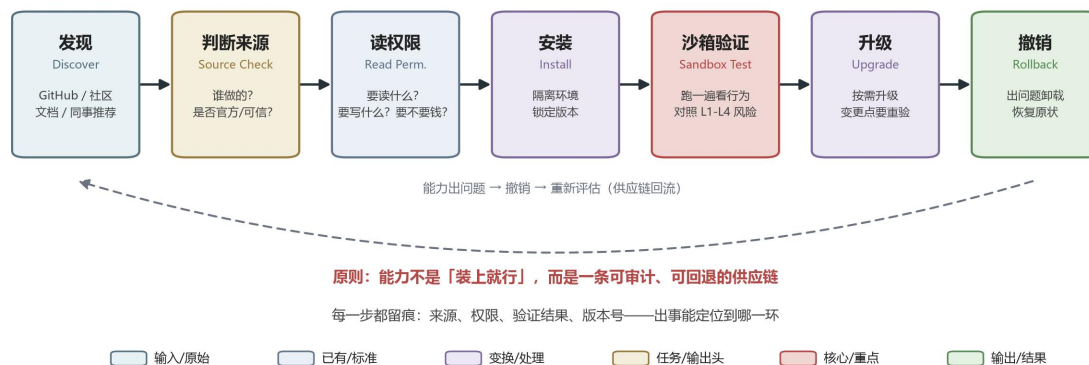


图 19 能力扩展层级

模型出厂时只会做一件事：读一段文字，吐一段文字。它不能读你电脑里的论文，不能查今天的新闻，不能动你的 Excel，也不能替你发邮件。这些“超能力”全是后来通过接口层接上去的。

所以判断一个 Agent 到底强不强，不要只看它用了多牛的 Model，而要看：它接了哪些 Tool、装了哪些 Skill / Plugin / MCP、连了哪些你的账号、你给了它多大操作权限。模型决定天花板，扩展能力决定它能不能下地干活。

7.2 发现：三层资源地图

F13 社区资源生态：Agent 能力从哪里来

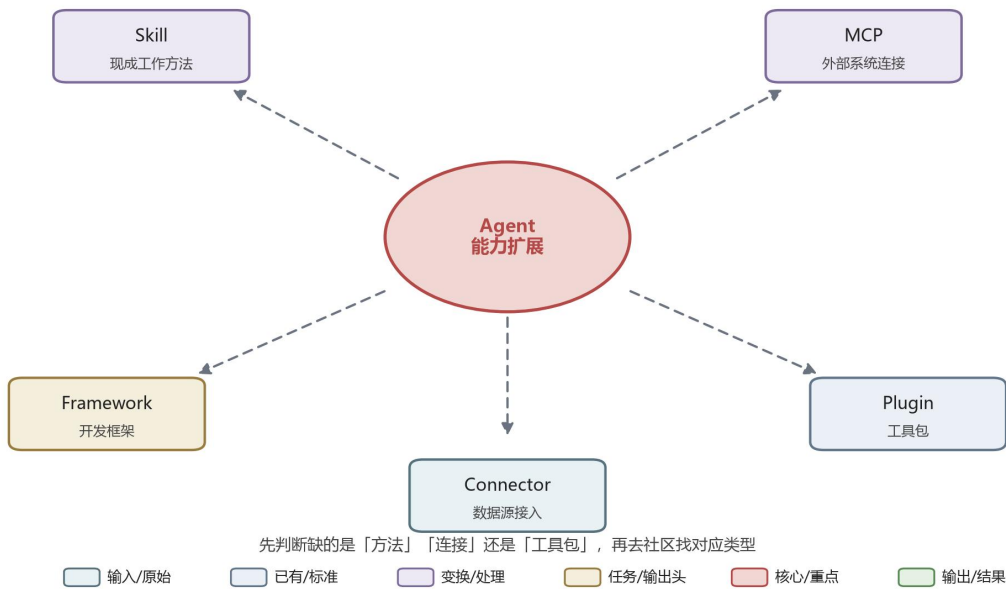


图 20 社区资源生态

不要靠刷社交平台碰运气。在心里建立一张资源地图，按三层去找，越靠上越可信：

- 1. **官方来源：**你所用 Agent 平台的官方文档、官方 Skill 商店、官方 GitHub 组织、官方 MCP 注册中心（registry.modelcontextprotocol.io）。信息最准、维护最勤，先查这里。
- 2. **聚合发现平台：**社区目录（如 SkillsMP、skills.sh 这类）会聚合公开的 Skill、MCP、Plugin，方便你搜索。它们的价值是“让你发现”，**不是让你直接装**。看到感兴趣的，记下来，回到第三层。
- 3. **源仓库：**打开这个 Skill / MCP 的 GitHub 或官方源码页，看谁做的、最近提交时间、Star 和 Issue 活跃度、README、License、有没有脚本、要什么权限。

一句话：**发现靠聚合站，决定靠源仓库。**聚合平台上的排名、评分、“热门”“推荐”，只代表它被很多人发现，不代表它安全、适合你、还在维护。

7.3 判断来源：读 SKILL.md 加 12 问

SKILL.md 就是一个 Skill 的“产品说明书加岗位说明书”，纯文本、几百行以内，用记事本就能打开。打开后按这个顺序读：

你要找的信息	它通常写在哪	你要判断什么
这个 Skill 是干嘛的	开头简介	和你要做的事对不对得上

你要找的信息	它通常写在哪儿	你要判断什么
支持哪些 Agent / 平台	安装说明	你用的平台装不装得上
前置条件	Requirements / Prerequisites	要不要额外装软件、要不要账号
读哪些文件、写哪些文件	权限说明 / 工作方式	会不会动到你的原始材料
会不会联网、会不会调付费 API	网络/费用说明	会不会偷偷跑钱、外发数据
附带什么脚本	Files / Scripts	脚本是最需要警惕的部分
怎么卸	Uninstall	装之前先确认能撤

读完后，在心里过一遍这 12 问：

1. 来源是谁？是知名公司官方仓库，还是匿名账号？
2. 源代码在哪里？能不能看到完整代码，而不是只有“一键安装”？
3. 最近有没有维护？最近一次提交是上周还是三年前？
4. 它会执行什么脚本？有没有和宣传功能无关的操作？
5. 它能读什么文件？会不会去翻你的文档、桌面、下载文件夹？
6. 它能写什么文件？会不会往系统目录、配置目录塞东西？
7. 它能访问哪些网站 / API？除了该用的服务，还会不会连到别的地址？
8. 它需要哪些账号权限？连邮箱时是“只读”还是“能发邮件”？连 GitHub 时是“只读代码”还是“能替你 push”？
9. 它会不会把我的数据发到第三方？
10. 它会不会调用付费 API？跑一次花多少钱、有没有上限？
11. 它有没有删除、覆盖、修改能力？误操作能不能找回来？
12. 能不能先在测试目录跑一遍？

特别强调两句话：**Star 数不等于安全**（热门只说明流行，不说明干净）；**“官方”两个字要看清楚**（有些聚合站名字带 Official，不代表它是某家公司官方办的，认准域名和 GitHub 组织名）。

打开一个 Skill 文件夹，里面通常有三类东西：说明文档（最安全，纯文本）、模板和资产（半成品、示例，本身不会自己跑，风险低）、脚本（真正会被执行的代码，风险最高）。看不懂脚本写了什么，就把它丢给 Agent，问：“这段脚本从头到尾干了什么？有没有删文件、联网、发数据的动作？”

7.4 读权限：用 gh CLI 把 OAuth 和 scope 看清楚

当你点“连接邮箱”时，跳出来的登录页问你“是否允许 XX 应用访问你的邮箱”——背后走的是 **OAuth**：你不把账号密码告诉这个应用，而是通过官方登录页给它发一张有时效的通行证。

要把这件事看清楚，最适合当“活标本”的是 GitHub 官方命令行工具 **gh**。**注意：gh CLI 本身是一个开发者工具，不是一个 Connector；这里用它，只是因为它把 OAuth 的授权、scope、撤销全过程暴露得最清楚，方便你理解任何 Connector 的授权页。**

在这台 Windows 机器上真实走一遍：

第一步：看当前授权状态。

```
gh auth status
```

真实输出大意：已登录到 github.com，当前账号为 TZUKWAN，token 存在系统密钥环里，当前 token 的权限范围（scopes）是 `gist, read:org, repo, workflow`。这串 scopes 就是“OAuth 授权边界”的实物——你一眼能看到：它能读写仓库（repo）、能操作工作流（workflow）、能读组织信息（read:org）、能写 Gist（gist），但它**没有** delete_repo 这种删除权限，也没有 admin:org 这种管理员权限。这就是授权该有的样子：能做的列得清清楚楚，不该有的没有。

第二步：理解每个 scope 的含义。

它请求的 scope	翻译成大白话	该不该给
repo	能读、写你的仓库代码	你要让它动代码，就给
workflow	能改 GitHub Actions 工作流	你要让它自动跑 CI，才给
read:org	只读你所在组织的信息	一般够用
delete_repo	能删仓库	没有明确需要就别勾

第一次跑 `gh auth login` 选浏览器登录，它会打开 GitHub 官方页面，列清楚这次要哪些权限，你同意后才拿到通行证。**关键不是这个机制多方便，而是你在那个页面上要逐行看：它要的每一项权限，是不是你真打算给的。**

第三步：事后收权或撤销。

```
gh auth refresh -s read:org # 重新走一次浏览器授权，收缩或扩大 scope
gh auth logout              # 一键撤销本机这张通行证
```

更彻底的做法，是直接去 GitHub 网页的 Settings → Applications → Authorized OAuth Apps 里，找到这个应用，点 Revoke。这和后面 7.9 讲的“去账号侧撤销授权”是同一个动作。

Notion、Slack、Google Workspace 这类真正的 Connector，本机无法替你登录授权（需要你本人账号、验证码、二次确认），但流程和 gh 一模一样：点连接 → 跳到官方登录页 → 逐行看权限 → 同意 → 回来核对授权状态。差别只在登录页是哪家公司的，授权那一页你要做的事完全相同。

7.5 安装：四种实现各是什么

接口层的四种包装形式，安装动作各有侧重。别把它们想成“四种零件”，它们是同一条供应链上四种不同的封装。

7.5.1 装 Skill

通用动作四步：找到它 → 读它的 SKILL.md → 放进指定位置（通常是一个叫 skills 的文件夹，不同平台路径不同）→ 重启或刷新让它认识。

以本机真实目录走一遍（命令路径是这台机器的真实情况，照做时换成你自己的用户名和平台路径）。先看本机有哪些 Skill：

```
Get-ChildItem "$env:USERPROFILE\.claude\skills" -Directory | Select-Object Name
```

本机真实存在的包括 arxiv-research、minimax-pdf、find-skills、skill-creator 等。
按关键词找：

```
Get-ChildItem "$env:USERPROFILE\.claude\skills" -Directory | Where-Object { $_.Name -match 'pdf|research' } | Select-Object Name
```

源仓库核验：看一个已装 Skill 的最后提交时间（仅当它本身是 git 仓库时才有输出）：

```
git -C "$env:USERPROFILE\.claude\skills\minimax-pdf" log -1 --format="%ci %s"
```

权限审计：打开 SKILL.md 前 60 行，再看它带了哪些文件、有没有可执行脚本：

```
Get-Content "$env:USERPROFILE\.claude\skills\minimax-pdf\SKILL.md" -TotalCount 60  
Get-ChildItem "$env:USERPROFILE\.claude\skills\minimax-pdf" -Recurse | Select-Object  
FullName, Length
```

判断标准回到 7.3 的表和 12 问：出现“联网下载”“删除文件”“读环境变量外发”这类字眼，先打问号。装完测试：重启 Agent，对它说“请读一下这个 Skill 的 SKILL.md，然后用它处理我这个 PDF”，再丢一个小 PDF 试一次。

标注：“在你自己的 Agent 产品里加载并测试”这一步，本教程运行环境无法替你完成——那是另一个图形化产品里的交互，不在这台命令行机器上。你需要在自己的产品里实际开一次，确认它真的认识这个 Skill、真按说明书走了。

7.5.2 装 Plugin

先纠正一个常见误解：Plugin 不必然是“官方商店成品”。准确说法是：Plugin 是平台定义的一类扩展包；不同平台的 Plugin 内部可能包含 Tool、Skill、Connector 等不同能力，边界以你所用平台的定义为准。

落到安装动作上：Plugin 通常来自平台官方商店，你在界面上点“安装”，它就挂上了。相对省心，但“平台商店”不代表“绝对安全”——你依然要在安装弹窗里看一眼它要什么权限、会不会收费、会不会读你的账号数据。商店审核一般会拦住明显的恶意件，但审不到“这个插件是否过度收集你的数据”这种灰色问题。

7.5.3 添加 MCP Server

再纠正一个常见误解：MCP 不是“写一次到处可用”的魔法。准确说法是：MCP 降低了多客户端重复适配同一工具的成本；但一个 MCP Server 能不能在你的客户端里正常工作，仍取决于客户端支持的 MCP 版本、transport（stdio / HTTP / SSE 等）、auth 方式和 capabilities。所以配完一定要实测连通性，不能“配置写对了就以为能用了”。

添加方式：在你的 Agent 配置文件里写一段“我要用哪个 MCP Server、它怎么启动、钥匙放哪”，改完重启。不同平台配置文件位置和格式不同，**不要照抄网上三年前的教程**，以你所用客户端官方文档为准。

以官方维护、用途明确的 filesystem MCP Server 为例（它只做一件事：把你指定的一个文件夹暴露给 Agent 读写，适合第一次试手）。在常见的 Claude 系客户端配置 JSON 里加一段（格式只作示意，键名和路径以官方文档为准）：

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "D:\\AISOP\\mcp-sandbox"
      ]
    }
  }
}
```

这段配置的关键，是把它可访问的目录**限定**在 D:\AISOP\mcp-sandbox 这一个文件夹。这一步是 MCP 安全的核心：只给它一个专用沙箱目录，不要把整个用户目录或 C 盘根目录填进去。

改配置前，先单独跑一下启动命令，确认包能拉起来、不报错：

```
npx -y @modelcontextprotocol/server-fileSystem "D:\AISOP\mcp-sandbox"
```

它启动后通常会停在那里等标准输入（因为 MCP 是通过 stdin/stdout 对话的进程），看到它没报错、没立刻退出，就说明这一步通了，按 Ctrl+C 退出。

标注：“用一个最小 MCP 客户端脚本真正连上、列出工具、调用一次”这一步，需要你在自己的开发环境里写几行客户端代码，或用客户端自带的连接测试功能来做。原理是：启动上面那个进程，按 MCP 协议握手，让它报出自己提供了哪些工具（一般是 `read_file`、`write_file`、`list_directory` 之类），再调一次 `list_directory` 看能不能列出沙箱目录里的文件。本教程运行环境不替你执行这一步。如果连不上，先查三件事：`npx` 在不在、目录路径写对没有、客户端有没有重启加载新配置。

添加 MCP 同样要过 12 问：这个 Server 是谁写的？它要访问什么？要什么密钥？会不会把数据转给第三方？一个标着“MCP”的开源 Server，本质和一个 Skill 一样，是会跑的代码。

7.5.4 添加 Connector

Connector 是最贴近普通用户的扩展方式：界面上一个“连接邮箱”“连接日历”“连接云盘”的按钮，点一下，跳到你那个账号的登录页，授权，回来就连上了。它把技术细节包起来了，你不用碰配置文件——但授权那一步要做的事，和 7.4 用 `gh` CLI 演示的一模一样：逐行看 `scope`，别直接滑到底点同意。

7.6 密钥放哪：环境变量的真实定位

先纠正一个误解：环境变量不是“保险箱”。它真正解决的问题是“不要把 Secret 写进代码和 Git”——仅此而已。真正的密钥保护，应优先使用系统凭据库、Secret Manager，或你所用平台自己的 Secrets 功能。

为什么这么说？把 API Key 直接写在对话 Prompt 里，是最危险的做法：你在对话框里贴一次，它就可能被记进聊天历史、被模型日志带走；写进代码再提交到 GitHub，业界每年都有成百上千个 Key 因为误传公开仓库被自动化扫描器扫走、被盗刷。环境变量的作用，是让 Agent 要用 Key 时按约定的名字去取，而不是每次从对话里读到、更不是写进代码提交。

在 Windows 上临时看一眼当前有哪些 Key 类环境变量名（只看名字，不要把值打印出来）：

```
Get-ChildItem env: | Where-Object { $_.Name -match 'KEY|TOKEN|SECRET' } | Select-Object Name
```

各平台设置环境变量的入口不同（有的在设置页，有的在配置文件里，有的在系统环境变量里），以平台文档为准。你只需要建立这个习惯：

凡是钥匙类的东西（API Key、Token、密码），优先用系统凭据库 / Secret Manager / 平台 Secrets；代码和 Git 里不写死、对话里不粘贴。环境变量是中间手段，不是终极保险箱。

另外记住 API Key 的三条铁规矩：不要写进普通对话 Prompt；不要提交到公开仓库；Key 要能随时作废、随时重发（每个服务商后台都有“重新生成 / 作废旧 Key”的功能，怀疑泄露就立刻换）。

7.7 沙箱验证：先在干净地方跑一次

供应链里最容易被跳过、却最关键的一环是**沙箱验证**：装完别急着拿真材料试，先在一个干净的、可丢弃的文件夹里跑一次。

做法很朴素：

1. 建一个专用沙箱目录（如上面 MCP 例子里的 D:\AISOP\mcp-sandbox），往里放一份副本文件，不要用原始材料。
2. 让新装的 Skill / Plugin / MCP 在这个目录里干一次它该干的活。
3. 观察三件事：它有没有跑出你预期的结果？它有没有去碰沙箱以外的目录？它有没有发起你没预料到的联网请求或付费调用？
4. 都正常，再回到真实任务里用；任何一项不对，按 7.9 撤销。

7.8 最小权限：L1-L4 风险分级表

F14 第三方资源风险分级 L1-L4



图 21 L1-L4 风险分级

装上了能力，就要管住它能干什么。核心原则一句话：只给完成这件事所需的最小权限，多一分都不给。

把 Agent 的所有可能动作，按“做错了能不能收回来”分成四级：

风险级别	典型动作	默认策略
L1 低风险	读文件、搜索网页、分析数据副本、读你给的材料	可以自动执行，不用每次问你
L2 可逆修改	新建文件、改工作副本、在 Git 上建分支、改草稿	可以执行，但要留记录（改了什么、改前是什么）
L3 高影响	覆盖你原来的文件、Git push 到远程、改云端共享文档、替你发消息或邮件	建议每一步都停下来让你确认
L4 不可逆 / 高风险	删除数据、支付、公开发布、改生产数据库、改权限、对外发正式公告	必须你明确说“确认执行”才做，且最好先备份

一个朴素的判断：凡是错了找不回来的事，最后那一下永远你自己按。

7.9 升级与撤销：让供应链能闭环

7.9.1 撤销 / 卸载

装上去的东西，要能撤下来。三种动作要分清：

- **临时禁用**：Agent 里往往有开关，把某个 Skill / Plugin / MCP 关掉，它暂时不生效，但文件还在。适合“现在不想用，以后可能还开”。
- **卸载**：把这个扩展从 Agent 里移除。适合“确定不用了”。以本机 Skill 为例，删掉文件夹再确认无残留：

```
Remove-Item "$env:USERPROFILE\.claude\skills\minimax-pdf" -Recurse -Force
Test-Path "$env:USERPROFILE\.claude\skills\minimax-pdf"
```

第二条应返回 `False`。但光删文件夹不算完整卸载——如果它在配置文件里登记过、或连过你的账号，还要去配置里移除、去对应账号撤销授权。

- **撤销账号授权**：如果它连了你的邮箱 / GitHub / 云盘，去对应账号的第三方授权页把它踢掉——光在 Agent 里卸载，有时你那个账号上的授权还挂着。

一个好习惯：每季度花十分钟，去常用账号（邮箱、GitHub、云盘、社交平台）的“第三方应用授权”列表里扫一眼，把不认识、不再用的授权全部撤销。

7.9.2 升级

社区件会更新：修 bug、加功能、改权限。但**不要开启全自动更新**。理由很简单：一个昨天还干净的 Skill，今天的版本可能加了联网、加了付费调用。正确姿势：

1. 更新前先看更新说明（`changelog / release notes`）——这版改了什么？
2. 尤其注意“新增了什么权限、新增了什么网络请求、新增了什么依赖”。
3. 更新后跑一次你常用的 Mission，确认它没跑偏。
4. 出问题就回到上一个版本。

平台官方商店的 Plugin 更新相对省心，但也别假设“官方更新 = 对你更好”。

7.9.3 完整走一遍：用 filesystem MCP 把供应链八环走穿

前面七节是拆开讲零件，这一小节把它们串成一条真实操作链。**目标能力**：让 Agent 安全地读写 `D:\AISOP\mcp-sandbox` 这一个文件夹，别的地方碰不到。照着走一遍，以后装任何 Skill / Plugin / MCP 都是这条链。

第 0 环：我缺一种能力。 我发现每次都得手动把文件拖进对话框，想让 Agent 自己在一个专用文件夹里读写。需求说清楚：**只要一个文件夹的读写权，不要全盘。**

第 1 环：发现来源。 先去官方 MCP 注册中心 `registry.modelcontextprotocol.io`，搜 `filesystem`，找到官方维护的 `@modelcontextprotocol/server-filesystem`。不是从聚合站搜一个 Star 最多的就装。

第 2 环：判断可信度。 打开它的源仓库：作者是 ModelContextProtocol 官方组织、License 是开源许可、最近有提交、Issue 区有人在维护。再读一遍它的 README——它只做一件事：把你指定的目录暴露出去。这一步过 7.3 的 12 问，重点看第 4、5、6、7 问：它跑什么脚本、读哪些文件、写哪些文件、连哪些地址。

第 3 环：读权限。 它不要 OAuth（不是连你个人账号），但它要“文件系统读写”这只手。关键动作是：在启动参数里只填那一个沙箱目录，把权限收窄到最小：

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "D:\\AISOP\\mcp-sandbox"]
    }
  }
}
```

注意那个 `D:\\AISOP\\mcp-sandbox`——这就是最小权限的实物。填成 `D:\\` 或用户根目录，就等于把整盘交给它。

第 4 环：安装。 把上面这段写进你客户端的 MCP 配置文件，重启客户端让它认识。

第 5 环：沙箱验证。 别急着接真活。先单独跑启动命令，确认包拉得起来：

```
npx -y @modelcontextprotocol/server-filesystem "D:\\AISOP\\mcp-sandbox"
```

它启动后停在那里等标准输入（MCP 走 `stdin/stdout` 对话），看到没报错、没立刻退出，按 `Ctrl+C` 退出。然后在客户端里调一次 `list_directory`，确认它只能列出沙箱里的文件；再试一次让它读沙箱外的路径，确认它拒掉。三样都对：结果对、不越界、没有没预料的联网或付费。

第 6 环：升级。 它更新时，先看 `release notes` 改了什么，尤其看“新增了什么权限、什么网络请求、什么依赖”。不开自动更新——昨天干净的版本，今天可能加了联网。更新后跑一次你常用的 Mission，确认没跑偏。

第 7 环：撤销。 不用了，就把配置文件里那一段 `filesystem` 删掉，重启客户端，确认它的工具列表里 `list_directory` 已经消失。这个 MCP 不连你个人账号，所以不用再去账号侧 Revoke；但如果是连了 GitHub / 邮箱的 Connector，撤销这一环必须回到对应账号的“第三方应用授权”页踢掉它——光在 Agent 里卸载，你账号上的授权可能还挂着。

走完这七环，你就给以后装所有扩展打了个样。记住：发现靠聚合站，决定靠源仓库；装之前先在沙箱跑一遍；不用了能干净撤掉。

本章 Mission

Mission 7-A: 给你正在用的 Agent 做一次“能力盘点”

列一张表：它现在用什么 Model？开了哪些 Tool？装了哪些 Skill / Plugin / MCP？连了哪些你的账号？每项标上“来源是官方还是社区”“最后一次更新是什么时候”“要了什么权限”。你会立刻看到哪些是想保留的、哪些是当时图热闹装上的、哪些授权早该撤了。

Mission 7-B: 走一遍完整的“安全安装”供应链

任选一条真实路径走通：要么在你自己的 skills 目录里装一个社区 Skill，要么配一个 filesystem 这类轻量 MCP。不急着装。先按 7.2 找到源仓库，按 7.3 读完 SKILL.md 和 12 问；有脚本就把脚本丢给 Agent 让它讲人话；再按 7.6 把密钥放进正确位置；最后才放进配置、在一个沙箱目录里跑一次，跑完按 7.9 完整卸载并确认无残留。整个过程就是给以后装所有扩展打样。

Mission 7-C: 做一次“权限大扫除”

打开你邮箱、GitHub、云盘、社交平台的第三方授权列表，把不认识的、超过三个月没用的授权全部撤销；同时检查你 Agent 里有没有开过 Computer Use、Terminal 这类高风险能力，没用就关掉。再给自己定一条规矩：从今往后，凡是 L4 动作，最后一下永远自己按。

本章所有涉及具体平台界面、配置文件路径、商店名称、MCP 注册中心地址的内容，都以 2026 年 9 月各平台官方文档现状为准；社区生态变化很快，安装前请回到官方文档和源仓库再核验一遍。本章 PowerShell 命令与目录为本机演示用真实环境，读者照做时请替换为自己的用户名与平台路径；凡标注“本环境无法替你完成”的交互步骤，请在你自己的 Agent 产品中实际执行并核验。

PART 8: 场景实验室

F16 场景实验室导航：7 个 LAB

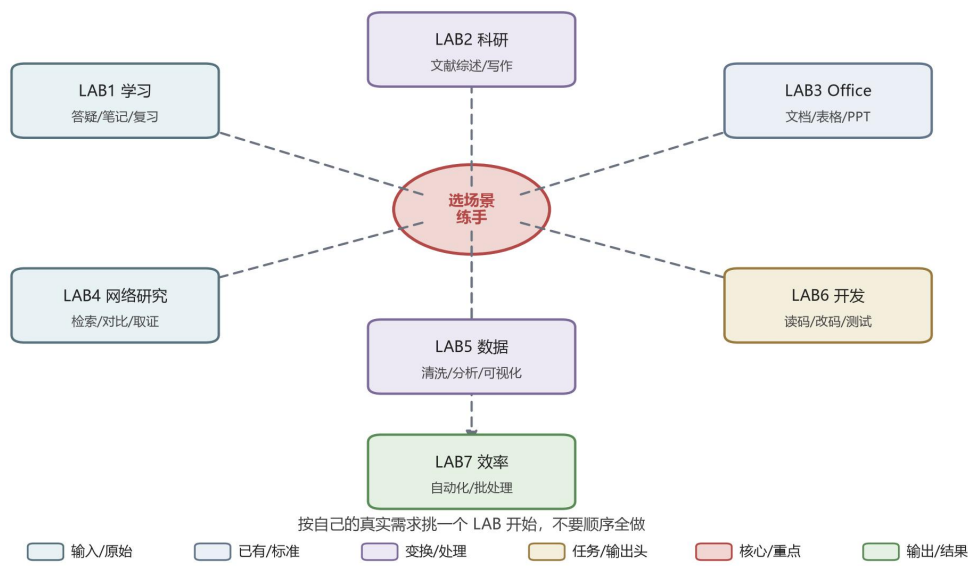


图 22 场景实验室导航

到这里，你已经学会怎么给 Agent 下任务、怎么管长任务、怎么判断它是不是真做对了、它会怎么失败，以及它由哪些零件组成。从这一部分开始，不再学“通用方法”，而是按专业挑一条路，把真事做一遍。

8.0 开工前先对齐三件事

F30 贯穿案例成长路线：四条线随章节一起升级



图 23 贯穿案例成长路线

第一，所有场景都走同一条主线。 不管你做学习、写综述、跑数据还是改软件，一个任务的生命周期都是：

定义任务 → 准备材料 → 执行 → 观察中间结果 → 验证 → 修复 → 交付 → 沉淀成模板

每个 LAB 都是这条主线在一种职业场景里的具体走法。你不用背八个新词，只要在每一步问自己：我现在在哪一步、这一步该由 Agent 干还是我干、怎么算过。

第二，每次复杂任务套 8 要素。 目标 / 上下文（喂给它的材料与背景） / 环境 / 权限 / 约束 / 过程 / 验收 / 交付。前面 Part 讲过，这里只提醒一句：验收必须写成能检查的句子，“做得好”不是验收。

第三，四条铁律落到本场景。 通用版是“写出来不等于做对、改了不等于修好、测了不等于全对、它说完成不等于真完成”。到了不同场景，换成各自的及格线：

版本	四句及格线
科研版	文献生成不等于文献真实；综述起草不等于结论成立；检索跑通不等于没有遗漏；它说“已核验引用”不等于真逐条打开过
数据版	代码写了不等于数算对了；动了数据不等于清洗完成；图生成了不等于关系成立；报告写“显著”不等于真显著
软件版	代码生成不等于功能完成；代码修改不等于 Bug 修复；测试通过不等于产品完全正确；Agent 声明完成不等于实际完成

后面每个 LAB 出现具体动作时，都可以回到这张表对照：我现在这条动作，对的是哪一句。

8.1 LAB 分级：不必做完七个才算学会

七个 LAB 对应七种典型职业场景，但不再平均用力。按三级挑：

级别	LAB	谁建议做
核心（所有人建议做）	A 学习、C Office、D 网络研究、G 日常效率	无论什么专业，这四条最高频
专业（按方向选）	B 科研、E 数据	走学术路线选 B，走实证/量化路线选 E，二选一或都做
进阶	F 软件开发	对软件本身好奇、或想用 Agent 改真实项目的人再上

一句话：做完一到两个 LAB 并沉淀成自己的模板，就算学会了；七个全看一遍而没一个走通，不算。难度沿用全书 L0 到 L7 刻度，每个 LAB 开头标出。

LAB A：学习 Agent（核心）

解决什么问题

手里有一门课：大纲、一叠 PPT、四周后考试。把“从材料到会考试”交给 Agent，你做判断人和验收人。难度 L2 到 L5。

旧做法为什么不对

让 Agent 把 PPT 总结一遍、画张思维导图，这不叫学习——这叫它替你读了一遍。真正决定考试成绩的，是你能不能在合上材料时把东西**想起来**，而不是你看着材料觉得“懂了”。所以这个 LAB 的闭环围绕“提取”而不是“阅读”设计。

学习闭环（核心动作链）

按顺序走，缺一环效果大打折扣：

1. **学习材料**：把大纲、PPT、教材、往年题放进一个文件夹。
2. **先预测，再学**：每章开始前，让 Agent 先考你“你觉得这章会讲什么、你已经会什么”，你先猜。猜错的地方，正是后面要重点盯的缺口——这叫 retrieval practice（提取练习）的前置：先暴露你以为会、其实不会的东西。
3. **Agent 提问，学生回答**：让它一次问一题，你答，它判。不许你先翻书。
4. **诊断错误类型**：把你答错的题归三类——概念记混了、根本没学过、还是理解反了。这一步叫 misconception diagnosis（错误概念诊断），三类错法的补救完全不同。
5. **针对性补充**：记混的做对比表，没学过的回原文重读，理解反的单独讲一遍反例。不要三类错都用“再看一遍”糊弄。
6. **延迟一天再测**：今天答错过的题，第二天换个问法再考一次。当天答对了不算数，隔一天还能答对才是真记住。
7. **间隔复习**：把你答错的知识点排成 1 天、3 天、7 天、14 天的复测表——这就是 spaced repetition（间隔重复），比考前突击省一半时间。

逐步实操：拿一门课真做一遍

假设你手上是“机器学习导论”，大纲、12 张 PPT、两周后测验。按上面七步，第一天这么走：

1. 建材料文件夹：course/ 下分 outline.md、slides/、old_exam.pdf，全部设成只读。

2. 先预测：对 Agent 说“第三章讲监督学习，我先猜它会讲线性回归和分类。你先别纠正，出三道题考我以为会的”。你先答，不许翻 PPT。

3. 它判卷：把答错的题记成 `errors.md`，每条标类型——记混了、没学过、还是理解反了。

4. 针对性补：记混的让它做对比表（线性回归 vs 逻辑回归）；没学过的让它回到 PPT 第几页重讲；理解反的让它单独给一个反例。

5. 延迟一天再测：第二天对它说“用昨天 `errors.md` 里的知识点，换个问法再考我”。

6. 排复测表：让它生成 `review_schedule.md`——错点 A 排第 1/3/7/14 天，错点 B 另排。

7. 每次出题必须带 PPT 出处：你验收时抽五个知识点，能在 PPT 上翻到原文对得上。
- 这七步跑通一次，就沉淀成你自己的“课程学习 Agent”指令模板，以后每门新课丢新材料进来照走。

完整学习闭环：7 天日程表

上面七步是动作链，真到日历上怎么排？以“机器学习导论、两周后测验”为例，把它落到连续 7 天，每天该做什么、产出什么、验收看什么，照表走：

天	上午（40 分钟）	下午/晚上（20 分钟）	当天产出	过线标准
第 1 天	第 3 章：先预测（猜这章讲什么）→ Agent 出题 3 道，合书先答 → 它判卷 → 把错题按“记混/没学过/理解反”三类记入 <code>errors.md</code>	针对性补：记混的做对比表，没学过的回 PPT 第几页，理解反的要一个反例	<code>errors.md</code> 有 ≥3 条带类型的错点	抽 5 个知识点，每条都能指出 PPT 出处
第 2 天	延迟再测：把昨天 <code>errors.md</code> 的题换个问法重考，不许翻书	答对的划掉，答错的升级进下一轮复习表	复测记录追加到 <code>errors.md</code>	昨天的错点今天换问法还能答对一半以上
第 3 天	第 4 章走同样闭环（预测→答→判→分类→补）	第 1 天错点的第 3 天间隔复测	第 4 章新错点 + 第 1 天错点复测结果	第 1 天错点这次答对率再升
第 4 天	第 5 章闭环	第 2 天错点第 3 天复测：把前三章做一次“盲画框架图”（合书画三章逻辑线）	一张盲画框架图	框架图缺的枝丫，就是还没记住的地方

天	上午（40 分钟）	下午/晚上（20 分钟）	当天产出	过线标准
第 5 天	第 6 章闭环	整理 review_schedule.md: 把所有错点排 1/3/7/14 天，打印或置顶	间隔复习表成型	每个错点都有下 一次复测日期
第 6 天	跨章混合考： Agent 从第 1 到 6 章随机抽 10 题， 合书做	错题再分类，回 PPT 补	混合测成绩单	10 题对 7 题以 上；错的都能定 位到章
第 7 天	模拟测验：按往 年题型出一套， 计时做	对答案，把仍错的点单 独列进“考前三天重点”清 单	模拟卷 + 重点清 单	模拟卷分数能预 告真实测验分数

这张表的三个纪律：一是**当天答对不算数**，所有错点必须跨天复测，第 2/3/4 天的下午都是在还前几天的账；二是**错题按类型补救**，记混、没学过、理解反三种错法各走各的路，不许都用“再看一遍”；三是**出题必带 PPT 出处**，验收时你随手抽五个点翻原文，对不上就打回重出。

跑满这 7 天，你会得到三个可复用的东西：`errors.md` 的模板（以后每门课换皮用）、`review_schedule.md` 的排表逻辑、以及一份“我这门课到底哪里虚”的诚实清单。两周后的第 8 到 14 天，就只做一件事——按 `review_schedule.md` 到点复测，直到所有错点连续两次答对。

学习闭环七步判断标准：每步输入、执行、看到什么、怎么算对

上面七步是动作链，真上手时你会卡在“我怎么知道这步做对了”。下面把每一步拆成四格：你给它什么、它做什么、你屏幕上应该看到什么、怎么判断这步过关了。照表走，不用靠感觉。

步	输入（你给它什么）	执行（它做什么）	你应看到什么	怎么判断这步对了
1 学习材料	大纲、PPT、往年题放进 course/	只读，建材料索引	它列出材料清单：几张 PPT、几份往年题、各多少页	清单数量和你文件夹里文件数一致；它没说“我来补充教材里没有的部分”
2 先预测	“第三章讲监督学习，我先猜线性回归和分类，你先别纠正，出题考我以为会的”	合书出题 3 道，考你预测过的点	三道题，不许翻 PPT，你先写答案	它出的题确实考你猜过的那两点，没跑去考你没提的内容；你答完它才判

步	输入（你给它什么）	执行（它做什么）	你应看到什么	怎么判断这步对了
3 提问判卷	你的答案	逐题判对错，错题入 errors.md	errors.md 新增记录，每题标了对错	errors.md 行数等于你答错的题数；它没把你蒙对的题也记成错
4 诊断错误类型	errors.md 里的错题	把错题归三类：记混/没学过/理解反	每条错题后面跟一个类型标签	每条错题都有且只有一个类型标签；你翻看这三类分得合理，没把“没学过”和“记混”混在一起
5 针对性补	带类型的错题	记混的做对比表、没学过的回 PPT 第几页、理解反的给反例	三种错法三种补救产物，不是一刀切“再看一遍”	记混的那条真的有一张对比表；没学过的那条真的指了 PPT 页码；理解反的那条真的给了一个反例
6 延迟一天再测	第二天把昨天 errors.md 的题换问法	换个说法重考同知识点	新题和昨天考的是同一个知识点，但措辞不同	你能认出“这题昨天考过”；今天答对了才算过，昨天答对今天换问法还对才算真记住
7 间隔复习排表	全部错题	生成 review_schedule.md ，每错点排 1/3/7/14 天	一张表，每行一个错点、四个复测日期	每个错点都有下一次复测日期；表上没有“待定”这种含糊日期

这张表的用法是：每做完一步，对着最后一列打勾。七步全打完勾，这一章的学习闭环才算走完。任何一步打不了勾，就回去重做那一步，不要往下走——学习和写综述一样，跳步的代价是后面全返工。

出题与验收纪律

- 它出的题重点考你第 2 步暴露的缺口，而不是 PPT 原文填空。
- 每个知识点后面必须标“来自第几张 PPT 第几页”，不许凭空补理论。
- 让它对照大纲列“大纲要求但 PPT 没讲清”的缺口清单，明确禁止它用常识补圆。
- 抽五个知识点翻 PPT 能对上；它出的题你不看答案先做，答案与原文一致。

最终沉淀

把上面七步写成你自己的指令模板：以后每门新课丢新材料进来，照走一遍。这就是课程学习 Agent。

卡住怎么办：学不进去和记不住

第一种卡点：**学不进去，坐 40 分钟还是在翻第一页**。这不是 Agent 不够聪明，是节奏错了。把“先预测”那步单独拎出来做——别直接让它讲第三章，先逼自己合书写三句“我猜这章讲什么”，写完再让它对答案。人对“自己猜错的部分”注意力天然集中，比被动听它讲强十倍。还卡就把 40 分钟切成两段：20 分钟只做预测+答题，不看材料；中间起来走 5 分钟；后 20 分钟只回 PPT 补刚才答错的地方。一段段硬啃不如一段段考。

第二种卡点：**当时答对了，第二天换个问法又错**。这不是你笨，是间隔还没到。当天答对不算数——这本来就是这个 LAB 的设计。翻 review_schedule.md，看这个错点排到了第几天复测，没到那天就先不管它；到了那天还错，就把它的间隔从 1/3/7/14 天改成 1/2/4/8 天，加密一轮。还不行就让 Agent 把这个知识点写成一个“反例题”（不是问“是什么”，而是给个错例让你挑哪错了），错例比正面题更能钉住概念。连续两次换问法都答对，才把它从复习表里划掉。

LAB B：科研 Agent（专业）

F29 8 要素示例：写一篇「AI Agent 教育应用」文献综述

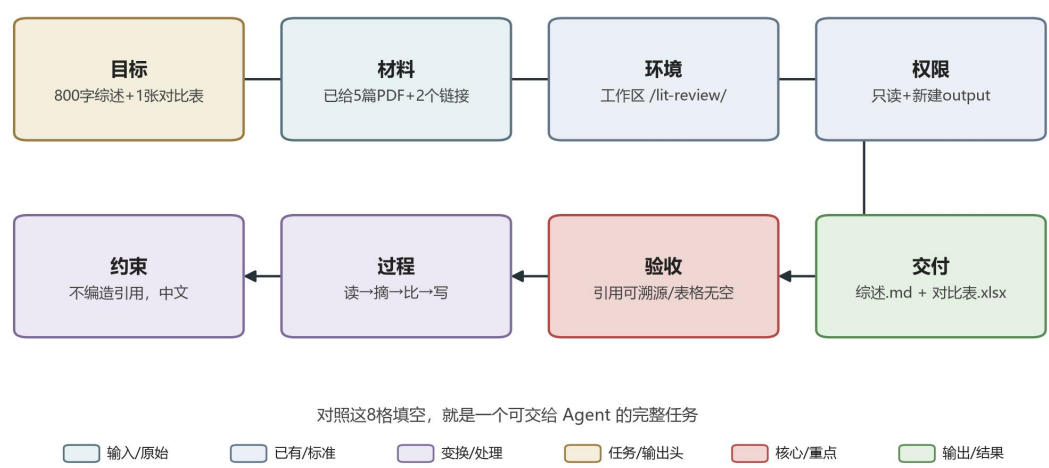
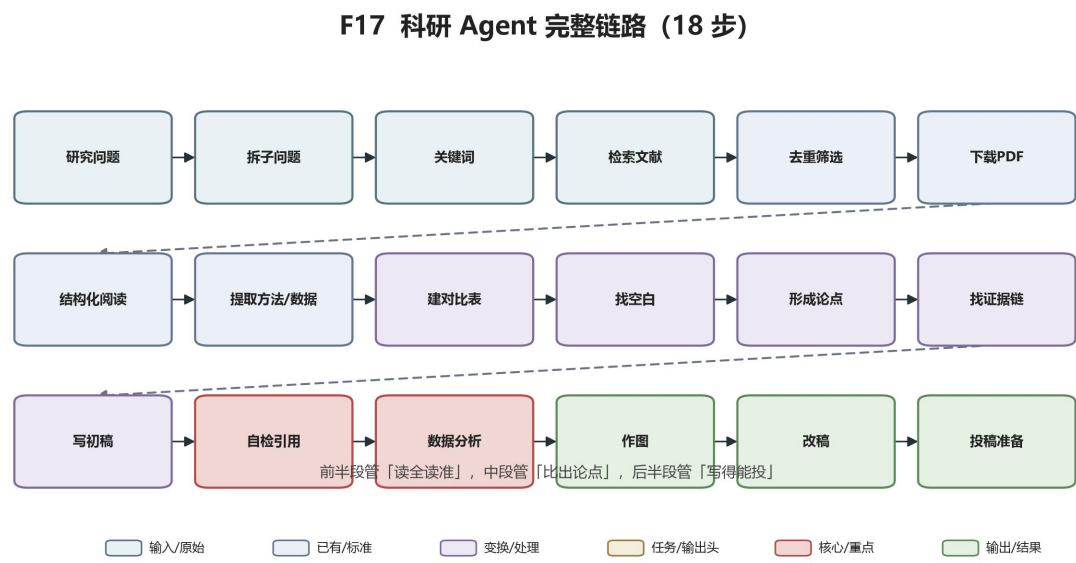


图 24 8 要素文献综述示例



这是全书最需要“一边用、一边守底线”的 LAB。科研的生命是真实：真实的文献、真实的数据、真实的方法。Agent 在这些环节的失效不是“做慢了”，而是“做假了”，而它做假时语气和做真时一模一样。

先分流：你做的是哪一类研究

不同研究类型，Agent 能帮的程度差很远。开工前先对号入座：

研究类型	长什么样	Agent 帮到哪	人必须守什么
普通课程综述	课程要求的小综述	读文献、整理、起草	选题与结论
Narrative review（叙述性综述）	围绕一个题目讲故事	找文献、列观点、起草	脉络判断、立场
Systematic review（系统综述）	按预设计划全面检索、合并证据	执行检索、去重、初筛、填表	protocol、纳入排除标准、最终判断
Scoping review（范围综述）	摸清一个领域“研究了什么、没研究什么”	检索、分类、图谱	分类框架
Bibliometric review（文献计量）	用文献数据做共被引、共现分析	跑数据、画图	解释、结论
Empirical study（实证研究）	有数据、有假设、有分析	跑统计、画图、改语言	研究设计、数据、因果解释

多数人是 narrative 或课程综述；想发顶刊、要做 systematic review 的，下面这段单独看。

系统综述的五条硬骨架

如果你走 systematic review，这五条一个不能少，缺了就不叫系统综述：

1. **Protocol（预注册方案）**：动笔前先把“研究问题、检索库、检索词、纳入排除标准、提取表”写成一份方案并锁定。后面不许边做边改词——改了就破坏可重复。
2. **Search strategy（检索策略）**：中文词、英文词、同义词、缩写、每个库的检索式，逐条写下来存档。
3. **Inclusion / exclusion criteria（纳入排除标准）**：什么类型的研究算、什么不算，写成可判定的句子，不靠感觉。
4. **Screening（筛选）**：按题录去重 → 按摘要初筛 → 按全文复筛，每一步记录“筛掉多少、为什么”。理想是双人独立筛，至少你自己要复核 Agent 的初筛理由。
5. **PRISMA**：最后画一张文献筛选流程图（识别多少、去重后多少、筛掉多少、最终纳入多少），这是系统综述的标准交付物。

任务责任矩阵：谁干，谁必须负责

环节	Agent 可执行	人必须负责
选题	反问、列举争议	研究价值与学术判断
检索	按 strategy 跑、拉题录、去重	检索策略与库的选择
初筛	按 criteria 分类并给理由	边界案例的最终判定
精读	逐篇填文献卡片	理解与批判
证据矩阵	叠表、归纳脉络	矩阵是否真的支撑结论
写作	起草、改语言	每一句论断的负责
引用	列条目、排格式	逐条打开原文核对
分析	跑统计、画图	方法选择、因果解释

一句话：Agent 负责“读得多、排得齐、写得顺”，人负责“选得对、判得准、担得起”。

六条底线（本 LAB 及格线）

1. **学术诚信**：Agent 是助手不是作者。按你所在机构和目标期刊的要求，如实声明它参与了哪些环节。别拿“AI 检测工具抓不抓得到”当判据——那工具误报漏报都被反复讨论过；唯一可靠的底线是如实披露，并对每句结论负责。
2. **引用真实性**：它极擅长造“看起来完全真实”的引用。综述定稿前，逐条打开原文，打不开、查不到、对不上的一律删。
3. **使用边界**：找文献、读文献、整理矩阵、查格式、改语言、写代码可以；替你提核心论点、替你下结论、把没跑过的分析写成自己跑过的，越线。

4. **数据来源**：任何数字回到原始出处（统计局、世行、论文附表、问卷原始记录），不用二手转述，不许它“估”。
5. **方法透明**：用了什么方法、为什么、有什么局限，要能写清楚；你讲不明白的方法，要么真搞懂，要么别用。
6. **可复现**：数据、清洗步骤、分析代码、随机种子都留档，另一个人能跑出你论文里同一组数。

怎么查你学校 / 期刊的 AI 政策

不靠猜，去官方页面查。先定两个主体：你在读（或要毕业）的学校，和你要投稿的期刊——前者管学位论文与课程作业，后者管投稿稿件，规定可能不同。用搜索引擎直接搜官方页：学校搜“[学校英文名] generative AI policy / 研究生 生成式人工智能 学术规范”，优先 .edu 域名的研究生院或学术诚信办公室页面；期刊搜“[期刊名] author guidelines AI”、“[出版社名] editorial policies AI”，优先出版社作者指南。可查入口示例（政策会更新，用前自行打开确认最新版）：Elsevier 作者指南的 AI 说明页、Springer Nature 的 Editorial policies: AI and AI-assisted technologies 页、各高校研究生院“学术规范 / Use of Generative AI”专栏（如 MIT、Stanford、Oxford、清华、北大）。查到后把链接、访问日期、关键句存进自己的披露清单，投稿前再核对一次。

最终沉淀

产物：题录表、证据矩阵、综述初稿、引用核验清单、独立审稿意见（换一个不带写作上下文的对话只当审稿人挑刺）、披露声明草稿。验收：矩阵每行对应一篇你真读过的文献；抽十条论断各有矩阵支撑；抽十条引用逐条打开原文对得上；分析附代码与原始数值可复现。

方向性深度：两周跑一个 narrative review 的节奏

如果你是第一次走，别一上来就奔 systematic review。多数人是 narrative 或课程综述，给一个可照抄的两周节奏：

- **第 1-2 天：分流+选题**。先回到上面分流表对号——你做的是哪一类。narrative review 不需要预注册 protocol，但要先和导师定“围绕哪条脉络讲故事”。
- **第 3-5 天：检索+拉题录**。让 Agent 按你给的关键词跑检索、导出题录、去重；但检索库和检索词是你定的，它只执行。
- **第 6-8 天：精读+填卡片**。逐篇读，每篇填一张文献卡片（研究问题/方法/结论/局限）。卡片它可以帮你排版，理解和批判是你的。

- **第 9-11 天：叠证据矩阵。** 把卡片叠成一张矩阵，归纳脉络。你要判断：这张矩阵真支撑你讲的故事吗？
- **第 12-13 天：起草+核验引用。** 它起草，你逐句负责；**每一条引用逐条打开原文**，打不开的一律删。
- **第 14 天：独立审稿+披露。** 新开一个不带写作上下文的对话，只让它当审稿人挑刺；再写一份披露声明。

每一步该 Agent 干还是你干，回到上面“任务责任矩阵”那一格——Agent 负责“读得多、排得齐、写得顺”，你负责“选得对、判得准、担得起”。

LAB C：Office 三件套（核心）

一条总原则：一个数字只有一个源头

Office 场景最大的事故是“三套数”——Excel 一个数、Word 一个数、PPT 一个数。根治办法只有一条：**Excel 是 source of truth（事实底座），Word 和 PPT 只能引用它，不许各自重算。** 所有 Office 工作都按这条拆成 C1 / C2 / C3。

风险前提：Office 文档格式很脆，Agent 改 Word 会乱样式、改 Excel 会覆盖公式、做 PPT 会标错数。所有操作在副本上做，原始材料锁只读。

C1 Excel：事实底座

链路：数据 → 质量 → 计算 → 表 → 图。

- 先报表的长相（行列数、每列类型），不许上来就画图。
- 写数据字典：每列什么意思、月还是年、税前还是税后；不确定的列列成问题问你，不替你猜。
- 质量清单：缺失、重复、明显录错（年龄 999、日期在未来）、单位是否统一，先列清单再动手。
- 在副本上清洗，每步说明理由。
- 计算结论每一个都要能点开单元格复核。

把“数据 → 质量 → 计算 → 表 → 图”这条链路落到鼠标和键盘上，以一份原始销售明细 `input_data.csv` 为例，一步一步怎么走：

第 1 步：打开 Excel，建一个工作簿。 打开 Excel，新建空白工作簿，按 **Ctrl+S** 立刻命名为 `sales_report.xlsx` 存到自己的项目文件夹。先别碰原始数据文件——原始 CSV 设成只读，后面所有操作都在这个新工作簿里做。

【截图占位：图 C1-1，空白工作簿刚建好，左上角已存为 `sales_report.xlsx`】

第2步：导入原始数据。 不要双击 CSV 直接让 Excel 打开（那样会在原文件上改）。正确做法：在新工作簿里点“数据”选项卡 → “自文本/CSV” → 选 `input_data.csv` → 预览确认列名和每列类型 → 点“加载”。这样原始 CSV 只读不动，数据进了工作簿第一张表。把这张表重命名为 明细。

【截图占位：图 C1-2，数据导入向导预览，能看到 15 行、各列名】

第3步：让 Agent 先“报长相”，不许上来就算。 对它说：“先告诉我这张表有多少行多少列、每列是什么类型、哪几列可能有缺失”。它应该先返回一份长相清单（行数 15、列数 6、日期列/金额列/文本列各几个），而不是直接出汇总。这一步卡住它，是为了防止它在你都不知道数据长什么样时就替你算完。

【截图占位：图 C1-3，Agent 返回的“表长相”文字说明】

第4步：写数据字典。 新建第二张表，命名为 数据字典，一列写列名、一列写含义。比如“金额：含税销售额，单位元；日期：成交月，YYYY-MM；区域：销售大区”。遇到它拿不准的列（比如某列写着“金额 A”），它应该列成问题问你，而不是替你猜。

【截图占位：图 C1-4，数据字典表，列名/含义两列】

第5步：列质量清单，在副本上清洗。 新建第三张表 质量检查，让它逐项报：缺失几行、重复几行、有没有越界（年龄 999、日期在未来）、单位是否统一。报完再动手洗——清洗在 明细 表的副本上做，原始导入那一份先留着。每删一行，让它在 质量检查 里记一句“删了第几行、为什么”。

【截图占位：图 C1-5，质量检查表：缺失/重复/越界/单位各一行计数】

第6步：计算汇总，每个数都能点开单元格。 新建 区域汇总 和 产品汇总 两张表，用数据透视表或 SUMIF 算。关键纪律：汇总表里的数必须是公式算出来的（点开单元格能看到 `=SUMIF(...)`），不许手输。算完立刻核一遍：区域汇总的总额、产品汇总的总额、明细表的总额，三个必须相等。本案例三个数都应是 62,818.00 元。

【截图占位：图 C1-6，区域汇总表，点中一个合计单元格，编辑栏露出 SUMIF 公式】

第7步：做表（格式化）。 给三张汇总表套统一样式：表头加底色、金额列用千分位加两位小数、列宽自动调整、冻结首行。这步交给 Agent 批量做，但你抽查两张确认它只动了格式、没改数字。

【截图占位：图 C1-7，格式化后的区域汇总表】

第8步：画图。 选中 产品汇总 那张表 → 插入柱状图，横轴产品、纵轴金额；再给 区域汇总 插一张饼图。每张图下面加一句话标题，写清这张图说明什么、单位是什么。不要让它一张图画三个系列互相打架。

【截图占位：图 C1-8，柱状图“各产品销售额（元）”】

这八步走完，你手里应该有一个 `sales_report.xlsx`，里面至少四张表（明细、数据字典、区域汇总、产品汇总）加两张图。验收就一句话：随便点一个汇总数字，能顺着公式追到明细行；三个总额对得上；原始 CSV 没被改过。

Excel 八步逐步验证方法：每步怎么确认没做错

上面八步是操作顺序，真做的时候每一步都可能翻车——数据导错了、公式被手输了、图画歪了。下面把八步每步的“怎么确认这步没做错”写成四格：你输入什么、它执行什么、你看到什么、怎么判断对错。照表逐步打勾，不要等八步全做完才回头查。

步	输入	执行	你应看到什么	怎么判断这步对了
1 建工作簿	原始 CSV 设只读，新建 <code>sales_report.xlsx</code> 存到项目文件夹	Excel 新建空白表	左上角文件名是 <code>sales_report.xlsx</code> ，原始 CSV 图标带小锁	点原始 CSV 右键看“属性”里只读已勾；新工作簿没存到临时目录
2 导入数据	数据→自文本/CSV→选 <code>input_data.csv</code>	15 行进第一张表，重命名 明细	表名是 明细，A1 到 F16 有数据，表头是列名	行数正好 15（不含表头）；点原始 CSV 看修改时间没变，说明没在原文件上改
3 报长相	“先告诉我行数、列数、每列类型、哪几列可能缺失”	它返回一份文字清单	清单写：15 行 6 列，日期列 1、金额列 1、文本列 4	它没直接开始画图或出汇总；它报的行数和你手动拉到底看到的一致
4 写数据字典	新建 数据字典 表	一列列名、一列含义	6 行，每行列名对得上 明细 表的表头	每个列名都有含义；遇到拿不准的列它列成问题问你，没替你猜
5 列质量清单	新建 质量检查 表	它逐项报缺失/重复/越界/单位	四行计数：缺失几行、重复几行、越界几行、单位统一与否	它报的数你抽查两三项能对上；清洗在 明细 副本上做，原始导入那份没动
6 计算汇总	新建 区域汇总 产品 汇总，用 SUMIF	汇总表出数，公式留在单元格里	点中合计单元格，编辑栏露出 <code>=SUMIF(...)</code>	区域汇总、产品汇总、明细底表三个总额都是 62,818.00 元；没有一个数是手输的

步	输入	执行	你应看到什么	怎么判断这步对了
7 格式化	让它批量套样式	表头加色、金额千分位、冻结首行	三张汇总表看起来整齐一致	你随机点两个数字，和第 6 步算出的原值没变；它只动格式没改数
8 画图	选 产品汇总 插柱状图，区域汇总 插饼图	两张图，各带一句话标题	柱状图横轴是产品名、纵轴是金额；饼图分四块对应四个区域	图上的数和表里的数对得上；一张图只有一个系列，没有三个系列叠在一起

这张表的关键纪律是第 6 步：三个总额必须相等。这是“一个数字一个源头”原则的体检表——只要三个数对不上，后面第 7 步格式化、第 8 步画图全是白做。发现对不上立刻停，回到第 5 步查清洗有没有删错行，不要往下走。

C2 Word：叙事表达

链路：数据 + 材料 → 结构 → 正文 → 表格 → 引用。

- 先盘点文件夹里有哪些 Word、各多长、大概讲什么，列清单给你确认。
- 先出结构（几章、每章讲什么），你点头再写正文。
- Word 里出现的每个数字，从 Excel 引用，不在 Word 里手输。
- 引用、参考文献按你学校/期刊格式排，排完抽查。

C3 PPT：演示表达

链路：audience（给谁看）→ message（你要他记住哪一句）→ story（怎么讲）→ slide structure（每页结构）→ visual（怎么可视化）→ review（挑错）。

- 先定“这一页只让观众带走一个结论”，再排版，不堆字。
- 每个数字同样回到 Excel，不现编。
- 页数控制在十页左右，每页一个结论。

批量与模板

“一百个 Word 标题统一改宋体小四”这类活交给 Agent 最划算，但必须先拿三个文件试跑、你检查无误再批量——批量出错时错得也很整齐。固定模板给它，要求“只填内容、不动样式”。最后让它当挑错人：错别字、格式不统一、数字前后不一致、引用缺漏。

真实案例

见 `examples/labs/lab_c_office/`：输入 `input_data.csv`（15 行销售明细），运行 `python run_lab_c.py` 产出 `sales_report.xlsx`（明细/区域汇总/产品汇总三表）、`sales_report.docx`、`talking_points.md`。复算证据：Excel 区域/产品合计与 Word 总额均为 **62,818.00 元**，三方一致——这就是“一个数字一个源头”的落地。逐项验收见该目录 `CHECKLIST.md`。

逐步操作链（照着 lab_c_office 复现一遍）

1. 事实底座：`input_data.csv` 是 15 行销售明细，**全程只读**，脚本不写回它。
2. 跑脚本：`cd examples\labs\lab_c_office`，再 `python run_lab_c.py`。
3. 看产出：它应生成 `sales_report.xlsx`（明细/区域汇总/产品汇总三张表）、`sales_report.docx`、`talking_points.md`，运行日志落在 `lab_c.log`。
4. 验“一个数字一个源头”：打开 Excel，区域汇总与产品汇总都应是 **62,818.00 元**；打开 Word，报告里的总额也必须是 **62,818.00 元**——三方对得上才算过，任何一处手输了别的数都要打回。
5. 批量前先试三个：遇到“一百个 Word 统一改宋体小四”这类活，先拿三个文件试跑，你检查无误再批量——批量出错时错得也很整齐。
6. 让它当挑错人：查错别字、格式不统一、数字前后不一致、引用缺漏。
7. 逐项打勾：`CHECKLIST.md` 共 8 项验收（含“原始输入未被修改”），全过才算完。

卡住怎么办：公式报错和数据对不上

第一种卡点：**Excel 公式报错，#DIV/0!、#N/A、#REF! 满屏**。先别让它“修公式”，先定位是哪一类错。**#DIV/0!** 是分母为零——某行金额被空字符串或 0 除了，让它在公式外套一层 `=IFERROR(原公式,0)` 或者先把空行筛掉；**#N/A** 是 `VLOOKUP` 没匹配上——多半是两边文本带空格、大小写不一致，让它先跑一遍 `=TRIM()` 清洗再匹配；**#REF!** 是它删了一列但公式还指着那列——这是最危险的，说明它改表结构时没同步改公式，立刻回到上一个备份，让它先把所有公式列出来再动表。修完每改一个公式，点一下编辑栏看它真的是公式而不是手输的数字——手输的数字下次一改源数据就对不上。

第二种卡点：**三个总额对不上，差了几十块**。这是“一个数字一个源头”原则被破坏了。别让它“凑一凑”，按三步查：先在 明细 表最下面用 `=SUM(金额列)` 拿到底表总额；再分别看 区域汇总 和 产品汇总 的合计行，看哪个和底表不一样；对不上的那个汇总表，点开合计单元格看公式——是不是 `=SUMIF` 的范围没选全（漏了最后两行）、是不是把“退货”行也加进去了、是不

是有一行被它手输了个数字而不是公式。差几十块这种小错，十有八九是某一行被手输覆盖了。找到那一行改回公式，三个总额立刻对齐。

LAB D：网络研究 Agent（核心）

解决什么问题

写一份调查报告：一家公司、一项政策、一个行业、一个竞品。靠收藏夹攒链接，既慢又容易只看到愿意看的那几篇。最终项目：每条关键结论后面都挂着真实可打开的来源。难度 L3。

搜索策略八步

别上来就搜。按顺序走：

1. **问题拆解**：把“这家公司怎么样”拆成 5 到 8 个可回答的小问题（做什么、谁开的、规模、最近动作、负面/诉讼、行业位置）。
2. **查询式设计**：每个小问题设计多组查询——官方名和俗称、中文名和英文名、同一事实的不同说法。
3. **来源层级**：先排好哪类来源可信——官方原始出处（政府网站、公司公告、法院文书、论文原文）最高；行业报告次之；媒体报道当线索；自媒体和营销稿只当线索不当证据。
4. **时间范围**：政策、价格、数据类信息必须标“这是哪年的数据”，过期信息单独标注。
5. **回到原始来源**：媒体转述的数字，追到公告/原文再用。
6. **主动寻找反证**——这一步最关键，单独说。
7. **交叉验证**：同一事实至少两个独立来源对一遍；互相转载的不算两个来源。
8. **证据表**：从第一天建一张来源表（标题 / 网址 / 发布日期 / 访问日期 / 支撑了哪个结论），报告里每个关键结论都能在表里找到行。

八步是骨架，每步具体怎么动手？拿“调查某竞品公司 X”为例，逐步落到键盘上：

第 1 步 问题拆解——怎么拆。 别用“X 公司怎么样”这种大问题搜。拿一张纸，先逼自己写出“我要给老板交一份报告，他会问哪几件事”。一般拆成这几格：主营业务是什么、创始人与股权背景、营收/规模量级、最近一年的重大动作（融资/新品/裁员）、负面与诉讼、在行业里排第几。写到 `questions.md` 里，每条后面留空待填。拆不好的标志：搜出来全是广告稿、或者怎么都搜不到——说明你问的是个没法回答的模糊问题，回去重拆。

第 2 步 查询式设计——怎么写查询式。 每个小问题准备 2 到 3 组查询式，覆盖四种说法：官方全称（“X 科技股份有限公司”）、俗称/简称（“X 科技”）、英文名（“X Tech”）、同一事实的不同问法（“X 营收”和“X 年收入”、“X 融资”和“X 投资方”）。查询式里善用引号锁

短语、用减号排除垃圾（X 公司 -招聘 -博客）。设计完先自己搜一遍试试，第一页全是招聘网站和软文的，说明这组词太泛，加限定词或换角度。

第 3 步 排来源层级——哪类来源先信。 建 `sources.md` 时就给每条来源打个等级标：一手（政府/监管网站、公司自己的公告与招股书、法院文书、论文原文）= 直接采信；二手（知名行业咨询机构报告、主流财经媒体）= 可作线索但要追一手；三手（自媒体、公众号、营销稿）= 只当“别人在说什么”，永远不当证据。写报告时，关键结论只挂一手和二手，三手内容最多写“坊间传闻”。

第 4 步 标时间——怎么判断过没过期。 每条数字后面必须手写它的年份和发布日期。营收数据超过两年、政策文件已被新文件取代、价格/市占率这类快变量，一律标注“截至 YYYY-MM”。发现一条“X 公司有 500 人”的说法，先看它哪年发的，三年前的数今天可能翻倍也可能裁完了。

第 5 步 追原始来源——怎么追。 媒体稿里写“据公告，X 去年营收 10 亿”，你别直接用这 10 亿。点它引的链接，找不到链接就自己去公司官网“投资者关系/公告”栏或证监会披露平台，搜“X 公司 年度报告”，翻到财报那一页，对一遍数字对不对。媒体漏小数点、把季度当全年的事天天发生，只有原始出处能救你。

第 6 步 主动找反证——怎么搜相反的话。 这一步是强制动作，不是可选项。专门另起一组查询，把你想问的每个正面结论配一个反面词：X 公司 风险、X 公司 诉讼、X 公司 裁员、X 公司 处罚、X 差评、X 倒闭、X 欠债。在裁判文书网、信用中国、企业信用信息公示系统这些官方负面库里单独搜一遍公司名。搜到反证就如实写进报告“风险”一节；一条都没搜到，也必须白纸黑字写一句“已按上述反面查询式检索，未发现公开负面记录”——这一句话本身就是证据，证明你真的找过。

第 7 步 交叉验证——什么才算两个来源。 同一条事实，找两个独立来源对一遍。判断独立的方法：这两个来源是不是都转自同一个公告？A 媒体写“据 B 报道”，B 又写“据 C 公告”——那其实只有 C 公告一个来源。真正两个来源是：公司年报写的数 + 第三方行业报告独立测出来的数，这种才算互证。互相转载、洗稿、矩阵号通发的，全算一个来源。

第 8 步 建证据表——怎么记。 从第一天就建 `sources.md`，列五栏：标题 / 网址 / 发布日期 / 访问日期 / 支撑了哪条结论。每读完一条来源立刻登记，别攒到最后回忆。写到报告正文时，每个关键结论后面挂一个证据表行号；写完倒查一遍：随机抽五条结论，五条都能在表里点开对应行，才算合格。表里查不到行的结论，一律删或改写。

搜索八步逐步结果检查：每步怎么知道搜对了

上面八步是骨架，真搜的时候每一步都可能跑偏——问题拆太泛、查询式太软、来源太水、时间没标。下面把每一步的“怎么知道这步搜对了”写成四格：你输入什么、它执行什么、

你看到什么、怎么判断这步过关了。照表逐步检查，不要搜完八步才回头发现第一步就拆错了。

步	输入	执行	你应看到什么	怎么判断这步对了
1 问题拆解	“这家公司怎么样”	它拆成 5 到 8 个可回答小问题，写进 questions.md	一张问题清单：做什么/谁开的/规模/最近动作/负面/行业位置	每个问题都能直接回答；没有“这家公司怎么样”这种没法答的大题；清单条数在 5 到 8 之间
2 查询式设计	每个小问题配 2 到 3 组查询	它列出官方名/俗称/英文名/不同问法	一张查询式清单，带引号锁短语、减号排垃圾	第一页搜出来不是全是招聘网站和软文；每组查询都有四种说法覆盖；排除词写进去了
3 排来源层级	搜回来的来源	它给每条打一手/二手/三手标	sources.md 里每条来源后面跟着一个等级标	关键结论只挂了一手和二手；三手内容只写了“坊间传闻”，没当证据用
4 标时间	每条数字	它在数字后面手写年份和发布日期	每条数字后面跟着“截至 YYYY-MM”	没有一条数字是光溜溜的；超过两年的数据单独标了“可能过时”
5 追原始来源	媒体转述的数字	它点引的链接或去官网披露平台对一遍	你看到原始出处（公告/年报/财报）上的数	媒体写的数和原始出处对得上；对不上的以原始出处为准，媒体那条作废
6 找反证	公司名+风险/诉讼/处罚等反面词	它专门搜一组反面查询，还去官方负面库查	报告里有一节“风险”，要么列了搜到的负面，要么写了“已检索未发现”	反面查询真的跑过了；一条都没搜到也白纸黑字写了那句“已检索”；不是跳过这步
7 交叉验证	同一条事实的两个来源	它找两个独立来源对一遍	两个来源不是互相转载的；A 不是转自 B，B 不是转自 C	两个来源是真独立的（年报+第三方报告）；互相转载的不算两个，只算一个

步	输入	执行	你应看到什么	怎么判断这步对了
8 建证据表	每读完一条来源立刻登记	<code>sources.md</code> 五栏：标题/网址/日期/访问日期/支撑哪条结论	一张表，行数等于你读过的来源数	随机抽五条结论，五条都能在表里点开对应行；表里查不到行的结论已经删或改写

这张表的用法和学习 LAB 那张一样：每步打完勾再走下一步。最容易跳的是第 6 步找反证——人都有收集正面证据的本能，Agent 也有。所以这一步是强制动作，不是可选项：哪怕搜出来全是好话，也必须把“已检索反面来源，未发现”这句话写进报告，证明你真的找过，而不是默认没有。

主动找反证

人和 Agent 都有同一个毛病：搜“这个公司好不好”，会顺手只收集说好的。所以流程里强制加一步：每下一个重要结论之前，让 Agent 专门搜一组相反的查询——“公司名 + 风险 / 诉讼 / 争议 / 倒闭 / 差评 / 被处罚”，看有没有打脸的证据。找到反证就写进报告，找不到也要写一句“已检索反面来源，未发现”。报告里必须有一节“本报告未能核实的部分”，查不到就老实写“未找到可靠来源”，不许用流畅的话把空白填满。

验收

随机抽五条结论，每条都能点开来源且来源确实支持这句话；至少三条标注访问日期并能看出数据年份；有专门一节写未核实部分。

逐步实操：拿一家竞品公司真做一遍

目标是写一份“某竞品公司”的调查报告。把上面八步落成动作：

1. 拆问题：把“这家公司怎么样”拆成 6 个小问题（做什么/谁开的/规模/最近动作/负面诉讼/行业位置），先写进 `questions.md`。
2. 设计查询：每个小问题配 2 到 3 组查询——官方名+俗称、中文名+英文名、同一事实的不同说法。
3. 排来源层级：先定好哪类可信——官方原始出处最高，行业报告次之，媒体当线索，自媒体只当线索不当证据。
4. 标时间：每条数字都标“这是哪年的数据”，过期信息单独标注。
5. 追原始：媒体转述的数字，追到公告/原文再用，二手转述不落结论。

6. 找反证：专门搜一组“公司名+风险/诉讼/争议/差评/被处罚”，找到写进报告，找不到也写一句“已检索反面来源，未发现”。
7. 交叉验证：同一事实至少两个独立来源对一遍；互相转载的不算两个来源。
8. 建证据表：从第一天就建 `sources.md`（标题/网址/发布日期/访问日期/支撑了哪条结论），报告里每条关键结论都能在表里找到一行。

卡住怎么办：找不到来源和搜出来全是软文

第一种卡点：**关键问题怎么都搜不到一手来源**。比如你要查“这家公司去年的研发投入”，官网只写“持续加大研发投入”，没有具体数字。这时候别让它“根据公开信息估算”——估出来的数就是编的。换三条路：一是去证监会/股转系统/交易所的披露平台，搜这家公司的年报、招股书、融资说明书，研发投入在财报附注里一定有；二是找第三方行业报告（券商研报、咨询机构），哪怕是二手的，至少它引用了原始出处，你顺着它的引用再追一手；三是直接在报告里写“截至 YYYY-MM，公开渠道未披露具体研发投入数字，仅知其为 X 行业头部公司”——查不到就老实写查不到，这比编一个数强一百倍。

第二种卡点：**搜出来全是软文和招聘稿，没有真东西**。说明你的查询式太泛。三个动作：一是加排除词，公司名 -招聘 -博客 -软文 -推广 -加盟，把营销内容先滤掉；二是换角度搜，别搜“公司名 怎么样”，改搜“公司名 年报”“公司名 处罚”“公司名 融资”“公司名 专利”——这些词出来的是硬材料；三是直接去官方负面库（信用中国、裁判文书网、企业信用信息公示系统）搜公司全称，那里出来的东西不会是软文。如果按上面三招搜完还是只有营销稿，说明这家公司要么太小、要么太新、要么太低调——这本身就是结论，写进报告“公开信息有限，多为品牌稿，未发现独立第三方深度报道”。

LAB E：数据分析 Agent（专业）

F18 数据分析 Agent 流程



关键：每步都保留脚本和中间文件，结论必须能被重新跑出来

输入/原始
 已有/标准
 变换/处理
 任务/输出头
 核心/重点
 输出/结果

图 26 数据分析流程

你有一份数据表，想知道里面说明什么。这个 LAB 分两层：E1 适合普通文科生做描述性分析，E2 适合硕博做研究型分析。先看自己落在哪层，别硬上 E2。

E1 基础数据分析（普通文科生）

链路：读数据 → 数据字典 → 质量 → 清洗 → 描述统计 → 可视化 → 报告。

- 先报表的长相，不分析。
- 数据字典：每列什么意思，不确定的问你，不猜。
- 质量清单：缺失、重复、越界、单位，先列后洗，在副本上做，原始数据不动。
- 描述统计 + 可视化：每张图配一句“这张图想说明什么、坐标轴是什么、范围多少”。
- 报告：每个结论后面跟“用了哪些数据、在什么范围内成立”，范围外不说满。

E2 研究型分析（硕博）

在 E1 基础上加一段真正的推断与建模：

- 先写假设：你要检验什么？同一个数据集，回答不同问题用不同方法。
- 方法选择：为什么用这个方法、它的假设是什么，讲清楚。
- 推断统计与模型：凡是报“显著”，必须同时给代码和原始数值、p 值、样本量，不许只给一张图说显著。
- Diagnostics（诊断）：模型在样本上表现如何，残差、假设是否满足。

- **Robustness（稳健性）**：换一种做法、换一个样本，结论变不变。一变就变的“规律”不能当发现写。
- **Sensitivity（敏感性）**：关键参数/阈值动一动，结论稳不稳。
- **Reproducibility（可复现）**：清洗与分析代码、随机种子、数据说明一并留档，别人能跑出同一组数。

四条铁规矩（本 LAB 及格线）

一，不看图“猜数据”——差多少、p 值、样本量，必须有数。二，不编造统计结果——每个均值、比例、相关系数都要贴出可复算代码。三，代码可复现——今天跑和三个月后跑结果一样。四，指标定义明确——“活跃用户”“高收入”第一次出现就写清怎么算、阈值多少，全文不变含义。

完成标准：E1 交什么，E2 交什么

- **E1 基础版（普通文科生）交六样**：数据字典、质量清单（缺失/重复/越界/单位各多少）、清洗后的数据、描述统计、每张图配一句“这图说明什么/坐标轴/范围”、一段“结论在什么范围内成立”的话。以 lab_e_data/ 为证：dirty_data.csv 22 行里剔成 cleaned_data.csv 10 行，收入均值恒为 **10,180.05 元**，年龄均值 31.1——这组数你今天跑和三个月后跑必须一样。
- **E2 研究版（硕博）在 E1 之上多交三样**：一段写清的假设、一段“为什么用这个方法+它的假设是什么”、以及凡是报“显著”就必须同时给代码、原始数值、p 值、样本量。再加三样留档：模型诊断（残差、假设是否满足）、稳健性（换个做法结论变不变）、可复现包（清洗与分析代码+随机种子+数据说明）。一变就变的“规律”不能当发现写。

真实案例

见 examples/labs/lab_e_data/：自建脏数据 dirty_data.csv（22 行，含缺失/越界/格式不一致三类），运行 python run_lab_e.py 产出 cleaned_data.csv、quality_report.md（数据字典+质量检查+剔除留痕）、analysis_report.md、income_by_city.png。复算证据：清洗后恒为 10 行，月收入均值恒为 **10,180.05 元**。逐项验收见 CHECKLIST.md。

LAB F：不会编程也能开发软件（进阶）

F19 软件开发 Agent 流程



图 27 软件开发 Agent 全流程

这是进阶 LAB，只建议对软件本身好奇、或真要拿 Agent 改一个真实项目的人做。教学目标说死：**借助 Agent 完成开发，不是从零教你编程。**你练的是“甲方加验收员”那套动作——找项目、读说明、跑起来、提需求、验收、存版本；代码由 Agent 写，你负责看结果、判对错。某行代码你看不懂很正常，本 LAB 不要求你看懂，只要求你确认它跑出来的效果对不对。

主线：跟着一个真实项目走，做的时候再解释术语

不再一个名词一个名词地讲课。教程已经把一个真实开源项目 `vendor` 进了 `examples/labs/labf_project/repo/`——就 是 `[tartley/colorama](https://github.com/tartley/colorama)`（BSD-3，纯标准库零依赖）。你跟着下面这条动作链走，术语在你真正遇到它时我才解释。先记住这一屏字，每一步都可能回头看：

代码生成 ≠ 功能完成
代码修改 ≠ Bug 修复
测试通过 ≠ 产品完全正确
Agent 声明完成 ≠ 实际完成

动作链（边做边懂词）：

1. **Clone（克隆）到本地**：把 GitHub 上的项目完整下到你电脑，连版本历史一起拿下来（后面要靠它回退）。这一步你只要让 Agent 执行、自己不用敲命令。
2. **打开 README**：项目根目录那个说明文件，是它的使用说明书。让 Agent 读给你听：解决什么问题、要什么前提、怎么启动。README 写得烂、根本跑不起来的项目趁早换。

3. **跑一次，看它报错还是跑通：**在自己屏幕上亲眼看到窗口、网页或“运行成功”。Agent 说“应该能跑了”不算，你看到才算。跑不起来时，最常见的麻烦是 **Dependency**（依赖）——这个软件需要的“其他零件”没装好；把报错原样贴给 Agent 让它解决。
4. **找到 demo01 跑起来：**进入 `demos/`，先跑最简单的 `demo01.py`。这一步在本机已验证： `run_demo.log` 里颜色网格已正常输出。
5. **让 Agent 画一张“文件地图”：**你看不懂代码没关系，让它当翻译——主流程在哪、哪个文件控制那个按钮、哪几个文件以后最常改。这张图存下来，后面所有修改靠它定位。
6. **挑一个最小改动：**改文字、换颜色、加一句欢迎语都行，越小越好。别直接说“帮我改一下”，先让它把需求列成 **Task List**（碰哪个文件、改哪行、怎么验证），你觉得靠谱再动手。
7. **立刻在你自己的窗口看效果：**改一个东西就跑一次看一眼，不对就改回去。不要一口气改十处再跑——坏了你不知道是哪处弄坏的。
8. **亲手 Test（测试）：**你像真实用户一样点一点、试几种情况。它自己说“我测过了”不算，你点过了才算。本机这个改动对应 `test_smoke.py`，运行输出 **SMOKE OK**。
9. **回归测旧功能（Regression Test）：**改了新地方，把以前能用的老地方再试一遍——修好了水龙头，顺便看看马桶还冲不冲。Agent 很容易“修一个坏三个”。
10. **Git 存档（Commit）：**Git 是游戏存档系统，每完成一个能正常工作的小改动就存一次，写一句“这次改了什么”（“把 `demo01` 末尾加一行 `banner`”才是合格说明，“改了点东西”不是）。本机这次改动已本地 `commit` 为 `2874e5f`。

走到这里你就完整经历了“找到 → 跑起来 → 改一点 → 测一测 → 存版本”的本地闭环。再往后的词——**Branch**（平行修改线，放心折腾的安全区）、**Push**（上传到 **GitHub**，对外动作，先确认且不含密钥）、**Pull Request**（改完别人项目后礼貌申请合并）、**Deployment**（部署到别人能访问的地方）、**Release**（正式宣布版本发布）——等你真的要对外发时再查，现在不必背。

十步操作链的真实证据（每步对应一个文件）

这十步不是嘴上说说， `examples/labs/labf_project/` 里每一步都留下了实物证据，你照表核对：

步	动作	本项目里的真实证据
1 Clone	浅克隆上游	<code> clone.log</code> ；克隆时上游 HEAD <code> 841634e</code>
2 读 README	了解项目	<code> repo/README.rst</code> （BSD-3，纯标准库零依赖）

步	动作	本项目里的真实证据
3 跑一次	看报错还是跑通	run_demo.log（demo01 颜色网格已输出）
4 找 demo01	跑最简单的例子	repo/demos/demo01.py
5 文件地图	让 Agent 当翻译	你让它画，存进项目笔记
6 最小改动	改文字/加一行	change.diff（真实改动：demo01 末尾加 4 行绿色 banner）
7 看效果	改完就跑一眼	run_demo.log 末尾出现那行 banner
8 亲手测	你像用户一样点	repo/test_smoke.py，跑出来是 SMOKE OK
9 回归测旧功能	改新的也试老的	重跑 demo01.py，颜色网格照旧
10 Git 存档	存一个能工作的版本	本地 commit 2874e5f（见 commit.log/git_log.txt）

其中第 6 步的真实改动（change.diff）长这样——你要学的就是“改动要小、要能一句话说清”：

```
print()
+# [LAB F 教学改动] 追加一行 banner，验证“改文案 -> 跑通 -> 提交”闭环
+print(Fore.GREEN + Back.RESET + Style.BRIGHT
+      + "LAB F smoke: colorama 颜色网格已渲染完毕。”
+      + Style.RESET_ALL)
```

第 8 步那个 test_smoke.py 也只有 46 行，核心就一句断言：print(“SMOKE OK: colorama 核心 API 全部通过”)——这就是“你亲手测过”的实物，不是它自己说“我测过了”。

权限边界（如实标注）

本机已落地的闭环全部在本地完成：git clone --depth 1 浅克隆上游（clone 时上游 HEAD 841634e），读 README，跑 demo01，在 demos/demo01.py 末尾加一行绿色 banner（diff 存 change.diff），新增 test_smoke.py 跑通，本地 commit 2874e5f。没有执行`git push`，没有向上游提 PR——第三方仓库无写权限，教学性改动只留在本地。真想贡献上游，正确姿势是 fork 到自己账号 → 在 fork 上 push → 再去网页发 PR。逐项证据见 examples/labs/labf_project/CHECKLIST.md。

心态收尾

会用 Agent 改一个按钮，不等于你会做软件；但它比“看了十本编程教程却从没跑通一个项目”的人离真实开发更近。真正的门槛从来不是写代码，而是：找得到、跑得起来、改得动、测得出来、坏了能退、做好了敢发。

LAB G：个人工作台（核心）

收缩为一件事：统一 Inbox

前面几个 LAB 围着“一个具体任务”，这个 LAB 只围着你每天的杂事转，但只做一件事——把 **Email / Calendar / Tasks / Files / Notes** 五个入口整理成一个统一 **Inbox**。具体自动化（定时扫、自动发、自动归档）留给 PART 9，这里只解决“乱”。

怎么做

- **Email**: 让它读收件箱，按“今天要回 / 这周处理 / 只是通知 / 垃圾”分四类，给需要回的起草草稿。起草可以，**发出去不行**（对外动作，必须你点）。
- **Calendar / Tasks**: 把散落在备忘录、聊天、邮件里的事抽成一张待办清单，按时间和紧急度排好，每周一早上给你一份“这周做什么”。
- **Files**: 让它把下载文件夹、桌面、截图文件夹按规则归一次档——这是 PART 1 那个 Mission 的长期版。
- **Notes**: 把平时攒的笔记、读过的东西先堆进同一个 Inbox，定期让它去重、补标签。

成功标准

你能说出三件它每周替你省下的、具体发生的事；它起草的邮件和纪要你改两笔就能用，而不是重写；凡“发出去、存进去、公开”的动作，永远停在你这里等确认。

逐步实操：第一天就这么干

1. **Email**: 让它读收件箱，按“今天要回/这周处理/只是通知/垃圾”分四类，给需要回的起草草稿。**起草可以，发出去不行**——对外动作必须你点。
2. **Calendar/Tasks**: 把散在备忘录、聊天、邮件里的事抽成一张待办清单，按时间和紧急度排好，每周一早上给你一份“这周做什么”。
3. **Files**: 让它把下载文件夹、桌面、截图文件夹按规则归一次档——这是 PART 1 那个 Mission 的长期版。
4. **Notes**: 把平时攒的笔记、读过的东西先堆进同一个 Inbox，定期让它去重、补标签。

5. 跑一周后验收：对照成功标准——能说出三件它替你省下的事；草稿改两笔就能用；没有任何一封邮件是它替你发出去的。

Inbox 整理第一周：每日操作清单

第一天那五步只是开头，真正把乱账理顺要一周。下面是第一周每天的具体动作，每天 15 到 20 分钟，按表走：

天	当日动作（15-20 分钟）	当日必做	验收
第 1 天	建统一 Inbox：新建一个文件夹叫 Inbox/ ，下面分 邮件/ 日程/ 文件/ 笔记/ 四个子目录。先不整理旧物，只建架子	架子建好，五个入口各归一位	五个乱入口都能在 Inbox/ 下找到对应子目录
第 2 天	Email 第一轮分类：让它读收件箱未处理邮件，按“今天要回/这周处理/只是通知/垃圾”分四类列给你看。你点头后，给“今天要回”的起草草稿	草稿起草好，但一封都不发	有一份分类清单；待发草稿都躺在草稿箱，没一封发出去
第 3 天	Files 第一轮归档：让它把下载文件夹和桌面按“按类型+年月”归一次档（图片/文档/安装包/其他，再按月份分）	桌面和下载清空，移到 Inbox/文件/	桌面只剩回收站和此电脑；下载文件夹没有散文件
第 4 天	Calendar/Tasks 抽清单：让它把散在聊天、备忘录、邮件里的事抽成一张待办清单，按“本周必做/本周可做/以后再说”分三档	抽出一份清单，不超过 15 条	每一条都能说出“谁、什么时候、要我做什么”
第 5 天	Notes 归集：把散落的笔记、微信收藏、读书划线全丢进 Inbox/笔记/ ，让它去重、补标签	一次性归集，不要求当天读	笔记不再散在五个 App 里，都在一个地方

天	当日动作（15-20 分钟）	当日必做	验收
第 6 天	邮件第二轮：处理第 2 天起草的草稿——你逐封看，“发”的点发送，“重写”的打回，“不回”的归档。顺便把这五天新到的邮件跑一遍分类	草稿清零	草稿箱空；没有它替你发出的信
第 7 天	周复盘：让它生成本周小结——你这五天收了多少邮件、回了多少、归了多少文件、清了多少待办；哪些动作你希望它下周自动做	写一份下周的自动化候选清单	你能说出三件它这周替你省下的具体事

第一周的纪律有三条：一是**只收不删**——归档是移动到 Inbox/ 下，不是删掉旧物，理乱账第一周不许下删除键；二是**对外动作一律停手**——起草邮件、整理日程可以自动，“发送、删除、对外提交”这些动作第一周全由你亲手点，先把自动段和手动段的边界看清了，下周才好谈自动化；三是**每天只做当天那一格**——不要第一天就想把半年的文件全理完，那是第二周以后的事。第一周的唯一目标，是让“东西都有地方放”这件事先成立。

Inbox 第一周每日完成标准：每天到底做到什么才算完

上面那张七天日程表是动作清单，但“做了”和“做完了”是两回事——你可能第三天就把文件归了一半却以为完了。下面把每天补一格“怎么判断今天真完了”，对着打勾再睡，不要凭感觉收工。

天	当日动作	你做了什么	你应看到什么	怎么判断今天真完了
第 1 天	建统一 Inbox 架子	新建 Inbox/ 下四个子目录	四个空文件夹：邮件/日程/文件/笔记	架子建好了，你能说出五个乱入口分别对应哪个子目录；今天没动任何旧物
第 2 天	Email 第一轮分类	它读未处理邮件分四类，你点头	一份分类清单，“今天要回”那几封有了草稿	草稿全躺在草稿箱，一封都没发出去；分类清单你扫过一遍，没有它自作主张发的信

天	当日动作	你做了什么	你应看到什么	怎么判断今天真完了
第 3 天	Files 第一轮归档	它把桌面和下载按类型+年月归档	桌面只剩回收站和此电脑；下载夹没有散文件	你走到电脑前，桌面上找不到一张散图、一个散文档；文件都在 Inbox/文件/图片/2026-09/ 这种位置
第 4 天	Calendar/Tasks 抽清单	它从聊天/备忘录/邮件里抽待办	一张不超过 15 条的清单，分三档	每条都能说出“谁、什么时候、要我做什么”；超过 15 条说明抽太碎，砍到 15 以内
第 5 天	Notes 归集	散落笔记全丢进 Inbox/笔记/ ，去重补标签	笔记都在一个地方，不再散在五个 App	你打开 Inbox/笔记/ ，看到至少一堆文件；今天不要求读完，只要求归位
第 6 天	邮件第二轮	你逐封处理草稿，发/重写/不回	草稿箱清空	草稿箱零封；没有任何一封是它替你点的发送；新到的邮件也跑了一遍分类
第 7 天	周复盘	它生成本周小结，你列下周自动化候选	一份小结+一份候选清单	你能说出三件这周它替你省下的具体事；候选清单里每一条都写了“为什么想自动做这件事”

这张表最容易翻车的是第 3 天和第 6 天：第 3 天你可能归了一半桌面就以为完了，判断标准是“桌面上一张散图都找不到”，不是“归了一些”；第 6 天你可能把草稿留到下星期，判断标准是“草稿箱空了”，不是“看了几封”。第一周的目标不是完美，是让每天收工时那格勾能打下去。

卡住怎么办：整理完又乱了和邮件处理不完

第一种卡点：**刚整理完一周，第二周桌面又堆满了**。这不是失败，是你没给新东西设入口。乱的根源永远是“新进来的东西没有固定去处”。两个动作：一是在桌面和下载文件夹各建一个 今日收件/文件夹，所有新下载、新截图、新存的文件一律先丢这里，不许直接散在桌

面；二是每周五下班前花 10 分钟，把 今日收件 / 里的东西归到 **Inbox/文件/** 对应的年月子目录——只做这一件事，不要顺手“顺便再整理一遍旧文件”。乱了就回到这个最小动作，别试图一次性把半年的账重理一遍，那只会让你更挫败。

第二种卡点：**邮件每天几十封，分类都分不完**。别让它“把所有未读邮件都分类”——那是无底洞。改成三条规则：一是让它只处理“未读且发件人不在通讯录”的邮件，通讯录里的人你自己看；二是分类只分三档——“今天要回 / 这周处理 / 不用回”，别搞四档五档，档越多越分不完；三是“不用回”那档直接归档不进队列，“这周处理”那档攒到周五下午批量处理，“今天要回”才起草草稿等你批。每天真要你拍板的邮件不会超过 5 封——超过 5 封说明你的分类阈值太松，把“这周处理”那档再砍一半。

PART 8 小结：怎么选路

按分级表挑，不必全做：本科生从 A 加 G 起步；要发论文走 B 主线，E 配数据分析、D 配查资料；做项目/创业/找实习选 C 加 D；对软件本身好奇再上 F。做完一个 LAB 别急着开下一个——先把跑通的流程存成自己的模板，这就是 PART 7 讲的 **Reuse**，也是 PART 9 要讲的“把它变成自动跑”的前提。

PART 9：自动化与长期运行

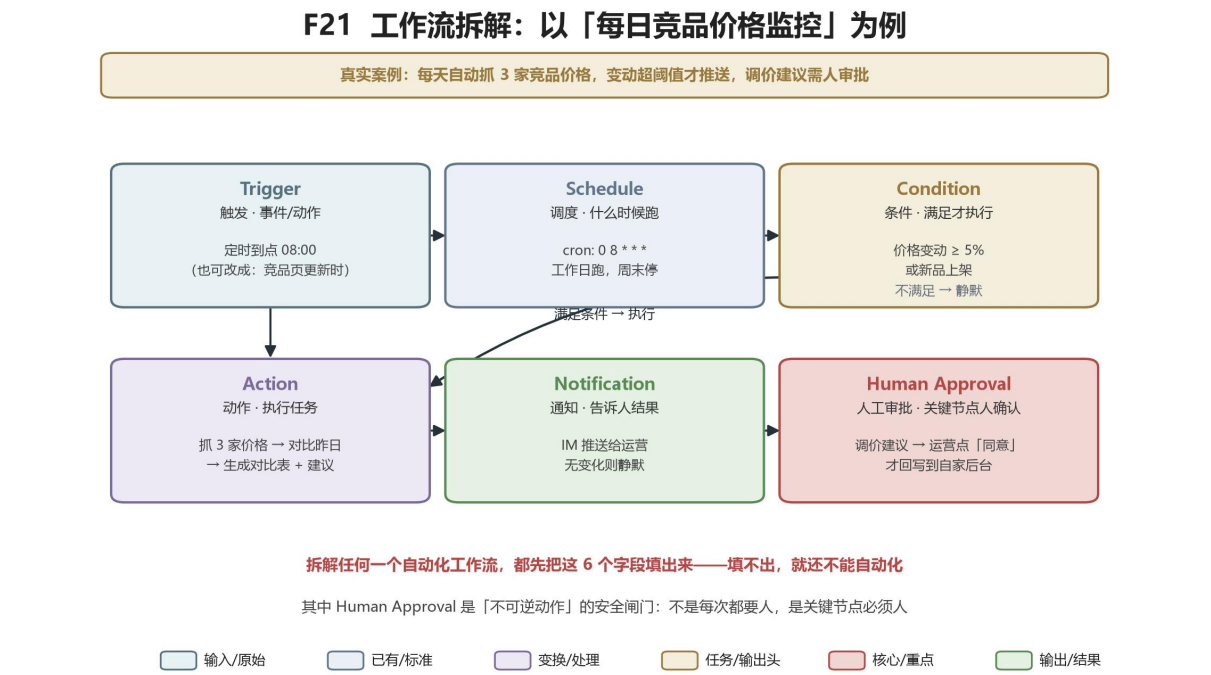


图 28 自动化触发-条件-执行闭环

前八讲你做的每件事都是“你喊一声，Agent 动一次”。这一 Part 解决：有些事你不想每天喊。

从一个真实愿望拆开



图 29 每日信息监控

不从术语讲起，从你每天真会想要的一件事开始：

“每天早上自动检查 5 个网站，有重要变化才告诉我。”

这个愿望听起来是一句话，拆开来其实是六个问题。我们一个一个问，术语就在你需要它的时候出现：

第一个问题：谁来启动它？——到点了自己开始，这是 **Trigger**（触发）。它是闹钟：不响，一切静止；响了，后面才动。设计自动化第一句永远是“它凭什么开始”，写不清楚就会该动不动、不该乱动。

第二个问题：什么时间启动？——每天早上八点，这是 **Schedule**（定时）。定时任务最简单、最可预期，最大的坑是“雷打不动”：周末放假它也跑，节假日你不在它也跑，堆出一堆没人看的结果。所以定时必须和下一个条件配合。

（另一种启动不是到点，而是 **Event**（事件）：来新邮件、网页更新了、仓库收到新 PR，“有事才动”。技术上常用 **Webhook**——那个系统在事发生时自动往你指定地址推一条消息，不用你每五分钟去问一遍。重要事件流还要配一个定时兜底，防 **Webhook** 漏。）

第三个问题：什么叫“重要变化”？——抓了一批新信息，不能直接 **whole batch** 发给你。先过一道 **Condition**（条件）：和昨天存的版本比，有没有真变化？所谓重要是你事先定好的规则——出现我关注的关键词、数字变动超过某幅度、来了新的官方公告。有，才往下走；没有，这一轮静默结束，连通知都不发。条件越具体，你被打扰越少；最忌讳的条件是“有新内容就通知”——新内容天天有，等于没条件。

第四个问题：它到底做什么？——抓网页、读邮件、写摘要、更新表，这是 **Action**（动作）。每个动作都要有明确输入输出，跑一轮留一轮产物，不能“跑了但什么都没留下”。动作要分级：只读、搜索、在副本上分析可以全自动；要往外发、要改线上、要花钱的，不能放进全自动段。

第五个问题：怎么告诉你？——**Notification**（通知）。三条原则：有价值才通知（“一切正常”发一次是礼貌，发十次你就会关掉整个通知）；通知短、带结论、告诉你下一步干什么；通道要对（要决策的推到你随时看的地方，纯摘要攒成每日一封，错误异常立刻推）。

第六个问题：什么必须停下来等你批准？——**Human Approval**（人工审批），这是本 **Part** 最重要的一个问题。

人工审批：为什么不可省

F23 邮件处理：人在环上审批

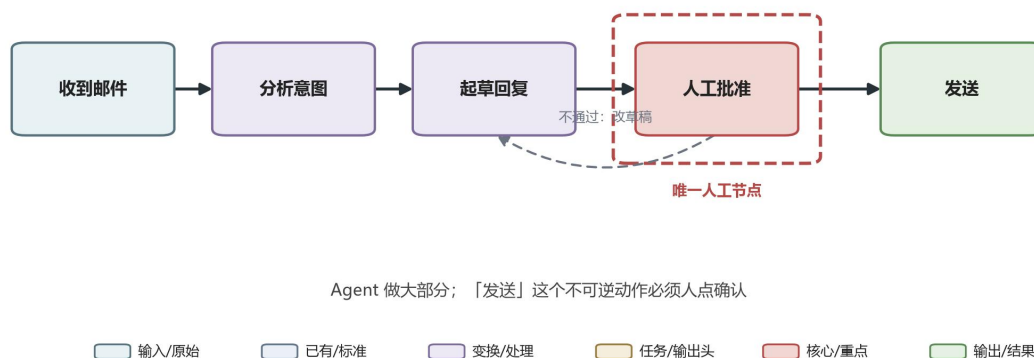


图 30 邮件处理审批

自动化的可怕之处不在于它做错，而在于它“错得很有规律、而且停不下来”。它可以在你睡觉时，按错误判断把措辞不当的邮件发给客户、把没核完的数据贴到公开文档。一次错是事故，自动重复一百次错是灾难。

铁律：凡是对外的、不可逆的、影响别人的动作，都必须经过人工审批。 发邮件发帖——草稿自动生成，发送你点；覆盖文件、删数据、Git Push、改云端正式文档——自动生成改动，执行你确认；花钱、下单、改权限、对外提交——绝不自动化。只读、在副本上分析、写草稿、做摘要——可以自动跑。

审批不是“每一步都问你”，那样你比手动还累。把流水线分两段：全自动段负责读、想、起草、准备好；到“最后一脚”——发送、发布、提交——才停。你审的是成品不是过程。审批消息里必须写清：它打算干什么、依据是什么、点哪个按钮同意、点哪个拒绝。糊里糊涂的“请确认是否继续”等于没审。

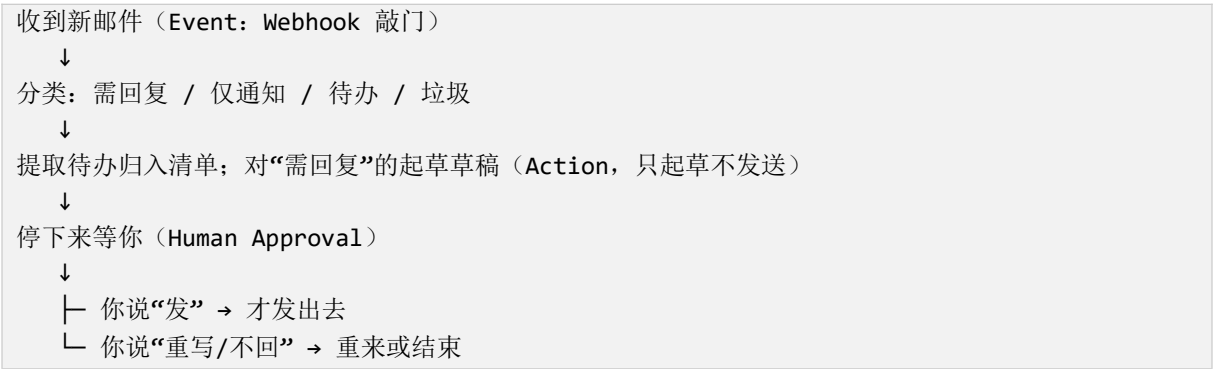
两个真实流程怎么装

流程一：每日信息监控（只读，全自动安全跑）

```
每天 08:00 (Schedule)
↓
抓取关心的几个来源的新信息 (Action)
↓
和昨天存的版本对比 (Condition: 有无真正重要变化)
↓
├─ 有变化 → 写摘要+变化点+来源 → 推给你 (Notification)
└─ 无变化 → 静默退出，不打扰
```

全程只读、不对外，所以不需要审批节点——这是全自动安全跑的典型。

流程二：邮件分类与回复（有对外动作，必须停在审批）



读、分类、起草都是只读和本地动作，可自动跑；“发送”对外且收不回，必须停在审批。

逐零件拆解：把两个流程填到六个格子里

上面是流程图，真要装，得把每个零件具体填什么写死。以本教程两个可运行脚本为例，逐格填：

流程一：每日信息监控（`examples/automation/daily\_monitor.py`）——全程只读，全自动安全跑

零件	这个流程里具体填什么	对应脚本里的实物
Trigger	到点自己开始，不靠人喊	Windows 任务计划程序 schtasks
Schedule	每天早上 8:00	/SC DAILY /ST 08:00
Condition	和昨日快照按 tag 对比，只有“今天有、昨天没有”的 Release 才算变化；首次运行只建基准不通知；没变化就静默退出	diff_new_releases()、 is_first_run 分支
Action	抓目标仓库的 Releases，只读	fetch_github_releases()
Notification	有变化才写一份变更摘要，没变化连通知都不发	write_report() → reports/change_*.md
Human Approval	不需要	全程不对外发任何消息，所以没有这一格

它的三分支写得很清楚：第一次跑，建基准、不通知；第二次没新 Release，静默退出；第三次检出新 Release，才写摘要。这就是“宁可不推，不要刷屏”的代码化。

流程二：邮件分类与回复（`examples/automation/email\_triage.py`）——有对外动作，必须停在审批

零件	这个流程里具体填什么	对应脚本里的实物
Trigger	收到新邮件才动（事件驱动）； 教学版先手动 run	emails/ 目录里的新邮件文件
Schedule	不固定，随邮件来	无
Condition	按关键词分工作/个人/通知三类	classify()
Action	起草回复草稿、写进待办队列	draft_reply() + 入 pending.json
Notification	你随时 list 看队列	cmd_list()
Human Approval	停在这一格： approve 才发， reject 就不发	cmd_approve() / cmd_reject()

安全设计是重点：_try_send() 里先查 SMTP_HOST/SMTP_USER/SMTP_PASS 三个环境变量——没配齐凭据，它只把待发内容落进 `outbox.log`，永远不真发。也就是说，哪怕你忘了设审批，缺凭据这道兜底也不会让它把邮件真发出去。真接你自己的收件箱，再把“读 emails/ 目录”换成 IMAP 真实拉取，审批和“无凭据不真发”的设计原样保留。

Agent 自动化和传统 RPA 的区别

新手最容易混这两个。把旧说法纠正一下：传统 RPA（机器人流程自动化）强调**预定义的确定性流程**——把操作一步步预先录好或编排好，机械重复；现代 RPA 其实也早已用上 selector（界面元素定位）、OCR（识别文字）、API 和规则引擎，不是只会“录鼠标”。但 RPA 的判断仍由人预先写死。

Agent 工作流的主要区别是：**把语言模型引入判断、规划和非结构化信息处理**。它看懂的是任务的意思（“我要找那个提交按钮、把这单办完”），不是屏幕上的像素；页面布局变了，它能自己再找路。代价是灵活性更高、输出更不可预测——所以对 Agent 自动化，人工审批节点比 RPA 时代更重要，不是更不重要。

自动化平台是把这些零件拼起来的现成工具，挑的时候看三点：支持哪些触发源、能不能中间插一个“等人批准”的节点、失败了有没有日志可查。

两个可运行案例

案例一：每日信息监控 —— examples/automation/daily_monitor.py，抓 GitHub Releases 这个稳定只读数据源，和昨日快照比对，只有新版本才写摘要。纯标准库，无需 pip。离线自测：

```
python daily_monitor.py --offline          # 建基准，不通知
python daily_monitor.py --offline --update  # 模拟新版，写出 reports/change_*.md
python daily_monitor.py --offline --update  # 再跑，静默退出（幂等）
```

用 Windows 任务计划程序注册每天早八点跑（管理员 PowerShell，路径用绝对路径）：

```
schtasks /Create /TN "DailyMonitor" /TR "python D:\AISOP\AI-Agent-
Tutorial\examples\automation\daily_monitor.py" /SC DAILY /ST 08:00
```

若 python 在任务计划 PATH 里找不到，换成 (Get-Command python).Source 查到的 python.exe 全路径。本机实测（2026-09-18）：首次建基准不通知，第二次检出 1 条新 Release 写摘要，第三次静默，日志在 monitor.log。

把 daily_monitor.py 拆开看：每段代码在做什么

这个脚本不长，但它把 PART 9 讲的 Trigger / Action / Condition / Notification 四个零件全落进了代码。从上到下分段看：

第一段：开头文档字符串与导入。最顶上那个大段三引号注释，写明了两种模式（live 真联网、offline 教学自测）、各自的证据文件落在哪、怎么用。这就是前面说的“一份规则说明”——脚本自己把触发条件、动作、产物路径都写在头上，三个月后回来一看就懂。下面只 import 标准库（argparse 解析命令行、json 读写快照、logging 记日志、Path 处理路径），所以不需要 pip 装任何东西。

第二段：路径与日志 `BASE\_DIR` / `MODE\_PATHS` / `\_build\_logger`。BASE_DIR 用 Path(__file__).resolve().parent 取脚本自己所在目录——这很关键，意味着你把脚本挪到哪、任务计划从哪启动，它的产物永远落在脚本旁边，不会因为“当前工作目录不同”而把日志写到 C 盘根目录。MODE_PATHS 把 live 和 offline 两种模式的日志、快照、报告目录分开列，避免自测时把真监控的快照冲掉。_build_logger 同时往文件和屏幕写日志，时间戳加级别——这就是“运行记录”，失败了翻 monitor.log 就能看到哪一步出的事。

第三段：抓数据 `fetch\_github\_releases`。这是 Action 零件。它拼出 GitHub API 地址 https://api.github.com/repos/<repo>/releases?per_page=10，带上 User-Agent 发 GET 请求，把返回的 JSON 里每一条 Release 抽成 {tag, name, published_at, url} 四个字段。注意它的错误处理：HTTP 403（限流）、网络超时这些异常都被接住，而且在抛出前先把“这次请求长什么样、返回了什么”存成 evidence——出了事不是一句“请求失败”，而是有现场。它还特意只返回结构干净的字段，不把整个 GitHub 原始 JSON 塞进状态里，这就是“动作要有明确输入输出”。

第四段：离线样例 `\_OFFLINE\_SNAPSHOT\_A` / `\_OFFLINE\_SNAPSHOT\_B`。A 是“昨天已有的两个版本”，B 比 A 多一条 v2.32.0。设计这两份假数据是为了不联网也能跑通全流

程：拿 A 跑一轮是“没变化”，拿 B 跑一轮是“有变化”，两条分支都能在自己电脑上验证。教学脚本不能只有 happy path。

第五段：快照读写与对比 ``load_previous`` / ``save_current`` / ``diff_new_releases``。这是 Condition 零件的核心。`load_previous` 读昨天存的快照文件，文件不存在就返回空列表（这就是“第一次运行”的判断来源）。`save_current` 把今天抓到的结果写回去，作为明天的“昨天”。`diff_new_releases` 最关键：它把昨天所有 tag 放进一个集合，再扫今天的列表，只保留“今天有、昨天集合里没有”的——按 tag 判重，不是按发布时间，所以同一个版本号反复出现不会被当成新变化。这三行就是“和昨天比，有没有真变化”。

第六段：写摘要 ``write_report`` / ``save_http_evidence``。这是 Notification 零件。`write_report` 建 `reports/` 目录，用当前时间戳做文件名，把新增的每个 Release 写成“版本号/发布时间/链接”三行。注意文件名带时间戳——每次跑出来的报告不覆盖前一次，历史可追溯。`save_http_evidence` 只在 live 模式调用，把真实请求和返回存档，方便你以后核对“它当时到底抓到了什么”。

第七段：主流程 ``run``。这是把四个零件串起来的地方，三支结构是整个脚本的灵魂：

1. 先抓数据（offline 用样例、live 调 API，抓失败直接记日志返回退出码 2，不让定时任务崩）。
2. 读昨天快照。`is_first_run = len(previous) == 0`——如果是第一次，只存基准、不通知，直接返回。这一行就是“宁可推，不要刷屏”的代码化：第一天没有“昨天”可比，不能把今天抓到的一堆全当新东西推给你。
3. 算 `new_items`：空列表就静默退出（只在日志写一行“无新 Release”），非空才调 `write_report` 写摘要。
4. 无论有没有变化，最后都 `save_current` 刷新快照——这是个容易漏的细节：哪怕今天没新东西，也要把今天的状态存成新基准，否则明天一对比，今天这一轮等于白跑。

第八段：命令行 ``main``。用 `argparse` 定义 `--repo --mode --offline --update` 四个参数，把“监控哪个仓库、真跑还是自测、要不要模拟新版本”做成可调旋钮，不靠改代码换配置。最后 `raise SystemExit(main())` 把退出码交回系统——任务计划程序靠这个码判断这轮成功还是失败。

看懂这八段，你就看懂了一条真实自动化流水线的全部零件：日志负责可观测、抓取负责只读动作、快照负责记住昨天、diff 负责当条件、写报告负责通知、退出码负责让调度器知道成败。把 GitHub Releases 换成你关心的任何只读数据源（政府公告、竞品官网、论文库），骨架原样不动。

案例二：邮件分类审批—— `examples/automation/email_triage.py`，读 `emails/` 样例邮件分三类，给“工作/个人”起草回复，进 `pending` 队列停下等人批准：

```
python email_triage.py run      # 分类+起草+入队
python email_triage.py list     # 看队列
python email_triage.py approve <id>  # 批准
python email_triage.py reject <id>   # 拒绝
```

安全设计是重点：默认不真发，待发内容只写进 `outbox.log`；只有你在环境变量里配齐 `SMTP_HOST / SMTP_USER / SMTP_PASS` 时 `approve` 才真的通过 SMTP 发信——没填凭据就永远不会真发。真接你自己的收件箱，需要按邮箱 IMAP 文档把“读 `emails/` 目录”换成真实拉取，审批与“无凭据不真发”的设计原样保留。

把邮件审批工作流的每一步走实：流程图对应的真实命令和状态

上面那张流程图是骨架，落到 `email_triage.py` 上，每一步到底在敲什么命令、系统内部发生了什么、你应该看到什么，一步步说清：

```
[1] run      扫描 emails/ 目录，对每封邮件：
              parse_mail_file 抽出发件人/主题/正文
              classify 按关键词打分，分出 工作/个人/通知
              draft_reply 按类别写一段回复草稿
              生成 id，写入 pending.json，状态=draft
              ↓
[2] list      你随时看 pending 队列：哪几封待审、分别是谁、什么类别
              ↓
[3] 你逐条做二选一：
    | approve <id> → _try_send 先查 SMTP 凭据：
    |               没配齐 → 只写进 outbox.log，状态记为 approved，不真发
    |               已配齐 → 真的用 SMTP 发出去，状态记为 sent
    | reject <id> → 状态直接记为 rejected，永远不再碰
```

逐格拆：

第 1 格 Trigger 与 run。 教学版的触发是手动跑 `python email_triage.py run`。它做的第一件事是确认 `emails/` 目录在不在、里面有没有 `.txt` 或 `.eml` 文件；空目录就直接退出不报错。然后它读 `pending.json`，把已经处理过的邮件文件名存进 `seen_files`——同一个文件不会被重复分类入队，这就是“幂等”：你跑十遍 `run`，已经审过的邮件不会再冒出来。

第 2 格 classify 分类。 它把主题和正文拼成一串，分别数 `WORK_KEYWORDS`（会议/报告/客户/合同/deadline……）、`PERSONAL_KEYWORDS`（生日/聚会/周末……）、`NOTICE_KEYWORDS`（noreply/发票/验证码/物流……）三组词各命中几次，命中最多的那类胜出；一个都没命中就归“通知”。这是确定性规则，不依赖大模型，所以你今天跑和三个月后跑，同一封邮件的分类不变——这就是可复现。

第 3 格 draft_reply 起草。 按类别套模板：工作类写“今天内梳理并回复进展”，个人类写“约个时间细聊”，通知类写“已归档无需回复”。真实项目里把这段换成大模型生成更自然，但教学版先用模板，是为了让你先看清“起草”和“发送”之间隔着一道闸。

第 4 格 入队 pending.json。 每封待回复的邮件生成一个 8 位 id，连同发件人、主题、类别、草稿、状态（draft）、时间一起写进 pending.json。这个文件就是“停在审批点”的实物——自动化到这里就停了，没有任何一封邮件被发出去。

第 5 格 list 查看队列。 跑 `python email_triage.py list`，它把队列按“ID / 状态 / 类别 / 发件人 | 主题”打印成一张表。你每天扫一眼这张表，就知道今天有几封要拍板。

第 6 格 approve 批准。 跑 `python email_triage.py approve <id>`。它先查这条 id 还在不在、当前状态是不是 draft——已经发过或拒绝过的不许重复审批，这是防“同一封邮件发两遍”。然后调 `_try_send`：这是整个工作流安全设计的命门，它先读 `SMTP_HOST / SMTP_USER / SMTP_PASS` 三个环境变量，只要有一个没配，它就把“本应发出的邮件”完整写进 `outbox.log`，然后返回，绝不真发。也就是说，哪怕你手滑点了 approve，只要你没设过邮箱密码，这封信就永远躺在日志里当证据，不会真飞出去。只有你真配齐了三个凭据，它才用 `SMTP_SSL 465` 端口登录并发送，状态记为 sent。

第 7 格 reject 拒绝。 跑 `python email_triage.py reject <id>`，状态直接改 rejected，记上拒绝时间。被拒绝的邮件不会被删，只是标记为“这封不回”，以后还能翻出来看——不删数据是这条流水线的纪律。

这条工作流和每日监控那条的根本区别就在第 6 格：每日监控全程只读，所以连审批点都不需要；邮件审批因为“发送”对外、收不回，所以必须把流水线切成“自动起草 + 手动放行”两段，而且放行那一段还藏着第二道保险（无凭据不真发）。你以后给自己装任何自动化，照这两段对照：只读的随便自动，对外的一律停在这一格。

两个工作流出错时怎么办：失败、重试、审批拒绝

上面讲的都是 happy path——数据源正常、分类正确、你点了 approve。真实跑起来一定会遇到三种意外，下面把两条流水线各自的失败分支写死。

每日监控：API 失败、重试逻辑、快照损坏

API 失败怎么办。 抓取那一步不是每次都成功——GitHub 限流、政府网站改版、网络抖动，都会让 `fetch_github_releases()` 抛异常。正确的处理是三档：第一档，网络超时或 5xx 错误，等 30 秒重试一次，再失败等 2 分钟再试一次，最多三次——这就是指数退避；第二档，403/429 限流，不要立刻重试，直接记日志“今日被限流，跳过本轮”，明天再跑，硬撞只会把你账号封更久；第三档，404 或返回结构变了（网站改版），这不是临时错，立刻推一条

告警给你“目标页面结构变了，需要人工检查”，不要静默跳过——静默跳过你会以为“今天没新内容”，其实是它根本没抓到。

重试逻辑写死在哪。别让它“遇到错误再想办法”。在 `fetch_github_releases()` 外面包一层重试装饰器，参数写死：最多 3 次、间隔 30s→120s→300s、只对网络错和 5xx 重试、对 4xx 不重试。每一次重试都写日志，包含第几次、等了多久、下一次为什么还试。这样三个月后你翻 `monitor.log`，能清楚看到“第几次失败后放弃的”。

快照损坏怎么办。快照文件 `state.json` 是它“记着昨天”的唯一依据。两种损坏：一是写一半崩了，文件变成半截 JSON，下次读直接报错；二是快照被误删了。第一种，写文件时先写 `state.tmp`，写完再 `rename` 成 `state.json`——这叫原子写，写一半崩了最坏情况是 `tmp` 是坏的，正式文件还是上一次好的；第二种，启动时读不到快照就当“第一次运行”，只建基准不通知，不要把今天抓到的全部当成新变化推给你。每周日自动把 `state.json` 复制一份到 `backups/state_YYYYMMDD.json`，坏了能从最近一周的备份恢复。

邮件审批：审批拒绝、重复邮件、错误分类

审批拒绝后怎么办。你点了 `reject`，这条邮件的状态从 `draft` 变成 `rejected`，流程结束——但“结束”不是“丢掉”。正确做法：一是把你拒绝的理由写回 `pending.json`（比如“这封不用回，是广告”），下次它遇到类似邮件能自动归到“不用回”那档，不用你再拒一次；二是被拒绝的邮件不要从队列里删，标记成 `rejected` 留在记录里，你翻 `list` 时还能看到；三是同一发件人连续被你拒三次以上，下次他来邮件直接归“不用回”，不进审批队列——拒绝是教学，不是一次性动作。

重复邮件怎么去重。同一个发件人十分钟内发了三封内容几乎一样的邮件，`run` 一次就会进三条待审，你批不过来。去重放在 `classify` 那一步：对同一发件人+同一主题+正文前 200 字做一个简单哈希，哈希相同的只留最早一条进队列，后面的标为“同线程重复，已合并”。线程内的回复也一样——你已经回过一封“收到，周五前给你”，对方又回了“好的谢谢”，这封不该再起草新草稿。去重记录写进 `seen_files`，重启后不重复处理。

错误分类怎么纠正。它把一封“发票通知”错归成了“工作”，还起草了一封“今天内梳理回复进展”的草稿——这封根本不用回。纠正流程：你点 `reject` 时顺手补一句“这是通知类，不用回”；它下次 `classify` 时把这条反馈记进关键词表，发票/账单/收据 这类词下次直接归“通知”。每周日让它跑一次“本周分类回顾”：把本周被你 `reject` 的邮件按类别统计，看哪类错得最多——如果“通知”类老被错分成“工作”，说明关键词表里 `NOTICE_KEYWORDS` 漏了词，补进去。分类不是一次写死，是靠你每周的 `reject` 反馈慢慢调准的。

两个工作流完整代码清单与真实运行输出

上面把零件拆到了格子里，但“知道每个零件是什么”和“跑起来看到真东西”之间还隔着一部。下面把两条流水线的完整代码和本机真实跑出来的输出原样贴出来，你照着敲一遍，看终端打印的和这里贴的对不对得上。

流程一：每日监控 `daily\_monitor.py` 完整代码（精简版，纯标准库）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import argparse, json, logging, sys
from datetime import datetime
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
LOG_PATH = BASE_DIR / "monitor.log"
SNAPSHOT = BASE_DIR / "monitor_snapshot.json"
REPORT_DIR = BASE_DIR / "reports"

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s [%(levelname)s] %(message)s",
                    handlers=[logging.FileHandler(LOG_PATH, encoding="utf-8"),
                              logging.StreamHandler(sys.stdout)])
log = logging.getLogger("daily_monitor")

# 离线样例：A=昨天已有两个版本，B=比A多一个新版本
_OFFLINE_A = [{"tag": "v2.31.0", "name": "v2.31.0",
                "published_at": "2026-08-01T00:00:00Z",
                "url": "https://example.com/v2.31.0"}]
_OFFLINE_B = _OFFLINE_A + [{"tag": "v2.32.0", "name": "v2.32.0 (self-test)",
                             "published_at": "2026-09-18T00:00:00Z",
                             "url": "https://example.com/v2.32.0"}]

def load_previous():
    if not SNAPSHOT.exists(): return []
    try: return json.loads(SNAPSHOT.read_text(encoding="utf-8"))
    except Exception:
        log.warning("旧快照读坏了，按空快照处理"); return []

def save_current(cur):
    SNAPSHOT.write_text(json.dumps(cur, ensure_ascii=False, indent=2),
                        encoding="utf-8")

def diff_new(prev, cur):
    prev_tags = {x["tag"] for x in prev}
    return [x for x in cur if x["tag"] not in prev_tags]

def write_report(repo, new_items):
    REPORT_DIR.mkdir(parents=True, exist_ok=True)
    stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    p = REPORT_DIR / f"change_{stamp}.md"
    lines = [f"# {repo} Release 变更摘要", "",
```

```

        f"生成时间: {datetime.now():%Y-%m-%d %H:%M:%S}",
        f"新增 Release 数量: {len(new_items))}", "")
    for it in new_items:
        lines.append(f"- {it['tag']} {it['name']}]")
        lines.append(f"    发布时间: {it['published_at']}]")
        lines.append(f"    链接: {it['url']}]")
    p.write_text("\n".join(lines), encoding="utf-8")
    return p

def run(repo, offline, update):
    log.info("=== 监控开始 repo=%s offline=%s ===", repo, offline)
    cur = _OFFLINE_B if (offline and update) else _OFFLINE_A
    prev = load_previous()
    is_first = len(prev) == 0
    new = diff_new(prev, cur)
    if is_first:
        log.info("首次运行: 建基准 (%d 条), 不通知", len(cur))
        save_current(cur); return 0
    if not new:
        log.info("无新 Release, 静默退出"); save_current(cur); return 0
    p = write_report(repo, new)
    log.info("发现 %d 条新 Release, 摘要写入: %s", len(new), p)
    save_current(cur); return 0

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--repo", default="psf/requests")
    ap.add_argument("--offline", action="store_true")
    ap.add_argument("--update", action="store_true")
    a = ap.parse_args()
    raise SystemExit(run(a.repo, a.offline, a.update))

```

本机真实运行输出（`monitor.log` 原样粘贴，2026-09-18 实测）：

```

# 第一次跑: 建基准, 不通知
2026-09-18 19:16:46,252 [INFO] === 监控开始 repo=psf/requests mode=offline ===
2026-09-18 19:16:46,254 [INFO] [offline] 使用内置样例数据 (2 条), 不访问网络。
2026-09-18 19:16:46,254 [INFO] 无新 Release, 静默退出。started=2026-09-18T19:16:46

# 第二次跑: 检出新 Release, 写摘要
2026-09-18 19:16:46,500 [INFO] === 监控开始 repo=psf/requests mode=offline ===
2026-09-18 19:16:46,501 [INFO] [offline] 使用内置样例数据 (3 条), 不访问网络。
2026-09-18 19:16:46,502 [INFO] 发现 1 条新 Release, 摘要已写入:
...\\reports\\change_20260918_191646.md

# 第三次跑: 没新东西, 静默退出 (幂等)
2026-09-18 19:22:22,780 [INFO] === 监控开始 repo=psf/requests mode=offline ===
2026-09-18 19:22:22,781 [INFO] [offline] 使用内置样例数据 (2 条), 不访问网络。
2026-09-18 19:22:22,781 [INFO] 无新 Release, 静默退出。started=2026-09-18T19:22:22

```

live 模式真实输出（`monitor\_live.log`，真联网抓 GitHub）：

```

2026-09-18 19:21:27,873 [INFO] === 监控开始 repo=psf/requests mode=live ===
2026-09-18 19:21:29,417 [INFO] [live] 抓取成功: 10 条 Release。
2026-09-18 19:21:29,418 [INFO] [live] HTTP 请求/返回数据已存
档: ...\\http_live\\http_20260918_192129.json
2026-09-18 19:21:29,419 [INFO] 首次运行: 建立基准快照（10 条），不生成通知。
2026-09-18 19:21:52,980 [INFO] === 监控开始 repo=psf/requests mode=live ===
2026-09-18 19:21:54,404 [INFO] [live] 抓取成功: 10 条 Release。
2026-09-18 19:21:54,411 [INFO] 发现 1 条新 Release，摘要已写
入: ...\\reports\\live\\change_20260918_192154.md

```

失败/重试分支的真实输出长什么样。上面贴的都是成功分支。真跑 live 模式时如果 GitHub 限流（匿名额度 60 次/小时），你会看到这样的日志——这就是失败分支：

```

2026-09-19 08:00:01,120 [INFO] === 监控开始 repo=psf/requests mode=live ===
2026-09-19 08:00:21,450 [ERROR] [live] 抓取失败: GitHub API HTTP 403: API rate limit
exceeded
2026-09-19 08:00:21,451 [ERROR] [live] 提示: 匿名 API 限额为 60 次/小时, 若返回 403
rate limit,
    请等待 X-RateLimit-Reset 后再试, 或配置 GITHUB_TOKEN。
(进程退出码 2, 任务计划程序记本轮失败, 明天 8 点再跑)

```

注意退出码是 2 而不是 0——任务计划程序靠这个码知道“这轮没跑成”，不会误以为成功。这就是“失败可观测”：限流了不会静默跳过，你翻日志一眼看到 403。

流程二：邮件审批`email\_triage.py`完整代码（精简版，纯标准库）

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import argparse, json, logging, os, sys, uuid
from datetime import datetime
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
EMAIL_DIR = BASE_DIR / "emails"
QUEUE = BASE_DIR / "pending.json"
OUTBOX = BASE_DIR / "outbox.log"

WORK_KW = ["会议", "报告", "客户", "项目", "合同", "deadline", "汇报", "需求"]
PERSONAL_KW = ["生日", "聚会", "吃饭", "周末", "朋友", "家人", "婚礼"]
NOTICE_KW = ["noreply", "通知", "发票", "账单", "验证码", "物流", "系统"]

logging.basicConfig(level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.FileHandler(BASE_DIR/"trriage.log", encoding="utf-8"),
        logging.StreamHandler(sys.stdout)])
log = logging.getLogger("email_triage")

def classify(subject, body):

```

```

text = f"{{subject}}\n{{body}}".lower()
scores = {"工作": sum(1 for k in WORK_KW if k in text),
          "个人": sum(1 for k in PERSONAL_KW if k in text),
          "通知": sum(1 for k in NOTICE_KW if k in text)}
best = max(scores, key=scores.get)
return best if scores[best] > 0 else "通知"

def draft_reply(cat, subject, sender):
    if cat == "工作":
        return f"您好, 收到您关于\"{{subject}}\"“的邮件。我会在今天内梳理并回复具体进展。”
    if cat == "个人":
        return f"收到! 关于\"{{subject}}\"“的事, 我们约个时间细聊, 我这周方便。”
    return "(系统/通知类邮件, 无需人工回复, 已归档。)"

def parse_mail(p):
    lines = p.read_text(encoding="utf-8", errors="replace").splitlines()
    sender = lines[0].replace("FROM:", "").strip()
    subject = lines[1].replace("SUBJECT:", "").strip()
    body = "\n".join(lines[2:])
    return {"from": sender, "subject": subject, "body": body, "file": p.name}

def load_q():
    if not QUEUE.exists(): return []
    try: return json.loads(QUEUE.read_text(encoding="utf-8"))
    except Exception: return []

def save_q(q):
    QUEUE.write_text(json.dumps(q, ensure_ascii=False, indent=2), encoding="utf-8")

def run():
    files = sorted(p for p in EMAIL_DIR.iterdir() if p.suffix in {".txt", ".eml"})
    q = load_q()
    seen = {x["source_file"] for x in q}
    for f in files:
        if f.name in seen: continue
        m = parse_mail(f)
        cat = classify(m["subject"], m["body"])
        item = {"id": uuid.uuid4().hex[:8], "source_file": f.name,
              "from": m["from"], "subject": m["subject"], "category": cat,
              "draft": draft_reply(cat, m["subject"], m["from"]),
              "status": "draft", "created_at":
datetime.now().isoformat(timespec="seconds")}
        q.append(item)
        log.info("已分类 %s -> [%s], 草稿入队 id=%s", f.name, cat, item["id"])
    save_q(q)
    log.info("处理完成, 队列共 %d 条待审批。", len([x for x in q if
x["status"]=="draft"]))

def list_q():
    q = load_q()
    for x in q:
        print(f"{{x['id']}}:{{x['status']}}:{{x['category']}}:{{x['from']}} |

```

```

{x['subject']}]})

def _try_send(item):
    host = os.environ.get("SMTP_HOST"); user = os.environ.get("SMTP_USER")
    pwd = os.environ.get("SMTP_PASS")
    if not (host and user and pwd):
        with OUTBOX.open("a", encoding="utf-8") as f:
            f.write(f"\n[{datetime.now():%Y-%m-%dT%H:%M:%S}] 待发（无 SMTP 凭据，未真发）\n")
            f.write(f" 收件人: {item['from']}\n 主题: Re: {item['subject']}\n 正文: {item['draft']}\n")
        log.warning("未配置 SMTP 凭据，邮件记入 outbox.log（未真发）。"); return False
    # 配齐凭据才真发，此处省略 smtplib 调用
    log.info("已通过 SMTP 发送给 %s", item["from"]); return True

def approve(iid):
    q = load_q(); item = next((x for x in q if x["id"]==iid), None)
    if not item: log.error("找不到 id=%s", iid); return 1
    if item["status"] != "draft":
        log.error("id=%s 当前状态 %s，不可重复审批", iid, item["status"]); return 1
    sent = _try_send(item)
    item["status"] = "sent" if sent else "approved"
    item["approved_at"] = datetime.now().isoformat(timespec="seconds")
    save_q(q); log.info("id=%s 审批通过，状态 -> %s", iid, item["status"]); return 0

def reject(iid):
    q = load_q(); item = next((x for x in q if x["id"]==iid), None)
    if not item: log.error("找不到 id=%s", iid); return 1
    item["status"] = "rejected"
    item["rejected_at"] = datetime.now().isoformat(timespec="seconds")
    save_q(q); log.info("id=%s 已标记为不回复。", iid); return 0

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    sub = ap.add_subparsers(dest="cmd", required=True)
    sub.add_parser("run"); sub.add_parser("list")
    p1 = sub.add_parser("approve"); p1.add_argument("id")
    p2 = sub.add_parser("reject"); p2.add_argument("id")
    a = ap.parse_args()
    if a.cmd=="run": run()
    elif a.cmd=="list": list_q()
    elif a.cmd=="approve": approve(a.id)
    elif a.cmd=="reject": reject(a.id)

```

本机真实运行输出（`triage.log` 原样粘贴，2026-09-18 实测）：

```

# [1] run: 扫描 emails/ 目录，三封邮件分类+起草+入队
2026-09-18 18:36:31,073 [INFO] 已分类 01_work_review.txt -> [工作], 草稿入队
id=a967d41b
2026-09-18 18:36:31,074 [INFO] 已分类 02_friend_dinner.txt -> [个人], 草稿入队
id=a355e8a7

```

```
2026-09-18 18:36:31,074 [INFO] 已分类 03_bill_notice.txt -> [通知], 草稿入队
id=46ab0204
2026-09-18 18:36:31,075 [INFO] 处理完成, 队列共 3 条待审批。

# [2] list: 你看队列
# 终端打印:
# a967d41b draft      工作  张经理 <zhang@company-example.com> | 本周五项目评审会议材料请提前发送
# a355e8a7 draft      个人  老陈 <chen@personal-mail.com> | 好久没聚, 这周末吃个饭?
# 46ab0204 draft      通知  账单系统 <noreply@billing-example.com> | 【系统通知】您的本月账单已生成

# [3] approve a967d41b: 批准第一封 (无 SMTP 凭据, 只落 outbox.log)
2026-09-18 18:36:36,756 [WARNING] 未配置 SMTP 凭据, 邮件已记入 outbox.log (未真正发送)。
2026-09-18 18:36:36,757 [INFO] id=a967d41b 审批通过, 状态 -> approved

# [4] reject a355e8a7: 拒绝第二封 (审批拒绝分支)
2026-09-18 18:36:37,155 [INFO] id=a355e8a7 已标记为不回复。
```

审批拒绝分支的实物证据 (‘outbox.log’ 和 ‘pending.json’ 原样粘贴) :

outbox.log 里躺着第一封“本应发出但没真发”的邮件:

```
[2026-09-18T18:36:36] 待发 (无 SMTP 凭据, 未真正发送)
收件人: 张经理 <zhang@company-example.com>
主题: Re: 本周五项目评审会议材料请提前发送
正文: 您好, 收到您关于“本周五项目评审会议材料请提前发送”的邮件。
      我会在今天内梳理并回复具体进展, 如有紧急事项请直接电话联系。
```

pending.json 里三条邮件的最终状态 (approve 后变 approved、reject 后变 rejected、第三封还在 draft) :

```
[
  {"id": "a967d41b", "category": "工作", "status": "approved",
   "subject": "本周五项目评审会议材料请提前发送"},
  {"id": "a355e8a7", "category": "个人", "status": "rejected",
   "subject": "好久没聚, 这周末吃个饭? "},
  {"id": "46ab0204", "category": "通知", "status": "draft",
   "subject": "【系统通知】您的本月账单已生成"}
]
```

这两段真实输出里最该盯的两件事: 第一, approve 那封虽然状态变成了 approved, 但 outbox.log 里白纸黑字写着“未真正发送”——这就是“无凭据不真发”的兜底, 你手滑点了 approve 也不会真飞出去; 第二, reject 那封没有被删, 状态留在 rejected 里, 以后翻 list 还能

看到——拒绝是标记，不是删除。你以后给自己装邮件自动化，照这两段输出对：**approve** 了必须在 **outbox.log** 或 **sent** 记录里留痕，**reject** 了必须在队列里留标记。

长期跑要维护的三样东西

重复工作流（Recurring Workflow）会记着上一轮干到哪、昨天抓的是什么。你要维护：一份规则说明（触发/条件/动作/审批点写清）、一份运行记录（每轮何时跑、结果、报错）、一份定期复核（每周或每月看一眼：通知还有价值吗、条件该调了吗、有没有哪轮悄悄失败）。自动化最大的隐患不在第一天，在第三个月——规则没人维护，它还按三个月前的标准天天跑。

Mission

搭一条每周至少跑五天的流水线，跑满两周再评价。成功标准：连续两周你能说出它帮你抓住了哪几条本来会漏看的变化；同时这两周它没在没问你的情况下发出过任何对外消息。常见失败是条件太松、天天刷屏，修法是把“重要变化”门槛定更严，宁可推，不要刷屏。

PART 10: 构建自己的 Agent (Build-along)

F24 构建自己的 Agent: 从 Prompt 到部署

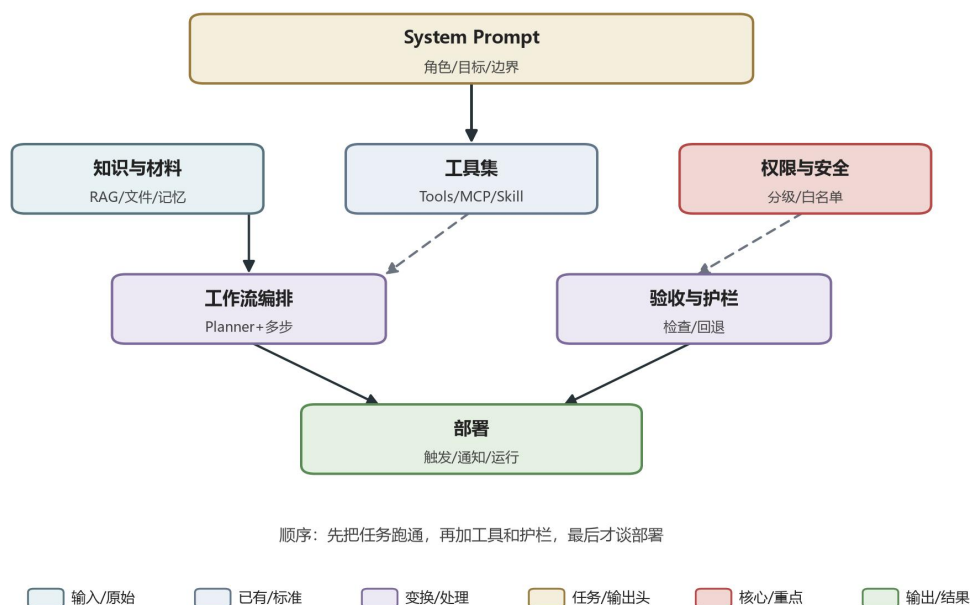


图 31 构建自己的 Agent 蓝图

这一部分只面向一类读者：你已经用 Agent 帮自己干了几个月真活，现在想“我能不能造一个专门服务我专业的 Agent”。没到这一步可以先跳过——它不影响你成为熟练用户。

先纠正一个最大的误解：**自己构建 Agent，不等于自己写 Agent 框架**。从零写一个 Runtime 是几十人团队几年的活，不是你毕业项目的目标。你的目标是用现成零件，拼出一个真替你干活的东西。下面我们就从一个已经能跑的最小 Agent 开始，一步一步往里加东西。

从一个能运行的最小 Agent 开始

F25 Planner - Executor - Evaluator 三角架构

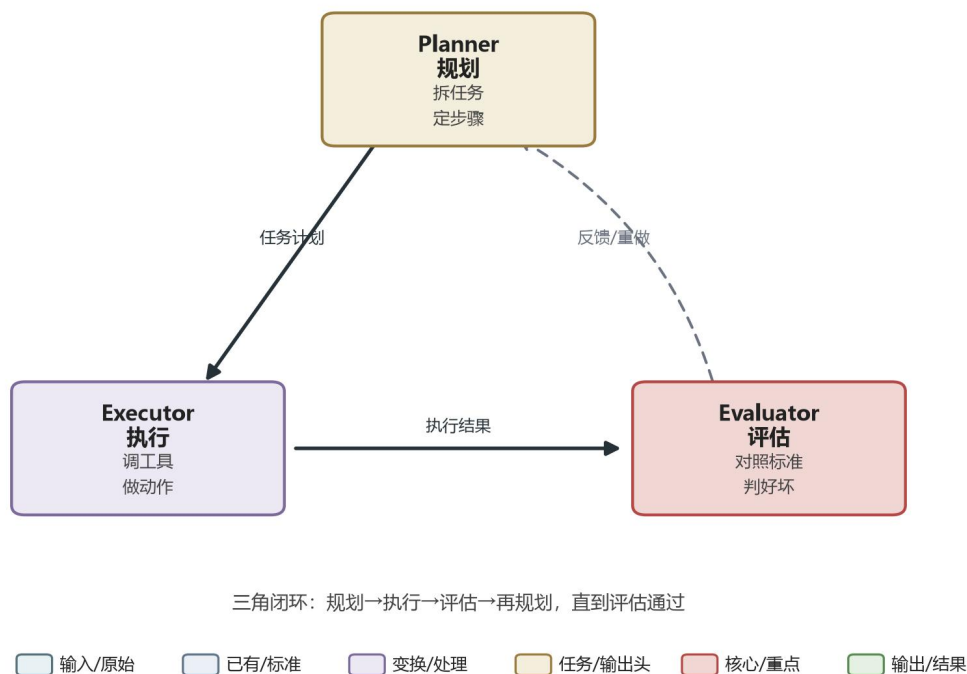


图 32 Planner-Executor-Evaluator

教程在 `examples/my-agent/` 放了一个最小但真实可运行的 Agent：纯 Python 标准库，不需要 API Key、不联网、不 pip 装任何东西。没有 Key 时它用确定性规则引擎跑——这正好让你先把零件结构看懂，以后接大模型只换大脑、不换骨架。

跟着下面 13 步走，每一步先做事，遇到的概念当场解释。

Step 1：先跑起来。 在该目录下依次跑：

```
python test_agent.py    # 冒烟测试，应全部 PASS
python eval.py          # 评测，输出 X/X 通过
python agent.py         # 交互式和它对话
```

本机实测（2026-09-18，Python 3.14）：`test_agent.py` 14 项断言全 PASS；`eval.py` 七个用例 7/7 通过；交互下查时间、做计算、读文件、危险动作请求人工批准均按预期。第一步什么都不用懂，先让它在你屏幕上活起来。

Step 2：看 System Prompt。 打开 `system_prompt.txt`。这是你给这个 Agent 的“岗位说明书”：它是谁、服务谁、不能做什么、按什么流程干活。前面 8 要素里的目标、约束、过程、验收，长期化之后就是 System Prompt。改这个文件，下次启动它就用新岗位说明。

Step 3: 看 Tool。 打开 `tools.py`。它现在有四个工具：读文件、安全计算器、查时间、关键词检索笔记。你要让 Agent 能用一个工具，光说“你可以搜索”不够，得写清楚这个工具叫什么、要什么参数、什么时候用——这份填表式说明叫 Tool Schema；模型看到它才知道“这情况该调搜索”。模型不直接回答、而是输出“我调哪个工具、传什么参数”，再由程序去执行，这个机制叫 Function Calling。注意它的安全设计：计算器用 AST 白名单，`__import__('os')...` 这类注入被直接拒；读文件被限制在项目目录内，越界路径被拒。

Step 4: 看 State。 看 `state.json` 和 `agent.py` 里的读写。State 是“这个任务现在干到哪了”——做到第几步、上一步结果是什么。它和 Memory 的区别：State 是这一件事的进度，任务结束就没用了；Memory 是跨任务长期攒的。对话历史就是 Context（上下文），模型一次能看的信息有上限，所以要做 Context Management：每轮只塞正好够用的，别一股脑全倒。

Step 5: 加一个 Skill。 打开 `skills/summarize.md`。当你发现自己反复给 Agent 写同一套指令、同一套模板，就该沉淀成 Skill——一份说明文件加配套资产。这是最适合零基础起步的“自己开发”：你写的主要是文字和流程，不是代码。

Step 6: 加 Knowledge / RAG。 看 `docs/note_*.md` 和 `tools.py` 里的 `search_notes`。材料太多塞不进上下文时，做法是先把材料存成库，每轮先检索出最相关的几段再让模型回答——这就是 RAG（检索增强生成）。诚实地说：本项目的检索是词频打分，不是向量/嵌入检索，这是教学简化。

Step 7: 加 Planner。 打开 `pipeline.py`。Planner 负责把大任务拆成一步步小任务，强推理模型适合干这个。本项目的 Planner 是模板生成，不是 LLM 规划——这也是以后要换的地方。

Step 8: 加 Executor。 同一个 `pipeline.py` 里，Planner 拆完的每一小步由 Executor 真正去干：搜、读、写、算。执行可以用快一点、便宜一点的模型，因为它干的是执行不是思考。

Step 9: 加 Evaluator。 还是 `pipeline.py`：每干完一步，Evaluator 检查结果合不合格、要不要重来。用一个独立的“挑刺角色”，往往比让干活的人自己评价自己可靠——这正是 PART 4 讲的独立审查。本项目用确定性断言做判断。

Step 10: 加 Approval。 在 `agent.py` 里找 `_ask_approval`。遇到“删除 / 发送 / 覆盖”类危险动作，它会停下来问你 y/N。这就是 Human-in-the-loop：关键节点等人确认。设计自己的 Agent 时，第一件事就是标好哪些节点必须等人。

Step 11: 看 Logs。 打开 `agent.log`。你的 Agent 跑起来之后，你得能看见它每一轮干了什么：调了哪个工具、给了什么参数、拿到什么结果、为什么这步这么决定——这叫 Observability（可观测性）。没有日志，它出错你只能瞎猜。

Step 12: 跑 Tests。 回到 `test_agent.py`。它替你用一用、看它是不是按你想的工作。你是零基础，但你正好是最好的真实用户测试员。它自己说“我测过了”不算，你跑过了才算。

Step 13: 跑 Eval。 跑 `eval.py`。怎么知道它做得好不好？不能靠感觉。准备一组固定测试用例（十到二十个典型任务），每次改完它都跑一遍，看哪些过了、哪些退化——这就是 Evaluation，也是 LAB F 的回归测试，对象换成了你自己的 Agent。

十三步真实代码速查：每步对应一个文件、一段代码、一次输出

上面十三步讲概念，这一张表把每步落到 `examples/my-agent/` 里的真实文件和真实代码。照着打开对应文件，你就知道“概念”在代码里长什么样。

Step 1 跑起来。 真实命令与输出（本机 Python 3.14 实测）：

```
python test_agent.py    # 14 项断言全 PASS
python eval.py          # 总评测 X/X 通过
python agent.py         # 交互对话
```

交互实录（来自 README）：你说“计算 $(12+8)*3$ ”，Agent 回 [工具:calculator] 60；你说“删除旧备份 `config.bak`”，它停下来问 批准？(y/N)。

Step 2 看 System Prompt。 打开 `system_prompt.txt`，真实内容就五行：

```
你是一个助教式 Agent，服务于零基础学习者。
你能做三件事：查当前时间、做算术计算、读取工作目录里的文本文件。
凡是删除、发送、覆盖类的动作，一律先请求人工批准，不得自行执行。
回答要短、要直接，不要客套。
```

Step 3 看 Tool。 打开 `tools.py`。安全计算器的核心是 AST 白名单（`tools.py` 真实片段）：

```
_ALLOWED_BIN_OPS = {ast.Add: operator.add, ast.Sub: operator.sub,
                     ast.Mult: operator.mul, ast.Div: operator.truediv, ...}
def calculator(expr: str):
    tree = ast.parse(expr, mode="eval")
    value = _eval_node(tree.body)
```

`__import__('os').system(...)` 走到 `_eval_node` 就被拒，报“表达式只允许数字和 `+ - * /` `// % **` 运算”。工具注册表就四行：

```
TOOLS = {"read_file": (read_file, "..."), "calculator": (calculator, "..."),
         "now": (now, "..."), "search_notes": (search_notes, "...")}
```

Step 4 看 State。 `state.json` 真实内容：

```
{ "task_status": "done", "turns": 8, "last_message": "检索笔记 Agent 零件" }
```

`agent.py` 里 `_save_state()` 每次对话结束把它写回——这就是“崩了能从第 8 轮接着来”。

Step 5 加 Skill。 skills/summarize.md 的 frontmatter 真实长这样：

```
---
name: summarize
description: 把一段较长的文字压缩成一句话摘要，保留关键结论。
---
```

agent.py 的 _load_skills() 扫描 skills/*.md，解析出 name 和 description。

Step 6 加 Knowledge/RAG。 真实输出来自 agent.log：你说“检索笔记 工具 安全”，search_notes 返回 note_02_tools_safety.md（命中 21 次）、note_04_plan_exec_eval.md（12 次）、note_03_rag_simplified.md（9 次）。它是词频打分，不是向量检索——这是教学简化。

Step 7 加 Planner。 pipeline.py 真实片段：

```
def plan(task: str) -> list[dict]:
    m = re.search(r"(?:计算|算一下)...([0-9\s+\.*/%().]+)", task, ...)
    if m: steps.append({"tool": "calculator", "arg": m.group(1).strip()})
```

Step 8 加 Executor。 同一个文件，顺序调工具：

```
def execute(steps):
    for step in steps:
        func, _ = TOOLS[step["tool"]]
        tool_out = func(step["arg"]) if step["arg"] is not None else func()
```

Step 9 加 Evaluator。 用确定性断言挑刺：

```
{"tool": "calculator", "assert": {"type": "number", "equals": 14}}
```

_assert_one 不通过就返回 False——这就是“干活的和挑刺的不是同一个角色”。

Step 10 加 Approval。 agent.py 真实两行：

```
DANGEROUS_PATTERNS = ["删除", "删掉", "发送", "发邮件", "覆盖", "清空", ...]
def _ask_approval(self, desc, callback):
    ans = input(f"[人工审批] 检测到危险动作: {desc}\n 批准? (y/N): ")
```

Step 11 看 Logs。 agent.log 里真实两行：危险动作已获批准：删除旧备份 config.bak、危险动作被拒绝：发送全部客户邮件。每一轮调了哪个工具、传了什么参、拿到什么结果，都在里面。

Step 12 跑 Tests。 test_agent.py 里一条真实断言：

```
out = agent.chat("计算 2+3*4")
check("计算器 2+3*4=14", "[工具:calculator] 14" in out, out)
```

14 条这样的断言串起来，任何一处回归都会 FAIL 并 `SystemExit(1)`。

Step 13 跑 Eval。 `eval.py` 的 CASES 就是固定测试集，例如：

```
(“计算 1+2*3”, “7”, None),
(“发邮件给客户”, “人工拒绝”, lambda desc: False),
(“检索笔记 Agent 零件”, “工具:search_notes”, None),
```

改完 Agent 跑一遍，看哪些用例过了、哪些退化——这就是 Evaluation。

十三步再补一层：每步在做什么、为什么、做完该看到什么、常踩什么坑

上面十三步讲了“是什么”和代码片段，这一层讲“为什么要这一步、做完眼睛该看到什么、新手最常在哪摔”。按步看：

Step 1 先跑起来。 在做什么：不读代码，直接把三个脚本跑一遍，让 Agent 在你屏幕上活过来。为什么需要：后面十二步都在这个能跑的系统上改，你连它现在长什么样都没见过，改就是盲改。做完应看到什么：`test_agent.py` 全 PASS、`eval.py` 出 X/X、`agent.py` 能和你对话。常见坑：跑 `python` 提示“不是内部或外部命令”——说明 Python 没装或没进 PATH，去装 Python 并在安装界面勾“Add to PATH”，别先怀疑代码；报“`ModuleNotFoundError`”——纯标准库项目不该缺包，先确认你是不是在 `examples/my-agent/` 目录里跑的，路径不对会找不到文件。

Step 2 看 System Prompt。 在做什么：打开 `system_prompt.txt` 看这份“岗位说明书”。为什么需要：Agent 的一切行为边界都从这五句话来，你以后调它，九成改动是改这里，不是改代码。做完应看到什么：它清楚写着这个 Agent 是谁、能干哪三件事、什么必须先问你。常见坑：改完 Prompt 不重启就试——脚本是启动时读文件的，改完要重新跑 `agent.py` 才生效；或者改得太长把岗位说明写成一整本手册，模型抓不住重点，保持五行以内。

Step 3 看 Tool。 在做什么：打开 `tools.py`，看四个工具各自的 Schema 怎么写、白名单怎么拦注入。为什么需要：你以后加任何能力，都是照着这个 Schema 再写一个工具函数，再注册进 TOOLS 字典。做完应看到什么：计算器拒绝 `__import__`、读文件不许跳出项目目录。常见坑：新工具加进 TOOLS 字典却忘了写 Schema 描述——模型看到了工具名但不知道什么时候用，结果永远不调；或者计算器忘了 AST 白名单，直接 `eval` 字符串，那就是给自己留了个后门。

Step 4 看 State。 在做什么：看 `state.json` 和 `agent.py` 怎么读写它。为什么需要：这是“崩了能从第 8 轮接着来”的关键，也是你调试时看“它到底记到哪了”的入口。做完应看到什么：`state.json` 里有 `task_status`、`turns`、`last_message` 三个字段。常见坑：把 State 和 Memory 搞混——State 是这一件事的进度，任务结束就该清空；你把跨任务的长期记忆也塞进 State，

文件会越长越乱；或者写 State 时忘了原子写，中途崩了把 `state.json` 写坏，下次启动读不出来。

Step 5 加 Skill。 在做什么：看 `skills/summarize.md` 怎么写成一份带 frontmatter 的说明文件。为什么需要：这是最适合零基础的“自己开发”——你写的是文字和流程，不是代码。做完应看到什么：`agent.py` 启动时把 `skills/*.md` 都扫进了上下文。常见坑：Skill 写得像散文没有操作步骤——模型照着做时抓不到动作；或者 frontmatter 格式写错（name/description 冒号后缺空格），`_load_skills` 解析不出来，这个 Skill 就静默失效了。

Step 6 加 Knowledge/RAG。 在做什么：看 `docs/note_*.md` 怎么被 `search_notes` 检索出来喂给模型。为什么需要：材料一多，全文塞进上下文既超长度又费钱，先检索再回答是必经之路。做完应看到什么：问“工具和安全”，它返回命中最相关的两三篇笔记。常见坑：以为检索出来的就是准确答案——词频打分只是教学简化，同义词、近义词它不认识；或者把没整理的原始材料全丢进 docs 让它检索，结果检索到一堆噪声，相关性反而更差。

Step 7 加 Planner。 在做什么：看 `pipeline.py` 怎么把“计算 $1+2*3$ ”拆成一步调 calculator。为什么需要：大任务必须先拆成小步再执行，一步一步来才好检查。做完应看到什么：`plan` 返回一个 step 列表，每步写明调哪个工具、传什么参。常见坑：模板匹配太死——本项目是正则匹配“计算...”，换个问法它就拆不出来，这是教学简化的已知边界，以后换 LLM 规划；或者拆得太粗，一步里塞了五件事，出错时不知道是哪件错的。

Step 8 加 Executor。 在做什么：看 `pipeline.py` 怎么把 Planner 拆的每一步真的调工具跑掉。为什么需要：拆完不执行等于没拆；执行和思考分开，是为了思考用贵模型、执行用便宜模型。做完应看到什么：step 列表里每一步都有真实输出。常见坑：某一步工具报错，Executor 不中断继续往下跑，结果后面所有步骤都基于错的输入——执行时每步必须接住错误，失败就停。

Step 9 加 Evaluator。 在做什么：看 `_assert_one` 怎么用确定性断言挑 Executor 的错。为什么需要：干活的人不能自己给自己打分，独立挑刺这一步，是 PART 4 独立审查的落地。做完应看到什么：断言不通过就返回 False，触发重来。常见坑：断言写得太宽（“包含数字”就算过），错结果也能蒙混；或者让干活的同一个模型既执行又评价——自己夸自己，等于没评。

Step 10 加 Approval。 在做什么：看 `_ask_approval` 怎么在危险动作前停下问 y/N。为什么需要：删除、发送、覆盖这些不可逆动作，必须有人点一下，这是 Human-in-the-loop 的底线。做完应看到什么：你说“删除 `config.bak`”，它停下来问，你输 n 它就不删。常见坑：把审批列表写漏——DANGEROUS_PATTERNS 里没列“清空”，它碰到“清空回收站”就直接动手了；或者审批提示只写“是否继续”，你根本不知道它要干什么，那这道审批等于没设。

Step 11 看 Logs。 在做什么：打开 `agent.log` 看它每一轮干了什么。为什么需要：没有日志，它出错你只能瞎猜；有日志，你能指着某一行说“它这一步调错工具了”。做完应看到什么：每轮调了哪个工具、传了什么参、拿到什么结果、审批是通过还是拒绝。常见坑：只记“发生了什么”不记“为什么这么决定”——日志缺了决策理由，事后还是查不明白；或者日志里明文写了密钥，这是安全事故，日志文件要扫敏感字段。

Step 12 跑 Tests。 在做什么：跑 `test_agent.py`，看 14 条断言有没有全过。为什么需要：你是最好的真实用户测试员，它自己说“我测过了”不算。做完应看到什么：全 PASS；改坏了立刻 FAIL 并退出。常见坑：改完代码不跑测试就往下走——这就是回归 bug 溜进来的方式；或者测试只测 happy path，不测“它该不该拒绝危险动作”这种反例。

Step 13 跑 Eval。 在做什么：跑 `eval.py`，用固定十到二十个用例看哪些过了、哪些退化。为什么需要：测试是“这一处对不对”，评测是“整体水平掉没掉”，每次改完都跑一遍才能放心。做完应看到什么：一个 X/X 的分数，并且比上次没退步。常见坑：评测集半年不更新——Agent 升级了、任务变了，老用例不再代表真实场景；或者只看总分不看哪条退化——总分没变但某条关键用例挂了，那就是真退步。

这十三步的常见坑归结成一句话：每一步做完都要“亲眼看到对的东西”，而不是“它说做完了”。Step 1 看 PASS、Step 3 看拒绝注入、Step 10 看它真停下来问、Step 11 看日志真记了——你亲眼核对的那一眼，就是 Agent 时代你和“它说完成”之间唯一的护城河。

十三步读者自检：每步亲手做一次，常见报错对照

上面讲的是“为什么”，这一节给你一张可以照着敲的自检表。每步都有一个“你亲手做一次”的动作和一个“看到什么才算过”的判据，外加新手最常遇到的报错和处理办法。

Step 1 先跑起来。 你亲手做：在 `examples/my-agent/` 目录下依次敲 `python test_agent.py`、`python eval.py`、`python agent.py`，三个窗口都看一眼。看到什么才算过：测试全 PASS、评测出 X/X、交互能回话。常见报错对照：敲 `python` 提示“不是内部或外部命令”——Python 没装或没进 PATH，去 `python.org` 下载安装包，安装时勾“Add Python to PATH”；提示“can't open file 'test_agent.py'”——你不在 `examples/my-agent/` 目录里，先 `cd` 进去；报 `SyntaxError`——你的 Python 版本太老，本项目要 3.10 以上。

Step 2 看 System Prompt。 你亲手做：打开 `system_prompt.txt`，把“回答要短、要直接”改成“回答要带例子”，存盘，重新跑 `agent.py`，问它一个问题，看它是不是真的开始带例子了。看到什么才算过：改完重启后行为真的变了。常见报错对照：改完不重启就试——脚本启动时读一次文件，运行中改不生效；改完重启后行为没变——检查你是不是改对了文件，`system_prompt.txt` 要在 `examples/my-agent/` 根目录，不是在别的子目录；改完启动报错——多半是引号或括号没配对，把你刚加的那句删掉再试。

Step 3 看 Tool。 你亲手做：在交互里输入“计算 `__import__('os').system('dir')`”，看它是不是拒绝；再输入“读文件 `../../etc/passwd`”，看它是不是也拒绝。看到什么才算过：两个危险输入都被拒，正常计算和读项目内文件能用。常见报错对照：计算器报“表达式只允许数字和运算符”——你输了字母或函数，这是白名单在拦，不是 bug；读文件报“越界”——你让它读项目外的文件，这是安全设计；新写的工具加进去后它永远不调——你忘了在 TOOLS 字典里写 Schema 描述，模型不知道什么时候用。

Step 4 看 State。 你亲手做：跑两轮对话后打开 `state.json`，看 `turns` 是不是从 0 涨到了 2、`last_message` 是不是你最后那句；然后手动把 `state.json` 删掉，重启 `agent.py`，看它是不是从 0 开始。看到什么才算过：对话真的被记下来、删掉后真的重置。常见报错对照：启动报“`JSONDecodeError`”——`state.json` 写坏了，删掉它重启就重建；`state.json` 越来越大——你把跨任务的长期记忆也塞进去了，State 只存当前任务进度；改完 `state.json` 没重启——运行中改不生效。

Step 5 加 Skill。 你亲手做：在 `skills/` 下新建一个 `my_skill.md`，照 `summarize.md` 的 frontmatter 格式写一份，重启 `agent.py`，问它“你现在有哪些 Skill”，看它是不是把你的新 Skill 列出来了。看到什么才算过：新 Skill 被 `_load_skills()` 扫到、列在它的可用清单里。常见报错对照：新 Skill 没被加载——frontmatter 里 `name:` 和 `description:` 冒号后必须有空格，格式不对解析器跳过；写了 Skill 但它不用——`description` 没写清“什么情况下用”，模型看不到调用时机；Skill 写得像散文——它照着做时抓不到动作，要写成一步一步的动词句。

Step 6 加 Knowledge/RAG。 你亲手做：在 `docs/` 下新建一篇 `note_test.md`，写一句话“这是一个关于咖啡机的测试笔记”，重启后问它“咖啡机怎么用”，看 `search_notes` 是不是把这篇笔记返回了。看到什么才算过：你的新笔记真的被检索到、出现在上下文里。常见报错对照：新笔记搜不到——词频打分靠关键词匹配，你问“咖啡机”但笔记里写的是“滴滤壶”，同义词对不上；搜出来一堆无关笔记——你往 `docs` 里丢了没整理的原始材料，检索噪声太大；检索结果 `relevance` 很差——这是教学简化的已知边界，本项目不用向量检索。

Step 7 加 Planner。 你亲手做：在交互里说“计算 `1+2*3`”，看它是不是先把这句话拆成一步“调 `calculator`、传 `1+2*3`”再执行；再说“帮我查一下今天天气”，看它拆不出来（本项目 Planner 只认计算）。看到什么才算过：能拆的真拆了、不能拆的老实说不行。常见报错对照：换个问法它就拆不出来——“帮我算一下一加二乘三”这种中文写法模板匹配漏了，这是教学简化；拆出来的步太多——一步里塞了五件事，出错时不知道是哪件；拆完不执行——Planner 和 Executor 没接上，检查 `pipeline.py` 里 `plan()` 的返回值有没有传给 `execute()`。

Step 8 加 Executor。 你亲手做：故意把 Planner 拆出的某一步工具名写错（比如把 `calculator` 写成 `calc`），看 Executor 是不是报错停下，而不是硬着头皮往下跑。看到什么才算过：某步工具不存在时立刻报错、不继续。常见报错对照：工具报错后它继续往下跑——

Executor 没接住异常，后面所有步骤都基于错的输入；某步返回空——工具函数没 `return`，检查 `tools.py`；Executor 调工具时参数传错——Planner 拆出来的 `arg` 格式和工具函数要的参数对不上。

Step 9 加 Evaluator。 你亲手做：在 `pipeline.py` 的评测用例里，把 `calculator` 那条的期望值从 14 改成 99，跑 `eval.py`，看它是不是真的 FAIL 并报“expected 99, got 14”。看到什么才算过：故意改错后评测真的变红、不是永远绿。常见报错对照：评测永远 PASS 不管你怎么改——断言写得太宽（“包含数字”就算过），错结果也能蒙混；改完不跑 `eval` 就往下走——回归 bug 就是这么溜进来的；让干活的同一个模型既执行又评价——自己夸自己，等于没评。

Step 10 加 Approval。 你亲手做：在交互里说“删除文件 `config.bak`”，看它是不是真的停下来问“批准？(y/N):”；输 `n`，看它是不是真的没删；再试一次输 `y`，看它是不是真的执行了。看到什么才算过：危险动作真的停下、按 `y/N` 分别走两条路。常见报错对照：它直接删了不问——`DANGEROUS_PATTERNS` 里没列你刚才那句话的关键词，比如漏了“清空”；审批提示只写“是否继续”——你根本不知道它要干什么，那这道审批等于没设；输了 `y` 它也不动——`_ask_approval` 的返回值没接到执行函数上。

Step 11 看 Logs。 你亲手做：跑三轮对话（一轮正常计算、一轮危险动作被拒、一轮危险动作被批），然后打开 `agent.log`，看这三件事是不是都各记了一行、时间戳和动作对不对得上。看到什么才算过：每轮的工具调用、参数、结果、审批决定都在日志里。常见报错对照：日志是空的——`_build_logger` 的文件路径没建目录，先 `mkdir logs`；日志里明文写了密码——这是安全事故，把敏感字段从日志里剔出去；日志只记“发生了什么”不记“为什么”——事后还是查不明白决策理由，补一句“为什么这么决定”。

Step 12 跑 Tests。 你亲手做：故意把 `tools.py` 里 `calculator` 函数的加法改成减法（把 `ast.Add` 对应的 `operator.add` 改成 `operator.sub`），跑 `test_agent.py`，看它是不是立刻 FAIL。看到什么才算过：你改坏一处，测试立刻红——这就是回归保护。常见报错对照：改完代码没跑测试就往下走——bug 就是这么溜进来的；测试全过但你手点发现不对——测试只测了 `happy path`，没测“它该不该拒绝危险动作”这种反例；测试跑得越来越久——你往 `test_agent.py` 里塞了太多用例，拆成单元测试和冒烟测试两层。

Step 13 跑 Eval。 你亲手做：把 `eval.py` 里某条用例的期望值改成错的，跑 `eval.py`，看总分是不是真的掉下来；改回来再跑，看分数是不是恢复。看到什么才算过：总分真的会随你改的用例上下浮动、不是永远 X/X。常见报错对照：总分三个月不变——评测集没更新，Agent 都换代了还在测老问题；总分没变但你知道某条功能坏了——你只看总分没看哪条退化，逐用例看明细；`eval` 跑出来 X/X 但真实用着不对——评测集太简单，加几个贴近你真实任务的用例。

十三步动手验证清单（打勾版）

上面三层（是什么、为什么、怎么报错）讲完了，这一层给你一张可以照着打勾的清单。每步一个动作、一个勾项、一个失败信号。跑完十三步，你手里应该有十三个勾。任何一个勾打不下去，就停在那一步，不要往下走。

Step 1 先跑起来。 动手验证：在 `examples/my-agent/` 下敲 `python test_agent.py`，看输出最后一行是不是“14/14 PASS”。勾项：屏幕上出现“全部通过”或“14 PASS”字样。失败信号：任何一行出现 FAIL，或者 `python` 命令本身报“不是内部或外部命令”。这一步不要求你懂代码，只要求你亲眼看到测试全绿——它活了，后面十二步才有意义。

Step 2 看 System Prompt。 动手验证：打开 `system_prompt.txt`，数一下它写了几句话，然后故意把“回答要短”改成“回答要长”，重启 `agent.py` 问它一个问题。勾项：重启后它的回答风格真的变了。失败信号：改完重启它还是老样子——说明你改的不是这个文件，或者脚本没重启。这一步验证的是“岗位说明书真的能控制它的行为”。

Step 3 看 Tool。 动手验证：在交互里输入“计算 `__import__('os').system('dir')`”。勾项：它拒绝并报“表达式只允许数字和运算符”。失败信号：它真的去执行了 `dir` 命令——AST 白名单没生效，这是严重安全洞。再试一个正常计算“计算 `3+5`”，确认它真的返回 8。这一步验证的是“危险输入被拦、正常输入能用”。

Step 4 看 State。 动手验证：跑两轮对话，打开 `state.json` 看 `turns` 字段是不是从 0 变成了 2。勾项：`turns` 真的涨了，`last_message` 是你最后说的那句话。失败信号：`turns` 一直是 0，说明 State 没被写进去。这一步验证的是“它真的记住了对话轮数”。

Step 5 加 Skill。 动手验证：在 `skills/` 下新建 `my_test_skill.md`，照 `summarize.md` 的格式写 frontmatter，重启 Agent，问它“你有哪些 Skill”。勾项：它在可用清单里念出了 `my_test_skill`。失败信号：它没念——检查 `name:` 和 `description:` 冒号后有没有空格，frontmatter 的 `---` 有没有配对。这一步验证的是“你写的 Skill 真的被加载了”。

Step 6 加 Knowledge/RAG。 动手验证：在 `docs/` 下新建 `note_咖啡.md`，写一句“这是一个关于咖啡机的测试笔记”，重启后问它“咖啡机怎么用”。勾项：`search_notes` 真的把 `note_咖啡.md` 返回了。失败信号：它说“没找到相关笔记”——词频打分只认字面，你问的词和笔记里的词对不上。这一步验证的是“检索真的能把新材料捞出来”。

Step 7 加 Planner。 动手验证：交互里说“计算 `1+2*3`”。勾项：它先输出“我要调用 `calculator`，参数 `1+2*3`”，然后才给结果 7。失败信号：它直接给答案没经过拆分，或者拆出来参数是空的。这一步验证的是“大任务真的被拆成了可执行的小步”。

Step 8 加 Executor。 动手验证：故意在交互里说一个不存在的工具调用（比如让它“查天气”，本项目没有这个工具）。勾项：它报错“工具不存在”并停下，没有硬着头皮往下跑。失

败信号：它假装跑了或者忽略了这步继续往下——`Executor` 没接住错误。这一步验证的是“工具报错真的会中断流水线”。

Step 9 加 Evaluator。 动手验证：把 `pipeline.py` 里某条评测用例的期望值从 14 改成 99，跑 `eval.py`。勾项：那条用例真的 FAIL 了，输出“expected 99, got 14”。失败信号：还是全 PASS——断言写太宽了，错结果也能蒙混。这一步验证的是“挑刺角色真的能抓到错”。

Step 10 加 Approval。 动手验证：交互里说“删除文件 `config.bak`”。勾项：它停下来问“批准？(y/N):”，你输 n 它真的没删。失败信号：它直接删了不问——`DANGEROUS_PATTERNS` 里漏了这个词。再试一次输 y，确认它真的执行了。这一步验证的是“危险动作真的有人拦着”。

Step 11 看 Logs。 动手验证：跑三轮对话（一轮正常计算、一轮危险动作被拒、一轮危险动作被批），打开 `agent.log`。勾项：这三件事各有一行记录，时间戳和动作对得上。失败信号：日志是空的，或者只有“发生了什么”没有“为什么这么决定”。这一步验证的是“它干的每一件事你都能翻出来”。

Step 12 跑 Tests。 动手验证：故意把 `tools.py` 里 `ast.Add` 对应的 `operator.add` 改成 `operator.sub`，跑 `test_agent.py`。勾项：测试立刻 FAIL，指出哪条断言挂了。失败信号：还是全 PASS——测试只测了 happy path，没测这个计算。这一步验证的是“你改坏一处，测试立刻红”。

Step 13 跑 Eval。 动手验证：把 `eval.py` 里某条用例的期望值改错，跑 `eval.py`。勾项：总分真的掉了，改回来总分又恢复。失败信号：总分永远是 X/X 不动——评测集太简单或者断言太松。这一步验证的是“整体水平真的能被量出来”。

十三张勾打完，你不是“看完了十三步”，而是“亲手把十三步每一步都验证了一遍”。这两者的差别就是：前者你记住了概念，后者你真的有了一个出了错能靠日志定位、改坏了能靠测试抓住、危险动作能靠审批拦住的 Agent。

回头一看：你已经用过哪些 Agent 概念

走完这 13 步，回头点一点你刚用过的概念。全书后面列过 23 个 Agent 零件，你在这个能跑的小 Agent 里其实已经亲手摸过其中一多半：System Prompt、Runtime、Tool Schema、Function Calling、Skill、Context Management、State、Memory、RAG、Planner、Executor、Evaluator、Sub-agent（派子任务）、Human-in-the-loop、Hooks（自动检查）、Sandbox（计算器沙箱/读文件限制）、Security（不写密钥、最小权限）、Observability（`agent.log`）、Evaluation（`eval.py`）、Deployment（怎么从“你电脑能跑”变成“日常能用”）……

剩下没在本项目里展开的——Plugin（比 Skill 更平台绑定）、MCP Server（把你自己的系统按标准暴露给任何支持 MCP 的 Agent，写一次到处能用）、API 设计——知道它们解决什么问题、什么时候需要，就够了，不必每个都自己实现。

零基础怎么迈第一步

正确顺序是反过来的：先挑一个现成平台或开源框架，把别人做好的 Runtime、工具调用、记忆管理直接拿来用；你真正要花心思的只有三样——写清楚 System Prompt（这个 Agent 到底服务什么、怎么干）、接好它要读的资料和要调的工具、设计好“怎么算干得好”的测试。框架会换、模型会升级，这三样不会变。

判断你有没有学会，标准很朴素：一个不懂技术的同学，拿着你写的说明书，能不能照着装好、跑起来、出一次合格结果；出了错，你能不能靠日志说出它错在哪一步。

毕业项目

F31 学习路径地图：从「会聊天」到「构建自己的 Agent」

能力进阶（每一级都在前一级可用的前提下才成立）

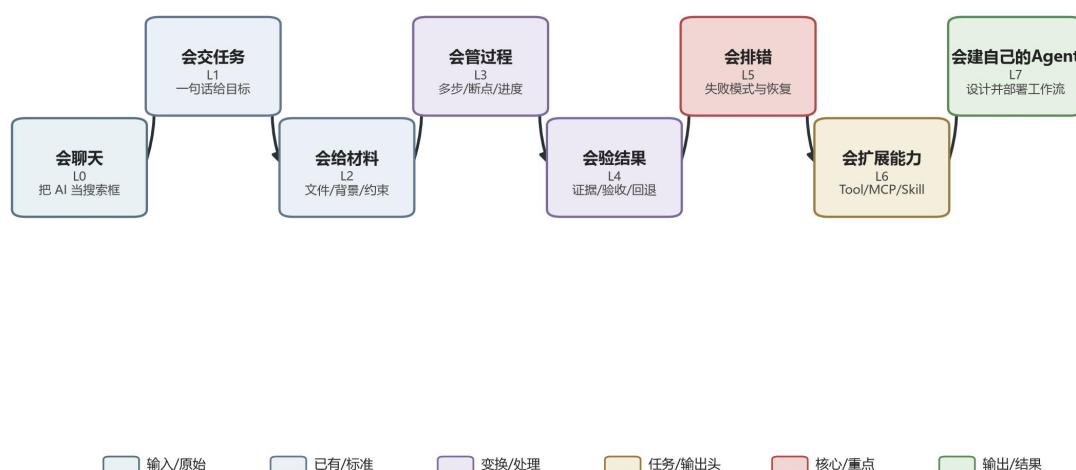


图 33 学习路径地图

走到这里，毕业项目是一件事：**设计、实现、测试并交付一个面向你自己专业的 Agent。**交付物不是“能跑”四个字，而是六件套：

1. **真实任务**：一个你本专业每周都做、做得烦的活（文献研究、案例梳理、史料整理、宏观数据跟踪、调研、财务分析、课程学习……），别选“听起来酷但你自己不会用”的题。
2. **可运行 Agent**：按 8 要素把岗位书写清，配好它要读的资料和要调的工具，按 Planner–Executor–Evaluator 串起来。
3. **测试集**：十到二十个测试用例。
4. **Eval 报告**：每次改完用同一套用例跑，记录哪些过、哪些漏、哪些胡说，附回归对比。
5. **风险说明**：一份“它做不到什么、什么情况必须你来”的诚实清单——哪些动作必须人工批、它的边界在哪。

6. 真实用户试用：找一个真实的人（同学、同事）真用一次，拿回反馈。

最后记全书的结论：**Agent** 的价值不在于它会说多少，而在于它在明确的目标、合适的工具、可控的权限和可靠的验证下，持续完成真实工作。你造出的这个 **Agent**，能不能让你下周还愿意用它、敢把真事交给它，就是毕业与否的唯一标准。

附录 A 术语表

英文	中文译法	一句话解释
Agent	智能体 / 代理	能自主规划并调用工具完成任务的 AI 程序
Chatbot	聊天机器人	只能对话、不能操作外部工具的 AI
Copilot	副驾驶	辅助人完成工作但不自主决策的 AI
Automation	自动化	按预设规则自动执行重复任务
Autonomous Workflow	自主 workflow	Agent 自主决定步骤并执行的流程
Prompt	提示词 / 提示	给 AI 的指令文本
Context	上下文	Agent 当前能看到的所有信息
Context Window	上下文窗口	一次对话能容纳的最大信息量
Memory	记忆	Agent 跨对话保留信息的能力
Tool	工具	Agent 可以调用的外部能力
Tool Calling	工具调用	Agent 决定调用某个工具的动作
Search	搜索	Agent 联网查找信息
Browser	浏览器	Agent 像人一样操作网页
Computer Use	电脑操作	Agent 直接操控鼠标键盘操作电脑
File System	文件系统	计算机的文件夹和文件管理
Terminal / Shell	终端 / 命令行	用文字命令操作计算机的界面
Code Execution	代码执行	Agent 运行程序代码来完成计算
API	应用程序接口	软件之间互相调用功能的标准化接口
Connector	连接器	连接外部应用（如邮箱、日历）的组件
Plugin	插件	给 ChatGPT 等平台扩展功能的模块
MCP	模型上下文协议	Model Context Protocol，让 Agent 连接外部工具和数据的标准协议
Skill	技能	一组可复用的指令+模板+脚本，教 Agent 完成某类任务

英文	中文译法	一句话解释
SKILL.md	技能说明文件	描述一个 Skill 的名称、用途、用法的说明文件
RAG	检索增强生成	先从知识库检索相关资料，再让 AI 基于资料回答
Workflow	工作流	按预设步骤串联的自动化流程
Sub-agent	子智能体	主 Agent 派生出来处理子任务的独立 Agent
Goal	目标	最终要得到什么
Context (工作框架)	上下文	Agent 为完成任务需要知道的背景
Resources	资源	文件、网页、数据、代码等可用的材料
Capabilities	能力	Agent 可调用的工具和功能
Verification	验证	证明工作真的完成且结果可信
Reuse	沉淀/复用	把成熟流程保存为模板、Skill 或自动化
Milestone	里程碑	长任务中的关键完成节点
Checkpoint	检查点	中途暂停让用户确认的节点
Hallucination	幻觉	AI 编造不存在的事实
Git	版本控制工具	记录文件修改历史、可以回退的工具
GitHub	代码托管平台	存放和分享代码的网站
Repository / Repo	代码仓库	GitHub 上的一个项目文件夹
Clone	克隆	把 GitHub 项目下载到本地
Commit	提交	保存一次代码修改记录
Branch	分支	独立修改代码的平行工作线
Push	推送	把本地修改上传到 GitHub
Pull Request	合并请求	请求把分支修改合并到主分支
Dependency	依赖	项目运行需要的其他软件包
OAuth	开放授权	一种让第三方应用安全登录你的账号的协议
API Key	应用程序密钥	调用 API 时证明身份的一串密码
Environment Variable	环境变量	存储配置信息的系统级变量
Sandbox	沙箱	隔离的安全运行环境
RPA	机器人流程自动化	传统的模拟人工操作软件的自动化技术

英文	中文译法	一句话解释
Webhook	网络钩子	事件发生时自动通知某个网址的机制
Planner	规划器	把大任务分解成小步骤的组件
Executor	执行器	实际执行步骤的组件
Evaluator	评估器	检查结果是否合格的组件
Human-in-the-loop	人在回路	关键步骤需要人确认后才继续
Prompt Engineering	提示工程	设计有效提示词的方法
AGENTS.md	Agent 规则文件	项目根目录下给 Agent 读的说明文件

附录 B 资源地图精选

以下资源按 Tier 分级（A=官方基础、B=社区发现平台、C=专业工具集成、D=社区实验），

优先选取官方文档、高价值发现平台和常用工具集成。筛选规则：Tier A 全部保留；Tier B 每个类别选 2-3 个最常用的；

Tier C 选与教程章节直接相关的；Tier D 仅选有明确学习价值的。完整 98 条索引见 01_调研/02_Agent 资源索引.csv。

所有资源核验日期：2026-09-18。

资源名称	类型	来源	用途	等级
Anthropic Agent Skills Standar	Standard	Official	SKILL.md open format spec (name/description frontm	A
Anthropic Skills Engineering B	Standard	Official	Official blog post announcing Agent Skills open st	A
MCP Official Specification	Standard	Official	MCP protocol spec (JSON-RPC 2.0, Host/Client/Serve	A
MCP Official Registry	Registry	Official	Official MCP server metadata registry with DNS-ver	A
MCP Blog / Roadmap	Standard	Official	MCP roadmap and release notes	A
Claude Code Docs (Plugins)	Docs	Official	Claude Code plugin/skill/hook package format docs	A
Claude Code Plugin Marketplace	Docs	Official	How to create/distribute a plugin marketplace JSON	A
OpenAI Codex (github.com/opena	Coding Agent	Official	Open-source terminal coding agent; AGENTS.md conve	A
OpenAI Agents API (Codex harne	API	Official	Cloud agents with Codex harness, fully managed	A
Gemini CLI Docs	Docs	Official	Gemini CLI extensions, skills, GEMINI.md, subagent	A
GitHub Copilot Agent Skills Do	Docs	Official	VS Code / Visual Studio / Copilot CLI Agent Skills	A
GitHub Copilot CLI Changelog	Docs	Official	Copilot CLI releases: plugins marketplace, MCP, ho	A
Microsoft Agent Framework	SDK	Official	Unified SDK merging AutoGen + Semantic Kernel	A

资源名称	类型	来源	用途	等级
OpenAI Plugin Directory	Directory	Official	Replaces App Directory (2026-07-09); packages skil	A
GitHub MCP Server	MCP	Official	Official GitHub MCP server; secret scanning GA	A
anthropics/skills	Skill Repo	Official	Official Anthropic skills: doc skills + examples (	A
openai/plugins (legacy)	Plugin Repo	Official	Codex plugin examples repo (replaces deprecated op	A
huggingface/skills	Skill Repo	Official	Official HF skills: hf-cli, model-trainer, best-mo	A
microsoft/skills	Skill Repo	Official	Microsoft official skills incl. Skills for Fabric	A
NVIDIA/skills	Skill Repo	Official	NVIDIA official skills	A
vercel-labs/skills	Skill Repo	Official	Source repo for skills.sh; 900K+ weekly installs	A
github/copilot-plugins	Plugin Repo	Official	Official Copilot plugins marketplace incl. advance	A
iflytek/skillhub (讯飞)	Skill Registry	Official	Enterprise self-hosted skill registry; RBAC, audit	A
Astron SkillHub (在线实例)	Skill Registry	Official	Hosted instance of iflytek SkillHub	A
ModelScope MCP Marketplace	MCP Directory	Official	中文 MCP 市场 (~5.4K servers)	A
Docker MCP Catalog	MCP Catalog	Official	Containerized, security-scanned MCP server catalog	A
Work IQ MCP (Microsoft)	MCP	Official	Microsoft 365 work context via MCP	A
Microsoft Power Automate	Automation Plat	Official	RPA + cloud flows; desktop automation; Microsoft 3	A
Claude Code	Coding Agent	Official	Terminal coding agent; plugins/skills/hooks/MCP	A
OpenAI Codex	Coding Agent	Official	Terminal + cloud coding agent; AGENTS.md	A
Gemini CLI	Coding Agent	Official	Open-source terminal agent; extensions/skills	A
GitHub Copilot CLI	Coding Agent	Official	Terminal coding agent; plugins/MCP/hooks	A
Antigravity	Coding Agent	Official	Google AI IDE; listed by skills.sh	A
Kiro CLI	Coding Agent	Official	AWS terminal coding agent; listed by skills.sh	A
OpenAI Agents SDK	SDK	Official	Multi-agent workflow SDK; handoffs, guardrails, tr	A

资源名称	类型	来源	用途	等级
Google ADK	SDK	Official	Google Agent Development Kit; GCP-native	A
SWE-bench	Benchmark	Academic	Solve real GitHub issues; Verified/Lite/Pro varian	A
tau2-bench	Benchmark	Academic	Multi-turn tool-use; retail/airline/telecom; pass^	A
Berkeley Function-Calling Lead	Leaderboard	Academic	BFCL V4 function calling AST evaluation	A
OSWorld	Benchmark	Academic	Real desktop GUI automation (Ubuntu/Win/macOS)	A
WebArena	Benchmark	Academic	Sandboxed web agent; 812 templated tasks	A
GAIA	Benchmark	Academic	General AI assistant; Levels 1/2/3	A
Terminal-Bench	Benchmark	Academic	Terminal coding benchmark	A
SkillsMP	Directory	Communit	Aggregates 2.39M+ public SKILL.md files for search	B
SkillHub.club	Directory	Communit	AI-evaluated skill marketplace; 115.3K skills, 65	B
AgentSkillsHub (agentskillshub)	Directory	Communit	Security-graded MCP/skill directory; 156K+ repos;	B
skills.sh (Vercel)	Directory+CLI	Communit	Agent Skills directory + leaderboard; npx skills a	B
agentskills.codes	Directory+CLI	Communit	Open registry of 19,296 installable skills; daily	B
SkillMD.ai	Docs+Directory	Communit	SKILL.md authoring guides and resource-scout meta-	B
agentskill.sh	Plugin Marketpl	Communit	Claude Code plugin marketplace; /learn self-improv	B
sickn33/antigravity-awesome-sk	Awesome List	Communit	1400+ skills registry-style; CLI; renamed from ant	B
Glama	MCP Directory	Communit	MCP server registry superset: 88K servers, maintai	B
PulseMCP	MCP Directory	Communit	Daily hand-curated MCP server directory; ~12K serv	B
mcp.so	MCP Directory	Communit	Community third-party MCP server collection	B
Smithery	MCP Platform	Communit	Discover/deploy/manage MCP servers; CLI, hosted re	B

资源名称	类型	来源	用途	等级
Pipedream MCP	MCP Platform	Communit	Exposes thousands of SaaS app actions as MCP tools	C
Composio	Integration Pla	Communit	AI-native agent integration; 850+ connectors; MCP	C
Composio Connect	MCP Gateway	Communit	Single MCP connection to 1000+ apps via 7 meta-too	C
Zapier	Automation Plat	Commerci	9000+ app automation; Zapier MCP exposes actions t	C
getAbstract MCP	MCP	Commerci	Semantic search/Q&A over getAbstract book summarie	C
n8n	Automation Plat	Communit	Workflow automation + AI agents; 500+ integrations	C
Make	Automation Plat	Commerci	Visual scenario builder; built-in AI Agents	C
Dify	Agent Builder	Communit	Open-source LLM app/agent platform; visual workflow	C
Coze (扣子)	Agent Builder	Official	ByteDance bot/agent builder; visual orchestration,	C
Langflow	Agent Builder	Communit	Low-code visual agent/MCP builder; LangChain ecosy	C
openai/skills (DEPRECATED)	Skill Repo	Official	DEPRECATED 2026-06-22; moved to Plugin Directory	D
travisvn/awesome-claude-skills	Awesome List	Communit	Curated Claude Code skills list by category	D
ComposioHQ/awesome-claude-skil	Awesome List	Communit	Large community skills list (~66K stars)	D
hesreallyhim/awesome-claude-co	Awesome List	Communit	Broad Claude Code eco: skills, hooks, slash comman	D
VoltAgent/awesome-agent-skills	Awesome List	Communit	1000+ cross-platform skills (Claude Code, Codex, G	D

附录 C 8 要素任务法模板

每次给 Agent 下任务前，按以下 8 要素逐项写清楚。复制此模板，填空即可。

要素	英文	要回答的问题	填写示例
目标	Goal	最终要得到什么？（有数量、有范围、有形式）	周末前把第三章整理成知识结构图，覆盖全部小节
材料	Materials	可以用什么？（材料在哪、哪些必读）	材料在 D:\课程\论文\文件夹，01.pdf 必读
环境	Environment	在哪里工作？（圈定活动范围）	只在 D:\课程\论文\文件夹内工作
权限	Permissions	可以做什么？（读/写/删/对外发送）	可读可写，不可删原始文件，不可对外发送
约束	Constraints	什么不能做？（红线）	不得编造文献；不得覆盖原始文件；中文写作
过程	Process	按什么步骤做？	先盘点文件→提取要点→填表→自检→交付
验收	Acceptance	怎么证明做完了？	知识结构图覆盖全部小节，每小节一句话
交付	Deliverables	最后交出什么？	一份 .md 文件 + 一张结构图 .png

使用方法：把这个模板复制到你的任务说明开头，逐项填写。8 项写齐了再发给 Agent，不要只写“帮我做 XX”。